

UNIVERSITY OF OKARA

Department of Computer Science

DATA STRUCTURES & ALGORITHMS

Lab Manual

LAB 01 | Python Foundations for DSA & Your Developer Toolkit

Week 1 of 16 • Duration: 3 hours (in-lab) + home practice

Prepared by

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

Applicable to: BSCS • BSIT • BSSE • BSAI • BSDS • BS Cyber Security and all programs offering DSA

Fall 2026 Edition

About This Lab Manual

This manual is part of a **16-week Data Structures & Algorithms (DSA) lab series**. It is designed so that a student starting with **zero programming knowledge** in Week 1 can confidently design, implement and analyse data structures by Week 16. Every lab builds on the previous one, so please do not skip labs.

Language policy: concepts are explained in a language-independent way (diagrams, pseudocode, dry runs). Full implementations are in **Python**, and every lab includes a **C++ Corner** so instructors teaching in C++ can use the same manual.

Every lab follows the same structure

Section	What you get
 Objectives & CLOs	What you will be able to do after this lab
 Motivation	Where this topic is used in real life, industry, research and the future (2026–2030)
 Concepts	Simple explanations with real examples, pseudocode and dry runs
 Code	Python implementation + C++ Corner
 Lab Tasks	Graded tasks:  Easy,  Medium,  Hard
 LeetCode	Real interview problems to practise
 Home Task &  Viva	Practice at home and prepare for oral questions
 Notebooks	Instructor notebook (complete code) and Student notebook (with TODO blanks and auto-check tests)

Lab 01 at a glance

Item	Details
Lab title	Python Foundations for DSA & Your Developer Toolkit
Week	1 of 16
Prerequisites	None. This lab starts from zero.
Tools	Google Colab (or Jupyter / VS Code), GitHub account, LeetCode account
Files	Lab01_Instructor.ipynb , Lab01_Student.ipynb
Next lab	Lab 02: Complexity Analysis (Big-O)

1. Learning Objectives

By the end of this lab, you will be able to:

1. Set up a complete developer toolkit: **Google Colab, GitHub and LeetCode**.
2. Explain in simple words what a **data structure** and an **algorithm** are, with real-life examples.
3. List and check the **five properties** of a correct algorithm.
4. Write Python programs using **variables, data types, input/output, conditions, loops and functions**.
5. Use Python's built-in collections: **list, tuple, dictionary and set**.
6. Understand how Python stores data in memory (**references, mutable vs immutable**), which is essential for linked lists and trees later.
7. Solve a problem using the professional workflow: **Understand** → **Pseudocode** → **Dry Run** → **Code** → **Test**.

Course Learning Outcomes (CLO) mapping

CLO	Description	Taxonomy level
CLO-1	Explain basic data structures and algorithm properties	Understand (C2)
CLO-2	Implement solutions to simple problems in a programming language	Apply (C3)
CLO-4	Use professional tools (version control, online judges)	Apply (P3)

2. Motivation: Why Should You Learn DSA?

Imagine you open Daraz and type “wireless earbuds”. Within a fraction of a second, you see the best results out of **millions of products**. How? Not because the computer is magic, but because engineers chose the **right way to store the data** (data structure) and the **right steps to search it** (algorithm). That is exactly what this course teaches you.

2.1 DSA in Your Daily Life

App you use	What happens behind the scenes	DSA used
Google Maps / Careem / inDrive	Finds the shortest route from your home to the university	Graphs, Dijkstra's algorithm, priority queue
WhatsApp	Shows recent chats on top; delivers messages in order	Queues, linked lists, hashing

App you use	What happens behind the scenes	DSA used
Daraz / Amazon search	Finds and ranks products instantly	Hash tables, trees, sorting
Ctrl + Z (Undo) in MS Word	Remembers your last actions in reverse order	Stack
Browser Back / Forward	Moves between visited pages	Doubly linked list / two stacks
Spotify / YouTube playlists	Next and previous song	Linked list
NADRA / bank records	Finds your record among crores of people using CNIC	Hashing, B-trees (databases)



2.2 DSA in Industry and Jobs

- Technical interviews at Google, Microsoft, Amazon and Meta, and at Pakistani software companies such as **Systems Limited, Arbisoft, 10Pearls, NetSol and Folio3**, are largely based on DSA problems.
- Platforms like **LeetCode, HackerRank and Codeforces** are used by companies to shortlist candidates. You will start using LeetCode from today.
- Freelancers and startup developers who understand DSA write software that stays fast when users grow from 100 to 1,000,000.



2.3 DSA in Research

- **Artificial Intelligence:** neural networks are built on arrays and matrices (tensors); search algorithms power game-playing AI and route planning in robotics.
- **Vector databases** used by ChatGPT-style systems rely on special graphs (e.g., HNSW) to search millions of embeddings quickly.
- **Bioinformatics:** DNA sequences are searched using string algorithms and hashing.
- **Remote Sensing & GIS:** satellite images are 2D/3D arrays; spatial data uses trees such as quadtrees and R-trees.

2.4 Why DSA Matters Even More in 2026–2030

The AI era does not replace DSA, it raises the bar

AI coding assistants can now write code in seconds. So what will make **you** valuable? The ability to **judge** whether that code is correct, efficient and scalable. Only someone who understands DSA can tell when AI-generated code will crash with 10 million users or waste server money.

- Companies will hire people who can **design solutions**, not just type code.
- Big data, AI and cloud systems all run on efficient data structures.
- DSA trains your **problem-solving mind**, a skill that no tool can replace.

3. Part A: Setting Up Your Developer Toolkit

Professional developers do not just write code; they **save, share and test** it. In this part you will create three accounts that you will use for the whole semester and beyond.

3.1 Google Colab: Run Python Without Installing Anything

1. Open **colab.research.google.com** and sign in with your Gmail account.
2. Click **File** → **Upload notebook** and select **Lab01_Student.ipynb** (provided by your instructor).
3. A notebook has two kinds of cells: **Text cells** (explanations) and **Code cells** (Python code).
4. Click inside a code cell and press **Shift + Enter** to run it. The output appears below the cell.
5. Save a copy to your Google Drive: **File** → **Save a copy in Drive**.

Note

You may also use **Jupyter Notebook** (via Anaconda) or **VS Code** with the Python and Jupyter extensions installed on your laptop. All lab notebooks work in all three.

3.2 GitHub: Your Professional Portfolio

GitHub is where developers store their code online. Recruiters often check a candidate's GitHub profile before an interview. By the end of this course, your GitHub will show **16 completed DSA labs**, which is real proof of your skills.

1. Go to **github.com** and sign up with a professional username (e.g., **ali-raza-dev**, not **cool_boy_786**).
2. Click **New repository**, name it **DSA-Labs**, choose **Public**, tick **Add a README file**, and click **Create**.
3. Inside the repository, click **Add file** → **Create new file** and type **Lab01/README.md** to create a folder for this lab.
4. After completing the lab, upload your notebook: **Add file** → **Upload files**, then commit.
5. In Colab you can also use **File** → **Save a copy in GitHub**.

3.3 LeetCode: Practise Like a Professional

1. Sign up at **leetcode.com**.
2. Open **Problems**, filter by **Easy**, and solve the problems listed in the LeetCode section of this manual.
3. Aim for **at least 2 problems per week**. By Week 16 you should have 30+ solved problems.

4. Part B: What Are Data Structures and Algorithms?

4.1 Data Structure

A **data structure** is a way of **storing and organising data** in a computer so that we can use it efficiently.



Real-life analogy: a general store

Think of a well-run general store. Rice, sugar and flour are kept on one shelf, soaps on another, and cold drinks in the fridge. When a customer asks for something, the shopkeeper finds it in seconds. If everything were thrown into one big pile, finding a packet of salt would take forever.

The items are the data. The way they are arranged is the data structure.
The right arrangement makes work fast.

Data structures are commonly divided into these groups:

Type	Meaning	Examples
Primitive	Basic building blocks provided by the language; store a single value	int, float, char, bool
Non-primitive (Linear)	Elements are arranged one after another in a sequence	Array, Linked List, Stack, Queue

Type	Meaning	Examples
Non-primitive (Non-linear)	Elements are arranged in a hierarchy or network	Tree, Heap, Graph
Hash-based	Elements are found directly using a key	Hash Table (Python dict, set)

4.2 Algorithm

An **algorithm** is a **finite sequence of clear steps** that solves a problem. It is the logic, not the code. The same algorithm can be written in Python, C++ or Java.

Real-life analogy: a recipe

A recipe for making tea is an algorithm: boil water, add tea leaves, add milk and sugar, boil again, pour into cup. The steps are clear, ordered and they finish.

Five properties every algorithm must have

Property	Meaning	Tea example
Input	Zero or more values given from outside	Water, milk, tea leaves, sugar
Output	At least one result is produced	A cup of tea
Definiteness	Every step is clear, with only one meaning	“Boil for 2 minutes”, not “boil for a while”
Finiteness	It must stop after a limited number of steps	The process ends when tea is poured
Correctness	It gives the right output for every valid input	Always produces tea, not coffee

4.3 Data Structure + Algorithm = Program

Niklaus Wirth, a famous computer scientist, wrote a book titled **Algorithms + Data Structures = Programs**. Every software you use is a combination of the two. Choosing a good data structure often makes the algorithm simple and fast.

4.4 Worked Example: Find the Highest Marks

Problem: A teacher has the marks of students: 67, 89, 45, 92, 78. Find the highest marks.

Step 1: Pseudocode (language-independent)

ALGORITHM FindMax(marks)

INPUT : a list of numbers called marks

OUTPUT: the largest number

1. max_value ← marks[0]
2. FOR each value in marks DO
3. IF value > max_value THEN
4. max_value ← value
5. RETURN max_value

Step 2: Dry run (trace by hand)

Step	value	value > max_value ?	max_value
Start	-	-	67
1	67	67 > 67 → No	67
2	89	89 > 67 → Yes	89
3	45	45 > 89 → No	89
4	92	92 > 89 → Yes	92
5	78	78 > 92 → No	92

Step 3: Python code

```
def find_max(marks):
    max_value = marks[0]      # step 1: assume first is largest
    for value in marks:      # step 2: check every value
        if value > max_value: # step 3: found a bigger one?
            max_value = value # step 4: remember it
    return max_value         # step 5: answer

print(find_max([67, 89, 45, 92, 78])) # Output: 92
```

The notebook also prints the dry run automatically, so you can compare it with your hand trace:

Automatic dry run

```
def find_max_trace(marks):
    max_value = marks[0]
    print(f"Start: max_value = {max_value}")
    for step, value in enumerate(marks, start=1):
        bigger = value > max_value
        if bigger:
            max_value = value
        ans = "Yes" if bigger else "No"
        print(f"Step {step}: value={value:>3} | bigger? {ans:3} |
max={max_value}")
    return max_value

find_max_trace([67, 89, 45, 92, 78])
```

 Check the five properties

Input: the list of marks. Output: 92. Definite: each line has one meaning.
Finite: the loop ends after the last item. Correct: it works for any non-empty list.

Think: what happens if the list is empty? (You will fix this in Task 3.)



5. Part C: Python Essentials for DSA

This part covers only the Python you need for DSA. Run every example in your notebook and change the values to see what happens. **Learning to code is like learning to swim: you must get into the water.**

5.1 Variables and Data Types

A **variable** is a name that refers to a value stored in memory. Python decides the type automatically.

```
name = "Ayesha"      # str   (text)
age = 19              # int   (whole number)
cgpa = 3.45           # float (decimal number)
is_enrolled = True    # bool  (True / False)

print(type(name), type(age), type(cgpa), type(is_enrolled))
```

5.2 Input and Output

```
student = input("Enter your name: ")
marks = int(input("Enter your marks: ")) # input() always gives text,
                                         # so convert with int()
print(f"{student} scored {marks} marks.") # f-string formatting
```

Operator	Meaning	Example → Result
+ - * /	Add, subtract, multiply, divide	7 / 2 → 3.5
//	Integer (floor) division	7 // 2 → 3
%	Remainder (modulus)	7 % 2 → 1 (used to check even/odd)
**	Power	2 ** 10 → 1024
== != < > <= >=	Comparison	5 > 3 → True
and or not	Logical	True and False → False

Operators in action

```
print(7 / 2, 7 // 2, 7 % 2, 2 ** 10)
n = 14
print(n, "is", "even" if n % 2 == 0 else "odd")
```

5.3 Conditions: if / elif / else

```
marks = 76
if marks >= 85:
    grade = "A"
elif marks >= 70:
    grade = "B"
elif marks >= 50:
    grade = "C"
else:
    grade = "F"
print("Grade:", grade)          # Grade: B
```

Indentation matters

Python uses **4 spaces** of indentation to group code. In C++ we use curly braces { }. Wrong indentation causes an **IndentationError** or wrong logic.

5.4 Loops: for and while

```
# for loop: repeat for each item
for i in range(1, 6):          # 1, 2, 3, 4, 5 (6 is excluded)
    print(i, "squared =", i * i)

# while loop: repeat while a condition is True
balance = 1000
months = 0
while balance < 2000:
    balance = balance * 1.10    # 10% profit per month
    months += 1
print("Months needed:", months) # 8
```

5.5 Functions: Write Once, Use Many Times

```
def calculate_bill(price, quantity, discount=0):
    """Return total bill after discount (in percent)."""
    total = price * quantity
    return total - (total * discount / 100)

print(calculate_bill(250, 4))      # 1000.0
print(calculate_bill(250, 4, 10))  # 900.0
```

Functions are the heart of DSA. Every operation we will build (insert, delete, search, sort) will be a function.

5.6 Python's Built-in Collections

Python gives us four ready-made collections. Each one is actually a data structure, and in later labs you will learn **how they work inside** and build your own.

Collection	Syntax	Ordered ?	Changeable?	Duplicates?	Studied in
List	[1, 2, 3]	Yes	Yes	Yes	Lab 03 (Arrays)
Tuple	(1, 2, 3)	Yes	No	Yes	Lab 03
Dictionary	{"a": 1}	Yes*	Yes	Keys: No	Lab 12 (Hashing)
Set	{1, 2, 3}	No	Yes	No	Lab 12 (Hashing)

* Since Python 3.7, dictionaries remember insertion order.

List: the most-used collection

```
marks = [67, 89, 45, 92, 78]
print(marks[0])      # 67   first element (index starts at 0)
print(marks[-1])     # 78   last element
print(marks[1:4])    # [89, 45, 92]   slicing
marks.append(55)      # add at end
marks.insert(0, 100)  # add at index 0
marks.remove(45)      # remove first 45
print(len(marks), max(marks), sum(marks))
```

Tuple: fixed data

```
point = (31.5204, 74.3587)    # (latitude, longitude): a fixed pair
print(point[0])
try:
    point[0] = 0               # tuples cannot be changed
except TypeError as e:
    print("Error:", e)
```

Dictionary: key → value pairs

```
prices = {"sugar": 160, "rice": 320, "milk": 220}
print(prices["rice"])        # 320   (very fast lookup by key)
prices["tea"] = 1450         # add new item
for item, price in prices.items():
    print(item, "→", price)
print(prices.get("honey", "not available"))  # safe lookup
```

Set: unique items only

```
roll_numbers = [12, 15, 12, 18, 15, 20]
unique = set(roll_numbers)
print(unique)           # {12, 15, 18, 20}
print(15 in unique)     # True (very fast membership check)
```

List comprehension: a short way to build lists

```
squares = [x * x for x in range(1, 6)]           # [1, 4, 9, 16, 25]
passed = [m for m in [67, 45, 92, 30] if m >= 50] # [67, 92]
print(squares, passed)
```

5.7 Strings

```
word = "Okara"
print(word.upper(), word.lower(), len(word)) # OKARA okara 5
print(word[::-1])                          # arakO (reverse)
for ch in word:
    print(ch, end=" ")                      # O k a r a
```

5.8 How Python Stores Data in Memory (Very Important for DSA)

In Python, a variable does **not** contain the value directly. It is a **label (reference)** pointing to an object in memory. Understanding this now will make **linked lists**, **trees** and **graphs** much easier later, because those structures are built entirely from references.

```
a = [1, 2, 3]
b = a           # b points to the SAME list, not a copy
b.append(4)
print(a)        # [1, 2, 3, 4] a also changed!
print(a is b)   # True: same object in memory

c = a.copy()    # a real copy
c.append(5)
print(a)        # [1, 2, 3, 4] a is safe
```

Mutable (can change in place)	Immutable (cannot change)
list, dict, set	int, float, str, tuple, bool

Connection to future labs

In Lab 08, each node of a linked list will hold a value and a **reference** to the next node, exactly like `b = a` above. If you understand this example, you already understand the core idea of a linked list.

6. C++ Corner

For instructors and students working in C++, here is the same Find-Max algorithm. Notice that the **algorithm (logic) is identical**; only the syntax changes.

C++

```
#include <iostream>
#include <vector>
using namespace std;

int findMax(const vector<int>& marks) {
    int maxValue = marks[0];
    for (int value : marks) {
        if (value > maxValue)
            maxValue = value;
    }
    return maxValue;
}

int main() {
    vector<int> marks = {67, 89, 45, 92, 78};
    cout << "Highest marks: " << findMax(marks) << endl;    // 92
    return 0;
}
```

Concept	Python	C++
Variable type	Decided automatically	Must be declared: int, float, string
Code blocks	Indentation	Curly braces { }
Dynamic list	list	vector<int>
Key-value store	dict	unordered_map<string,int>
Unique items	set	unordered_set<int>
Print	print(x)	cout << x << endl;

7. Common Mistakes Beginners Make

Mistake	Example	Fix
Forgetting to convert input	<code>age = input()</code> then <code>age + 1</code> → error	<code>age = int(input())</code>
Index starts at 1 (wrong)	<code>marks[5]</code> on a 5-item list → <code>IndexError</code>	Valid indexes are 0 to <code>len-1</code>
Using <code>=</code> instead of <code>==</code>	<code>if x = 5:</code>	<code>if x == 5:</code>
Wrong indentation	Mixing tabs and spaces	Always use 4 spaces
Copying a list with <code>=</code>	<code>b = a</code> changes both	<code>b = a.copy()</code>
Not testing edge cases	Code fails on an empty list	Test empty, single-item and large inputs

8. Lab Tasks

Solve all tasks in `Lab01_Student.ipynb`. Each task has a **test cell**: when it prints  your solution is correct. For each task, **write pseudocode first**, then code, then test with your own inputs including edge cases. Difficulty:  Easy  Medium  Hard.

Task 1 Student Result Card

Write `result_card(name, marks)` that takes a name and a list of 5 subject marks (each out of 100), prints the total, percentage and grade ($A \geq 85$, $B \geq 70$, $C \geq 50$, otherwise F), and **returns** (`total, percentage, grade`).

```
Input : result_card("Hamza", [78, 85, 90, 66, 72])
Output: Hamza | Total: 391/500 | Percentage: 78.2% | Grade: B
```

Task 2 Even or Odd Counter

Given a list of integers, return (`even_count, odd_count`) using the `%` operator.

```
Input : [12, 7, 9, 20, 33, 18]
Output: (3, 3)
```

Task 3 Safe Find-Max

Improve `find_max` so that it returns `None` for an empty list instead of crashing. **Do not** use Python's built-in `max()`.

Task 4 General Store Bill

A store keeps prices in a dictionary. Write `make_bill(prices, cart)` where `cart` is a dictionary of item $\rightarrow$ quantity. Print each line and the totals. If the subtotal is above Rs. 5,000, apply a 5% discount. If an item is not in the price list, print a warning and skip it. **Return** the final total.

```
prices = {"sugar": 160, "rice": 320, "oil": 560, "tea": 1450}
cart    = {"rice": 5, "oil": 3, "tea": 2, "honey": 1}
rice    x 5 = 1600
oil     x 3 = 1680
tea     x 2 = 2900
Warning: 'honey' not found
Subtotal: 6180 | Discount: 309.0 | Total: 5871.0
```

Task 5 Palindrome Checker

A palindrome reads the same forward and backward (e.g., “madam”, “Never odd or even”). Ignore spaces and capital letters.

Solve it **twice**: once using slicing `[::-1]`, and once with a loop using two indexes (one from the start, one from the end). The second method is the **two-pointer technique**, which you will use again in Lab 04.

Task 6 🟡 Character Frequency

Write `char_frequency(text)` that returns a dictionary showing how many times each character appears (ignore spaces). Then print the most frequent character.

```
Input : "data structures"
Output: {'d': 1, 'a': 2, 't': 3, 's': 2, 'r': 2, 'u': 2, 'c': 1, 'e': 1}
Most frequent: 't' (3 times)
```

Task 7 🔴 Second Highest Marks

Find the **second highest unique** value in a list in **one pass** (a single loop), without sorting and without `max()`. Return `None` if it does not exist.

```
[67, 89, 45, 92, 78] → 89
[50, 50, 50]         → None
[10, 20, 20]         → 10
```

Task 8 🔴 Two Sum (Your First Interview Problem)

Given a list of numbers and a target, return the **indexes** (`i, j`) of two numbers that add up to the target (or `None`). First solve it with two nested loops. Then solve it using a **dictionary** in a single loop. Run the bonus experiment in the notebook to see the speed difference. You have just discovered why data structures matter, which is the topic of Lab 02!

```
two_sum([2, 7, 11, 15], 9) → (0, 1)    because 2 + 7 = 9
```

🏆 9. LeetCode Practice

Solve these on **leetcode.com** and upload screenshots of the “Accepted” result to your GitHub `DSA-Labs/Lab01` folder.

#	Problem	Level	Concept practised
1480	Running Sum of 1d Array	Easy	Loops, lists
1672	Richest Customer Wealth	Easy	Nested lists, sum
412	Fizz Buzz	Easy	Conditions, %
9	Palindrome Number	Easy	Two-pointer / reversal

#	Problem	Level	Concept practised
1929	Concatenation of Array	Easy	List operations
1	Two Sum	Easy	Dictionary (hash map), same as Task 8



10. Home Task

1. Complete any lab tasks you could not finish in the lab.
2. Create your **DSA-Labs** repository on GitHub and upload your completed **Lab01_Student.ipynb** in the **Lab01** folder.
3. Write a short **README.md** in the Lab01 folder listing what you learned in 3–5 bullet points.
4. **Mini-project (choose one):** (a) a **Contact Book** using a dictionary with add, search, update, delete and display options in a menu loop; or (b) a **Quiz Game** that stores questions and answers in a list of dictionaries and shows the final score.
5. Solve at least 2 of the LeetCode problems listed above.

? 11. Viva Questions

1. What is a data structure? Give one real-life example.
2. What is an algorithm? How is it different from a program?
3. Name the five properties of an algorithm and explain finiteness.
4. What is the difference between primitive and non-primitive data structures?
5. Differentiate between linear and non-linear data structures with examples.
6. What is the difference between a list and a tuple in Python?
7. Why is searching in a dictionary or set faster than in a list? (Hint: Lab 12)
8. What happens when you write **b = a** for a list? How do you make a real copy?
9. Which data structure is used for the Undo feature? Why?
10. Why is DSA still important when AI tools can write code?



12. Assessment Rubric

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Correctness	All tasks pass all tests	Most tasks correct	Some tasks correct	Mostly incorrect
Logic & pseudocode	Clear pseudocode and dry run for every task	Pseudocode for most tasks	Partial pseudocode	None
Code quality	Meaningful names, functions, comments	Mostly clean	Hard to read	Copied or unreadable
Complexity & edge cases	States Big-O and handles all edge cases	Some edge cases / Big-O	Only basic cases	Not considered
Viva	Explains every line confidently	Explains most parts	Basic understanding	Cannot explain



Lab Rules

- Bring your own laptop with Colab or Jupyter working.
- Arrive on time; late arrival may result in no attendance for the lab.
- Copying another student's code or pasting AI output without understanding it will be awarded zero. You must be able to explain every line in the viva.



13. References and Further Learning

- Official Python Tutorial: docs.python.org/3/tutorial
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, **Introduction to Algorithms** (CLRS), 4th ed., MIT Press, 2022.
- M. T. Goodrich, R. Tamassia, M. H. Goldwasser, **Data Structures and Algorithms in Python**, Wiley.
- MIT OpenCourseWare 6.006 Introduction to Algorithms; Harvard CS50x (free online courses).
- Visualise code step by step: pythontutor.com; animated data structures: visualgo.net.



About the Author

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

GitHub: github.com/RamzanPUCIT

LinkedIn: linkedin.com/in/ramzan-ai

ResearchGate: researchgate.net/profile/Muhammad-Ramzan-105

© 2026 Muhammad Ramzan. Prepared for the Department of Computer Science, University of Okara.

UNIVERSITY OF OKARA

Department of Computer Science

DATA STRUCTURES & ALGORITHMS

Lab Manual

LAB 02 | Complexity Analysis: Measuring Algorithm Efficiency (Big-O)

Week 2 of 16 • Duration: 3 hours (in-lab) + home practice

Prepared by

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

Applicable to: BSCS • BSIT • BSSE • BSAI • BSDS • BS Cyber Security and all programs offering DSA

Fall 2026 Edition

About This Lab Manual

This manual is part of a **16-week Data Structures & Algorithms (DSA) lab series**. It is designed so that a student starting with **zero programming knowledge** in Week 1 can confidently design, implement and analyse data structures by Week 16. Every lab builds on the previous one, so please do not skip labs.

Language policy: concepts are explained in a language-independent way (diagrams, pseudocode, dry runs). Full implementations are in **Python**, and every lab includes a **C++ Corner** so instructors teaching in C++ can use the same manual.

Every lab follows the same structure

Section	What you get
 Objectives & CLOs	What you will be able to do after this lab
 Motivation	Where this topic is used in real life, industry, research and the future (2026–2030)
 Concepts	Simple explanations with real examples, pseudocode and dry runs
 Code	Python implementation + C++ Corner
 Lab Tasks	Graded tasks:  Easy,  Medium,  Hard
 LeetCode	Real interview problems to practise
 Home Task &  Viva	Practice at home and prepare for oral questions
 Notebooks	Instructor notebook (complete code) and Student notebook (with TODO blanks and auto-check tests)

Lab 02 at a glance

Item	Details
Lab title	Complexity Analysis: Measuring Algorithm Efficiency (Big-O)
Week	2 of 16
Prerequisites	Lab 01 (Python basics, loops, functions, lists, dictionaries)
Tools	Google Colab / Jupyter / VS Code (Python 3); any C++ compiler for the C++ Corner
Files	Lab02_Instructor.ipynb , Lab02_Student.ipynb
Next lab	Lab 03: Arrays and Python Lists

1. Learning Objectives

By the end of this lab, you will be able to:

1. Explain why we measure algorithms by **number of steps** and not only by seconds.
2. Count the basic operations of simple code and express growth with **Big-O notation**.
3. Recognise the common complexity classes: **$O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^n)$** .
4. Analyse **best, average and worst case** of an algorithm.
5. Calculate the **space complexity** of a program.
6. Measure running time experimentally in Python and compare it with theory.
7. Choose the more efficient of two solutions for the same problem.

Course Learning Outcomes (CLO) mapping

CLO	Description	Taxonomy level
CLO-1	Explain time and space complexity and asymptotic notations	Understand (C2)
CLO-3	Analyse the efficiency of algorithms using Big-O	Analyse (C4)

2. Motivation: Why Does Efficiency Matter?

In Lab 01 you saw that the dictionary version of Two Sum was hundreds of times faster than the nested-loop version, even though both gave the same answer. Today you will learn the **language** professionals use to describe that difference: **complexity analysis** and **Big-O notation**.



A real story: the result website on result day

Every year when board or university results are announced, some result websites crash. Lakhs of students search for their roll number at the same time. If the website searches roll numbers one by one ($O(n)$), every request takes long and the server collapses. If it uses an index ($O(\log n)$) or a hash table ($O(1)$), it survives easily.

Same hardware, different algorithm, completely different outcome.

2.1 Complexity in Daily Life

Situation	Slow way	Fast way
Finding a word in a dictionary book	Read every page from the start: $O(n)$	Open the middle, go left or right: $O(\log n)$
Finding a contact in your phone	Scroll through all contacts: $O(n)$	Type the name, search index: $O(\log n)$ or $O(1)$
Checking attendance of 60 students	For each name, scan the whole list: $O(n^2)$	Mark roll numbers in a set: $O(n)$
Getting the first item in a queue	-	Always one step: $O(1)$

2.2 Complexity in Industry

- Every technical interview ends with the question: **“What is the time and space complexity of your solution?”** If you cannot answer, your solution is considered incomplete.
- Cloud companies (AWS, Azure, Google Cloud) charge by computing time. An $O(n^2)$ algorithm on big data can cost thousands of dollars more than an $O(n \log n)$ one.
- Mobile apps must be efficient to save battery and memory on low-end phones, which are very common in Pakistan.

2.3 Complexity in Research

- **AI:** the Transformer model behind ChatGPT has attention with $O(n^2)$ cost in sentence length. Much modern research (e.g., linear attention, FlashAttention) is about reducing this complexity.
- **Cryptography:** your bank's security depends on problems that take exponential time to break.
- **P vs NP**, the most famous open problem in computer science, is entirely about complexity classes.



2.4 Why It Matters in 2026–2030



Data keeps growing, so efficiency matters more every year

When n grows from 1,000 to 1,000,000,000, an $O(n)$ algorithm becomes 1 million times slower, but an $O(n^2)$ algorithm becomes **1 trillion times** slower.

As data grows with AI and IoT, choosing the right complexity class is the difference between software that works and software that never finishes.

AI tools often produce working but inefficient code. Your ability to say “this is $O(n^2)$, we can do $O(n)$ ” is a skill that keeps you ahead.



3. Part A: How Do We Measure an Algorithm?

3.1 Why Not Just Use a Stopwatch?

We could time a program in seconds, but the result depends on many things that have nothing to do with the algorithm:

- The speed of the computer (a new laptop vs an old lab PC).
- The programming language (C++ is usually faster than Python).
- Other programs running at the same time.
- The size and type of input.

So instead, we count **how many basic steps** the algorithm performs as a function of the **input size n** . This is machine-independent. A basic step is a simple operation: an assignment, a comparison, an arithmetic operation or accessing a list element.

3.2 Counting Steps: Worked Example

```
def total_marks(marks):          # n = len(marks)
    total = 0                    # 1 step
    for m in marks:              # loop runs n times
        total = total + m        # 1 step each time -> n steps
    return total                 # 1 step
# Total steps ≈ n + 2
print(total_marks([10, 20, 30]))
```

The exact count is $n + 2$. When n is very large (say 1 crore), the “+2” does not matter at all. What matters is that the work **grows in direct proportion to n** . We say this algorithm is **$O(n)$** , read as “order n ” or “Big-O of n ”.

3.3 Rules for Finding Big-O

Rule	Meaning	Example
1. Drop constants	Constant factors do not change the growth	$3n + 5 \rightarrow O(n)$
2. Keep the biggest term	For large n , the largest term dominates	$n^2 + 100n + 7 \rightarrow O(n^2)$
3. Sequential parts add	Two loops one after another	$O(n) + O(n) = O(n)$
4. Nested parts multiply	A loop inside a loop	$O(n) \times O(n) = O(n^2)$
5. Different inputs, different letters	Two lists of sizes a and b	Nested over both $\rightarrow O(a \times b)$



4. Part B: Common Complexity Classes

Big-O	Name	Example	$n = 1,000 \rightarrow$ steps
$O(1)$	Constant	Access <code>marks[5]</code> , dictionary lookup	1
$O(\log n)$	Logarithmic	Binary search (Lab 04)	≈ 10
$O(n)$	Linear	Find max, linear search	1,000
$O(n \log n)$	Linearithmic	Merge sort, quick sort (Lab 07)	$\approx 10,000$
$O(n^2)$	Quadratic	Bubble sort, checking all pairs	1,000,000
$O(2^n)$	Exponential	Naive recursive Fibonacci (Lab 06)	$\approx 10^{300}$ (never finishes)



Remember the order

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$

Everything up to $O(n \log n)$ is considered efficient for large data. $O(n^2)$ is fine only for small inputs. Exponential is practical only for tiny inputs.

4.1 O(1): Constant Time

```
def get_first(items):
    return items[0]          # one step, whether the list has 5 or 5 crore
                             items

prices = {"sugar": 160, "rice": 320}
print(get_first([4, 8, 15]), prices["rice"]) # dictionary lookup is O(1) on
average
```

4.2 O(n): Linear Time

```
def contains(items, target):
    for x in items:          # in the worst case we check all n items
        if x == target:
            return True
    return False

print(contains([5, 3, 9, 1], 9))
```

4.3 O(n²): Quadratic Time

```
def has_duplicate_slow(items):
    n = len(items)
    for i in range(n):      # n times
        for j in range(i + 1, n): # up to n times
            if items[i] == items[j]:
                return True
    return False

print(has_duplicate_slow([3, 1, 4, 1, 5]))
```

The inner loop runs $(n-1) + (n-2) + \dots + 1 = \mathbf{n(n-1)/2}$ times. After dropping constants and smaller terms, that is **O(n²)**.

4.4 O(log n): Logarithmic Time

An algorithm is O(log n) when it **cuts the problem in half** at every step. Starting with $n = 1,000,000$, halving gives $500,000 \rightarrow 250,000 \rightarrow \dots \rightarrow 1$ in only about **20 steps**, because $2^{20} \approx 1,000,000$.

```
def halving_steps(n):
    steps = 0
    while n > 1:
        n = n // 2          # problem size is cut in half
        steps += 1
    return steps

for n in [8, 1_000, 1_000_000, 1_000_000_000]:
    print(f"n = {n:>13}, -> {halving_steps(n)} steps")
```

4.5 Seeing the Growth

Run this cell to print a growth table. Notice how fast $O(n^2)$ and $O(2^n)$ explode compared to the others.

```
import math

print(f"{'n':>6} {'log n':>7} {'n log n':>9} {'n^2':>10} {'2^n':>22}")
for n in [1, 10, 20, 50, 100, 1000]:
    two_n = 2 ** n if n <= 60 else float("inf")
    print(f"{'n':>6} {'math.log2(n)':>7.1f} {'n * math.log2(n)':>9.0f} {'n * n':>10} {'two_n':>22}")
```



5. Part C: Best, Average and Worst Case

The same algorithm can take a different number of steps on different inputs of the same size. Consider searching for a roll number in a list of n students:

Case	When it happens	Steps	Big-O
Best case	The target is the first element	1	$O(1)$
Average case	The target is somewhere in the middle	$\approx n/2$	$O(n)$
Worst case	The target is last or not present	n	$O(n)$

In practice we usually report the **worst case**, because it gives a **guarantee**: the algorithm will never be slower than this. When an engineer says “linear search is $O(n)$ ”, they mean the worst case.



Other notations you will meet

- **Big-O (O):** upper bound, “at most this much” (used most often).
- **Big-Omega (Ω):** lower bound, “at least this much”.
- **Big-Theta (Θ):** tight bound, both upper and lower.

In interviews and in this course, Big-O for the worst case is enough unless your theory course says otherwise.



6. Part D: Space Complexity

Space complexity is the extra memory an algorithm needs as the input grows. We usually count **auxiliary space**: memory used in addition to the input itself.

```
def reverse_new(items):          # 0(n) extra space: builds a new list
    result = []
    for x in items:
        result.insert(0, x)
    return result

def reverse_in_place(items):     # 0(1) extra space: only two index
    variables
    left, right = 0, len(items) - 1
    while left < right:
        items[left], items[right] = items[right], items[left]
        left += 1
        right -= 1
    return items

print(reverse_new([1, 2, 3, 4]), reverse_in_place([1, 2, 3, 4]))
```



The time–space trade-off

Often you can make a program **faster by using more memory**. Two Sum with a dictionary uses $O(n)$ extra memory to get $O(n)$ time instead of $O(n^2)$. Engineers choose based on which resource is more precious.

Can you spot a hidden cost? `result.insert(0, x)` shifts every element, so `reverse_new` is actually **$O(n^2)$ time**. You will understand why in Lab 03.



7. Part E: Measuring Time in Python

Theory predicts growth; experiments confirm it. Python’s `time.perf_counter()` gives a precise clock.

```
import time

def measure(func, data):
    start = time.perf_counter()
    func(data)
    return time.perf_counter() - start

def linear_work(data):
    total = 0
    for x in data:
        total += x
    return total

def quadratic_work(data):
    count = 0
    for x in data:
        for y in data:
            count += 1
    return count

print(f"{'n':>6} {'O(n) seconds':>14} {'O(n^2) seconds':>16}")
for n in [250, 500, 1000, 2000]:
    data = list(range(n))
    print(f"{'n':>6} {measure(linear_work, data):>14.6f} {measure(quadratic_work, data):>16.6f}")
```

When n **doubles**, the $O(n)$ time roughly doubles, but the $O(n^2)$ time roughly becomes **four times** larger. That is the signature of quadratic growth.

8. C++ Corner

Complexity is language-independent: the same loops have the same Big-O in C++. C++ just runs each step faster.

C++

```

#include <iostream>
#include <vector>
#include <chrono>
using namespace std;
using namespace std::chrono;

long long quadraticWork(const vector<int>& data) {
    long long count = 0;
    for (int x : data)
        for (int y : data)
            count++;
    return count;
}

int main() {
    for (int n : {1000, 2000, 4000}) {
        vector<int> data(n, 1);
        auto start = high_resolution_clock::now();
        quadraticWork(data);
        auto ms = duration_cast<microseconds>(high_resolution_clock::now() -
start).count();
        cout << "n = " << n << " -> " << ms << " microseconds" << endl;
    }
}

```

9. Complexity Cheat Sheet

Code pattern	Big-O
Simple statements, no loops	$O(1)$
One loop from 0 to n	$O(n)$
Loop where i doubles or halves each time	$O(\log n)$
Two nested loops over n	$O(n^2)$
Loop over n, inside a halving loop	$O(n \log n)$
Two separate loops over n and m	$O(n + m)$
<code>x in list</code>	$O(n)$
<code>x in set, dict[key]</code>	$O(1)$ average
<code>list.append(x), list.pop()</code>	$O(1)$ amortised
<code>list.insert(0, x), list.pop(0)</code>	$O(n)$
<code>sorted(list)</code>	$O(n \log n)$

10. Common Mistakes

Mistake	Why it is wrong	Correct thinking
“My code ran in 0.01 s, so it is efficient”	Small inputs hide bad growth	Test with large n and analyse Big-O
Writing $O(2n)$ or $O(n/2)$	Constants are dropped	Both are $O(n)$
Ignoring built-in costs	x in list inside a loop is $O(n)$ each time	That loop becomes $O(n^2)$; use a set
Nested loops are always $O(n^2)$	Inner loop may run a fixed number of times	Loop of $n \times$ loop of 5 = $O(n)$
Forgetting space	A new list of size n uses $O(n)$ memory	Always state time and space

11. Lab Tasks

Solve all tasks in `Lab02_Student.ipynb`. Each task has a **test cell**: when it prints  your solution is correct. For each task, **write pseudocode first**, then code, then test with your own inputs including edge cases. Difficulty:  Easy  Medium  Hard.

Task 1 Count the Steps

Write `count_steps(n)` that returns how many times the line `count += 1` runs in the code below, **without running the loops** (use a formula). Then state its Big-O in a comment.

`for i in range(n): / for j in range(n): / count += 1` followed by a separate loop `for k in range(n): count += 1.`

```
count_steps(10) → 110      # n*n + n
count_steps(0)  → 0
```

Task 2 Identify the Big-O

For each function in the notebook, fill the dictionary `answers` with its Big-O as a string, choosing from `"O(1)"`, `"O(log n)"`, `"O(n)"`, `"O(n log n)"`, `"O(n^2)"`.

Task 3 Contains Duplicate: $O(n^2) \rightarrow O(n)$

Write `has_duplicate_fast(items)` that returns `True` if any value appears at least twice. It must be $O(n)$ time using a `set`. Compare it with `has_duplicate_slow` from the manual.

In a comment, state the time and space complexity of both versions.

```
[3, 1, 4, 1, 5] → True
[1, 2, 3]       → False
```

Task 4 Missing Roll Number

A class has roll numbers 1 to n , but one student's roll number is missing from the attendance list (the list has $n-1$ numbers, in any order). Write `find_missing(rolls, n)` in $O(n)$ time and $O(1)$ extra space.

Hint: the sum of 1 to n is $n(n+1)/2$.

```
find_missing([3, 1, 5, 2], 5) → 4
```

Task 5 Prove the Growth Experimentally

Write `doubling_ratio(func, n)` that measures `func(list(range(n)))` and `func(list(range(2 * n)))` and returns the ratio `time(2n) / time(n)`.

Use it on `linear_work` and `quadratic_work` from the manual. A ratio near 2 suggests $O(n)$; near 4 suggests $O(n^2)$. Explain your results in a markdown cell.

Task 6 Fast Power: $O(n) \rightarrow O(\log n)$

Computing x^n by multiplying x by itself n times is $O(n)$. Write `fast_power(x, n)` that is **$O(\log n)$** using this idea: if n is even, $x^n = (x^2)^{(n/2)}$; if n is odd, $x^n = x \cdot x^{n-1}$. Do not use `**` or `pow()`.

Also return the number of multiplications performed, so you can see it grows like $\log n$.

```
fast_power(2, 10) → (1024, multiplications)    # around 5–6, not 10
```

Task 7 Common Elements of Two Lists

Two sections submitted lists of students who joined a society. Write `common_students(a, b)` that returns a **sorted list** of names present in both lists, without duplicates.

Write it once with nested loops ($O(a \times b)$) and once with sets ($O(a + b)$) as `common_students_fast`. State both complexities.

```
a = ["Ali", "Sara", "Hamza", "Ali"], b = ["Sara", "Usman", "Ali"]
→ ["Ali", "Sara"]
```



12. LeetCode Practice

Solve these on **leetcode.com** and upload screenshots of the “Accepted” result to your GitHub `DSA-Labs/Lab02` folder.

#	Problem	Level	Concept practised
217	Contains Duplicate	Easy	Set, $O(n)$ vs $O(n^2)$
268	Missing Number	Easy	Math / $O(1)$ space
1342	Number of Steps to Reduce a Number to Zero	Easy	Halving $\rightarrow O(\log n)$
1365	How Many Numbers Are Smaller Than the Current Number	Easy	Brute force vs counting

#	Problem	Level	Concept practised
1295	Find Numbers with Even Number of Digits	Easy	Analysing digit loops
50	Pow(x, n)	Medium	Fast power, $O(\log n)$



13. Home Task

1. Complete any unfinished lab tasks and push the notebook to [DSA-Labs/Lab02](#).
2. For every function you wrote in Lab 01, write its time and space complexity as a comment.
3. Write a one-page note (in a markdown cell) comparing $O(n)$ and $O(n^2)$ with your own real-life example from Pakistan (e.g., NADRA, results, cricket scores).
4. **Mini-project:** build a small “Algorithm Race” in the notebook that times linear search vs set lookup for $n = 10^3, 10^4, 10^5, 10^6$ and prints a neat table.
5. Solve at least 2 LeetCode problems from the list and write the complexity of each solution.

? 14. Viva Questions

1. Why do we count steps instead of measuring seconds?
2. What does $O(n)$ mean in simple words?
3. Why do we drop constants in Big-O? Give an example.
4. Arrange in increasing order: $O(n^2)$, $O(1)$, $O(n \log n)$, $O(\log n)$, $O(n)$.
5. What is the difference between best, average and worst case?
6. Why is a halving loop $O(\log n)$?
7. What is space complexity? What is the space complexity of reversing a list in place?
8. Explain the time–space trade-off with Two Sum.
9. What is the complexity of `x in list` and `x in set`?
10. If n doubles, how does the running time of an $O(n^2)$ algorithm change?



15. Assessment Rubric

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Correctness	All tasks pass all tests	Most tasks correct	Some tasks correct	Mostly incorrect
Logic & pseudocode	Clear pseudocode and dry run for every task	Pseudocode for most tasks	Partial pseudocode	None
Code quality	Meaningful names, functions, comments	Mostly clean	Hard to read	Copied or unreadable
Complexity & edge cases	States Big-O and handles all edge cases	Some edge cases / Big-O	Only basic cases	Not considered
Viva	Explains every line confidently	Explains most parts	Basic understanding	Cannot explain



Lab Rules

- Bring your own laptop with Colab or Jupyter working.
- Arrive on time; late arrival may result in no attendance for the lab.
- Copying another student's code or pasting AI output without understanding it will be awarded zero. You must be able to explain every line in the viva.



16. References and Further Learning

- Big-O Cheat Sheet: bigocheatsheet.com
- Python TimeComplexity wiki: wiki.python.org/moin/TimeComplexity
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, **Introduction to Algorithms** (CLRS), 4th ed., MIT Press, 2022.
- M. T. Goodrich, R. Tamassia, M. H. Goldwasser, **Data Structures and Algorithms in Python**, Wiley.
- MIT OpenCourseWare 6.006 Introduction to Algorithms; Harvard CS50x (free online courses).
- Visualise code step by step: pythontutor.com; animated data structures: visualgo.net.



About the Author

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

GitHub: github.com/RamzanPUCIT

LinkedIn: linkedin.com/in/ramzan-ai

ResearchGate: researchgate.net/profile/Muhammad-Ramzan-105

© 2026 Muhammad Ramzan. Prepared for the Department of Computer Science, University of Okara.

UNIVERSITY OF OKARA

Department of Computer Science

DATA STRUCTURES & ALGORITHMS

Lab Manual

LAB 03 | Arrays: Static Arrays, Dynamic Arrays and 2D Matrices

Week 3 of 16 • Duration: 3 hours (in-lab) + home practice

Prepared by

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

Applicable to: BSCS • BSIT • BSSE • BSAI • BSDS • BS Cyber Security and all programs offering DSA

Fall 2026 Edition

About This Lab Manual

This manual is part of a **16-week Data Structures & Algorithms (DSA) lab series**. It is designed so that a student starting with **zero programming knowledge** in Week 1 can confidently design, implement and analyse data structures by Week 16. Every lab builds on the previous one, so please do not skip labs.

Language policy: concepts are explained in a language-independent way (diagrams, pseudocode, dry runs). Full implementations are in **Python**, and every lab includes a **C++ Corner** so instructors teaching in C++ can use the same manual.

Every lab follows the same structure

Section	What you get
 Objectives & CLOs	What you will be able to do after this lab
 Motivation	Where this topic is used in real life, industry, research and the future (2026–2030)
 Concepts	Simple explanations with real examples, pseudocode and dry runs
 Code	Python implementation + C++ Corner
 Lab Tasks	Graded tasks:  Easy,  Medium,  Hard
 LeetCode	Real interview problems to practise
 Home Task &  Viva	Practice at home and prepare for oral questions
 Notebooks	Instructor notebook (complete code) and Student notebook (with TODO blanks and auto-check tests)

Lab 03 at a glance

Item	Details
Lab title	Arrays: Static Arrays, Dynamic Arrays and 2D Matrices
Week	3 of 16
Prerequisites	Lab 01 (Python basics), Lab 02 (Big-O)
Tools	Google Colab / Jupyter / VS Code (Python 3); any C++ compiler for the C++ Corner
Files	Lab03_Instructor.ipynb , Lab03_Student.ipynb
Next lab	Lab 04: Searching Techniques (Linear, Binary, Two Pointers, Sliding Window)

1. Learning Objectives

By the end of this lab, you will be able to:

1. Explain how an array is stored in **contiguous memory** and why index access is **$O(1)$** .
2. Calculate the memory address of any element using the **address formula**.
3. Perform and analyse the array operations: **traverse, access, update, search, insert, delete**.
4. Write a simple **Python class** and use it to build your own **Dynamic Array** with automatic resizing.
5. Explain **amortised $O(1)$** append and why Python lists double their capacity.
6. Work with **2D arrays (matrices)**: traversal, row/column sums, transpose and multiplication.
7. Apply arrays to real problems such as rotating, merging and cleaning data.

Course Learning Outcomes (CLO) mapping

CLO	Description	Taxonomy level
CLO-1	Explain the organisation of arrays in memory	Understand (C2)
CLO-2	Implement array operations and a dynamic array	Apply (C3)
CLO-3	Analyse the complexity of array operations	Analyse (C4)

2. Motivation: The Most Important Data Structure

If you learn only one data structure in your life, it should be the **array**. It is the foundation of almost everything else: strings, stacks, queues, heaps, hash tables, images, matrices and the tensors used in deep learning are all built on arrays.

2.1 Arrays in Daily Life

Real thing	How it is an array
A row of lockers numbered 0, 1, 2 ...	Each locker holds one item; you go directly to locker number 7
Seats in a cinema hall (Row C, Seat 12)	A 2D array: row and column
Mobile phone photo (e.g., 1080 × 1920)	A 2D array of pixels; with colour it is 3D (R, G, B)
Excel sheet / result gazette	A 2D table of rows (students) and columns (subjects)

Real thing	How it is an array
Daily temperature of Lahore for a month	A 1D array of 30 numbers
Chess / Ludo board	An 8×8 or 15×15 grid

2.2 Arrays in Industry

- **Image and video processing** (Instagram filters, Snapchat lenses) apply mathematics to huge 2D/3D arrays of pixels.
- **Databases** store table rows in pages that are essentially arrays on disk.
- **Games** use 2D arrays for maps and boards; **spreadsheets** are 2D arrays with formulas.
- Python's `list`, C++'s `std::vector` and Java's `ArrayList` are all **dynamic arrays**, used in nearly every program.

2.3 Arrays in Research

- **Deep Learning:** every image, sentence and model weight is stored as a multi-dimensional array (tensor) in NumPy, PyTorch and TensorFlow.
- **Remote Sensing & GIS:** satellite images (e.g., Sentinel-2, Landsat) are stacks of 2D arrays, one per spectral band; NDVI for crop health is computed element-by-element on arrays.
- **Scientific computing:** weather prediction and physics simulations operate on gigantic grids.

2.4 Why It Matters in 2026–2030

Arrays + hardware = speed

GPUs and AI chips are designed to process **arrays in parallel**. Because array elements sit next to each other in memory, the hardware can load them very quickly (cache-friendly). This is a big reason why array-based code is fast and why AI frameworks are built on arrays. Understanding arrays is your first step towards NumPy, Pandas, PyTorch and high-performance computing.

3. Part A: What Is an Array?

An **array** is a collection of elements of the **same type** stored in **contiguous (side-by-side) memory locations**. Each element is identified by an **index**, starting from **0**.

An array in memory

```

Index:      0      1      2      3      4
           +-----+-----+-----+-----+
marks:  | 67 | 89 | 45 | 92 | 78 |
           +-----+-----+-----+-----+
Address: 1000 1004 1008 1012 1016      (each int = 4 bytes)

```

3.1 Why Is Access $O(1)$? The Address Formula

Because elements are side by side and all have the same size, the computer can **calculate** the address of any element directly, without visiting the others:

**Address formula (1D)**

Address($A[i]$) = Base address + $i \times$ size of each element

Example: base = 1000, each int is 4 bytes. Address of $A[3] = 1000 + 3 \times 4 =$ **1012**. One multiplication and one addition, no matter how big the array is. That is **$O(1)$** .

This is also why indexes start at 0: index 0 means “0 elements after the base address”.

3.2 Static vs Dynamic Arrays

	Static array	Dynamic array
Size	Fixed when created	Grows and shrinks automatically
Examples	C/C++ <code>int a[10];</code>	Python <code>list</code> , C++ <code>vector</code> , Java <code>ArrayList</code>
If full	Cannot add more elements	Allocates a bigger block and copies elements
Memory	Exactly what you asked for	Keeps some extra capacity for future growth



4. Part B: Array Operations and Their Cost

We will now implement each operation **by hand**, as if we had only a fixed-size block of memory. This shows what Python lists do internally.

4.1 Traversal: Visit Every Element, $O(n)$

```
marks = [67, 89, 45, 92, 78]
for i in range(len(marks)):
    print(f"marks[{i}] = {marks[i]}")
```

4.2 Access and Update: $O(1)$

```
marks = [67, 89, 45, 92, 78]
print(marks[3])      # access:  $O(1)$ 
marks[3] = 95        # update:  $O(1)$ 
print(marks)
```

4.3 Insertion at a Position: $O(n)$

To insert at index **pos**, every element from **pos** to the end must **shift one place right** to make room. In the worst case (inserting at index 0) all n elements move.

Dry run

Insert 50 at index 1 in [67, 89, 45, 92, _]

```
Step 1: shift 92 right  -> [67, 89, 45, _ , 92]
Step 2: shift 45 right  -> [67, 89, _ , 45, 92]
Step 3: shift 89 right  -> [67, _ , 89, 45, 92]
Step 4: place 50        -> [67, 50, 89, 45, 92]
```

```
def insert_at(arr, size, pos, value):
    """arr has fixed capacity; size = number of used slots."""
    if size == len(arr):
        raise OverflowError("Array is full")
    for i in range(size, pos, -1): # shift right, starting from the end
        arr[i] = arr[i - 1]
    arr[pos] = value
    return size + 1

arr = [67, 89, 45, 92, None]          # capacity 5, size 4
size = insert_at(arr, 4, 1, 50)
print(arr, "size =", size)
```

4.4 Deletion at a Position: $O(n)$

To delete the element at index **pos**, every element after it **shifts one place left** to fill the gap.

```
def delete_at(arr, size, pos):
    for i in range(pos, size - 1):    # shift left
        arr[i] = arr[i + 1]
    arr[size - 1] = None              # clear the last used slot
    return size - 1

arr = [67, 50, 89, 45, 92]
size = delete_at(arr, 5, 1)
print(arr, "size =", size)
```

4.5 Linear Search: $O(n)$

```
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i                # found: return the index
    return -1                       # not found

print(linear_search([67, 89, 45, 92, 78], 92), linear_search([1, 2], 7))
```

Operation	Time	Why
Access / update a[i]	$O(1)$	Address formula
Traverse	$O(n)$	Visit every element
Search (unsorted)	$O(n)$	May check every element
Insert / delete at end	$O(1)$	No shifting needed
Insert / delete at beginning or middle	$O(n)$	Elements must shift



5. Part C: Python Classes in 5 Minutes

From now on we will build our own data structures. In Python, we do this with a **class**. A class is a **blueprint**; an object is a thing built from that blueprint.



Analogy

A class is like the **map of a house**. Using one map you can build many houses (objects). Each house has its own rooms (attributes) and things you can do in it (methods).

```

class Student:
    def __init__(self, name, roll):      # constructor: runs when an object is
created
        self.name = name                # attribute
        self.roll = roll
        self.marks = []

    def add_mark(self, m):               # method
        self.marks.append(m)

    def average(self):
        return sum(self.marks) / len(self.marks) if self.marks else 0

    def __str__(self):                  # how the object looks when printed
        return f"{self.roll}: {self.name} (avg {self.average():.1f})"

s = Student("Ayesha", "BSCS-21")
s.add_mark(80); s.add_mark(90)
print(s)

```

self refers to the object itself. **__init__** sets up the object, and **__str__** decides how it prints. We will use special methods like **__len__** and **__getitem__** so our structures behave like Python lists.



6. Part D: Build Your Own Dynamic Array

How does a Python list grow without limit if memory blocks are fixed? Answer: when the block is full, it **allocates a new block (usually about double the size), copies all elements, and frees the old block**.

Resizing by doubling

```

Capacity 2: [10, 20]          append 30 -> FULL!
New capacity 4: copy 10, 20  -> [10, 20, 30, _]
append 40                      -> [10, 20, 30, 40]
New capacity 8: copy 4 items -> [10, 20, 30, 40, 50, _, _, _]

```

```

class DynamicArray:
    def __init__(self):
        self.capacity = 1
        self.size = 0
        self.data = [None] * self.capacity    # our fixed block of memory

    def __len__(self):
        return self.size

    def __getitem__(self, i):
        if not 0 <= i < self.size:
            raise IndexError("index out of range")
        return self.data[i]

    def _resize(self, new_capacity):
        new_data = [None] * new_capacity
        for i in range(self.size):            # copy: O(n)
            new_data[i] = self.data[i]
        self.data = new_data
        self.capacity = new_capacity

    def append(self, value):
        if self.size == self.capacity:
            self._resize(2 * self.capacity)    # double when full
        self.data[self.size] = value
        self.size += 1

    def __str__(self):
        return str([self.data[i] for i in range(self.size)]) + f" (capacity {self.capacity})"

arr = DynamicArray()
for x in [10, 20, 30, 40, 50]:
    arr.append(x)
    print(arr)

```

6.1 Amortised $O(1)$: Why Doubling Is Clever

A resize costs $O(n)$, but it happens rarely. If we append n items, the total copying work is $1 + 2 + 4 + \dots + n/2 + n < 2n$. Spread over n appends, that is less than 2 **copies per append on average**, so each append is **amortised $O(1)$** .

```

class CountingArray(DynamicArray):
    copies = 0
    def _resize(self, new_capacity):
        CountingArray.copies += self.size
        super()._resize(new_capacity)

for n in [1_000, 10_000, 100_000]:
    CountingArray.copies = 0
    a = CountingArray()
    for i in range(n):
        a.append(i)
    print(f"n = {n:>7}, total copies = {CountingArray.copies:>7}  ({CountingArray.copies / n:.2f} per append)")

```

 What if we grew by +1 instead of $\times 2$?

Then every append would trigger a copy, and appending n items would cost $1 + 2 + \dots + n = \mathbf{O(n^2)}$. Doubling turns that into $O(n)$ total. A small design decision with a huge impact.

7. Part E: 2D Arrays (Matrices)

A **2D array** stores data in **rows and columns**. In Python we represent it as a **list of lists**.

```
#           Maths  Physics  CS
result = [ [ 78,      85,      90 ], # row 0: Ali
            [ 66,      72,      95 ], # row 1: Sara
            [ 88,      91,      79 ] ] # row 2: Hamza

print(result[1][2])                # Sara's CS marks -> 95
rows, cols = len(result), len(result[0])

for r in range(rows):              # row-wise traversal
    print("Row", r, "total =", sum(result[r]))

for c in range(cols):              # column-wise traversal
    col_total = 0
    for r in range(rows):
        col_total += result[r][c]
    print("Subject", c, "average =", round(col_total / rows, 1))
```

The classic 2D trap

`grid = [[0] * 3] * 3` creates **three references to the same row**. Changing `grid[0][0]` changes all rows! Always use `grid = [[0] * 3 for _ in range(3)]`.

```
wrong = [[0] * 3] * 3
wrong[0][0] = 9
print("wrong:", wrong)

right = [[0] * 3 for _ in range(3)]
right[0][0] = 9
print("right:", right)
```

7.1 Row-Major Storage and the 2D Address Formula

Memory is one-dimensional, so a 2D array is stored **row after row** (row-major order, used by C, C++ and NumPy by default).

Address formula (2D, row-major)

Address($A[i][j]$) = Base + ($i \times \text{number_of_columns} + j$) $\times$ element_size

Example: a 3×4 int matrix at base 2000. Address of $A[2][1] = 2000 + (2 \times 4 + 1) \times 4 = 2036$.

7.2 Transpose and Matrix Multiplication

```
def transpose(m):
    rows, cols = len(m), len(m[0])
    t = [[0] * rows for _ in range(cols)]
    for i in range(rows):
        for j in range(cols):
            t[j][i] = m[i][j]
    return t

def mat_mult(a, b):          # (n x m) * (m x p) -> (n x p):
    0(n*m*p)
    n, m, p = len(a), len(b), len(b[0])
    c = [[0] * p for _ in range(n)]
    for i in range(n):
        for j in range(p):
            for k in range(m):
                c[i][j] += a[i][k] * b[k][j]
    return c

A = [[1, 2, 3], [4, 5, 6]]
print(transpose(A))
print(mat_mult([[1, 2], [3, 4]], [[5, 6], [7, 8]]))
```

Matrix multiplication is the core operation of neural networks. A single layer computes **output = input × weights**, which is exactly this triple loop (done on GPUs in parallel).

8. C++ Corner

C++

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int marks[5] = {67, 89, 45, 92, 78};           // static array: fixed
    size                                           size
    cout << "Address of marks[0]: " << &marks[0] << endl;
    cout << "Address of marks[1]: " << &marks[1] << endl;    // 4 bytes
    later

    vector<int> v;                                // dynamic array
    for (int i = 0; i < 10; i++) {
        v.push_back(i);
        cout << "size=" << v.size() << " capacity=" << v.capacity() <<
    endl;
    }

    int grid[3][4] = {0};                         // 2D static array
    grid[2][1] = 7;
    cout << grid[2][1] << endl;
    return 0;
}
```

Run this and watch `capacity()` jump (1, 2, 4, 8, 16 on most compilers). That is the same doubling strategy you just implemented.

9. Common Mistakes

Mistake	Result	Fix
Accessing <code>a[len(a)]</code>	IndexError	Last valid index is <code>len(a) - 1</code>
Modifying a list while looping over it	Skipped elements	Loop over a copy or build a new list
<code>[[0]*c]*r</code> for a grid	All rows change together	<code>[[0]*c for _ in range(r)]</code>
Using <code>insert(0, x)</code> in a big loop	Hidden $O(n^2)$	Append, then reverse once
Mixing up rows and columns	Wrong results / IndexError	<code>m[row][col]</code> ; rows = <code>len(m)</code> , cols = <code>len(m[0])</code>

10. Lab Tasks

Solve all tasks in `Lab03_Student.ipynb`. Each task has a **test cell**: when it prints  your solution is correct. For each task, **write pseudocode first**, then code, then test with your own inputs including edge cases. Difficulty:  Easy  Medium  Hard.

Task 1 Array Statistics in One Pass

Write `stats(arr)` that returns (minimum, maximum, average) using a **single loop** and no built-in `min`, `max` or `sum`. Return `None` for an empty array.

```
stats([67, 89, 45, 92, 78]) → (45, 92, 74.2)
```

Task 2 Move Zeroes to the End

Move all `0`s to the end of the array **in place** while keeping the order of the non-zero elements. Use $O(1)$ extra space. Return the same list.

```
[0, 1, 0, 3, 12] → [1, 3, 12, 0, 0]
```

Task 3 Rotate an Array

Rotate the array to the right by `k` steps **in place** (`k` may be larger than `n`).

Hint (reversal trick): reverse the whole array, then reverse the first `k` elements, then reverse the rest. This is $O(n)$ time and $O(1)$ space.

```
rotate([1, 2, 3, 4, 5, 6, 7], 3) → [5, 6, 7, 1, 2, 3, 4]
```

Task 4 Merge Two Sorted Arrays

Two sections' merit lists are each sorted in ascending order. Write `merge_sorted(a, b)` that returns one sorted list in $O(n + m)$ time. Do not use `sort()` or `sorted()`. (This idea is the heart of Merge Sort in Lab 07.)

```
merge_sorted([1, 4, 9], [2, 3, 10, 12]) → [1, 2, 3, 4, 9, 10, 12]
```

Task 5 Complete the Dynamic Array

Extend the `DynamicArray` class from Part D (redefined in the notebook) with these methods:

`insert(pos, value)` (shift right, resize if full), `remove_at(pos)` (shift left and return the removed value), and `index_of(value)` (return the index or `-1`). Raise `IndexError` for invalid positions.

Task 6 Rotate an Image by 90°

A grayscale image is an $n \times n$ matrix. Rotate it **90° clockwise in place**.

Hint: transpose the matrix in place (swap `m[i][j]` with `m[j][i]` for $j > i$), then reverse each row.

```
[[1, 2, 3],      [[7, 4, 1],
 [4, 5, 6],    →  [8, 5, 2],
 [7, 8, 9]]      [9, 6, 3]]
```

Task 7 Spiral Order of a Matrix

Return all elements of an $m \times n$ matrix in **spiral order** (clockwise from the top-left). Keep four boundaries: top, bottom, left, right, and shrink them after each side.

```
[[1, 2, 3],
 [4, 5, 6],    → [1, 2, 3, 6, 9, 8, 7, 4, 5]
 [7, 8, 9]]
```

11. LeetCode Practice

Solve these on **leetcode.com** and upload screenshots of the “Accepted” result to your GitHub `DSA-Labs/Lab03` folder.

#	Problem	Level	Concept practised
283	Move Zeroes	Easy	In-place two indexes (Task 2)
26	Remove Duplicates from Sorted Array	Easy	In-place overwrite
88	Merge Sorted Array	Easy	Merging (Task 4)
1572	Matrix Diagonal Sum	Easy	2D indexing
189	Rotate Array	Medium	Reversal trick (Task 3)
48	Rotate Image	Medium	Transpose + reverse (Task 6)
54	Spiral Matrix	Medium	Boundaries (Task 7)



12. Home Task

1. Finish all tasks and push `Lab03_Student.ipynb` to GitHub.
2. Add a `shrink` rule to `MyArray`: when size falls to one quarter of capacity, halve the capacity. Explain why we shrink at 1/4 and not at 1/2.
3. Write `matrix_add`, `is_symmetric` and `row_with_max_sum` functions for 2D arrays with tests.
4. **Mini-project: Result Sheet Analyser.** Store a class result (at least 10 students $\times$ 5 subjects) as a 2D array. Print each student's total, percentage and grade, each subject's average and topper, and the class position list.
5. Solve at least 3 LeetCode problems from the list.



13. Viva Questions

1. What is an array? Why must all elements be of the same type?
2. Why is accessing `a[i]` $O(1)$? Write the address formula.
3. Why is inserting at the beginning of an array $O(n)$?
4. Differentiate between static and dynamic arrays.
5. How does a dynamic array grow? Why double instead of adding 1?
6. What does amortised $O(1)$ mean?
7. What is row-major order? Calculate the address of `A[1][2]` in a 3×3 int array at base 500.
8. Why is `[[0]*3]*3` dangerous?
9. What is the time complexity of multiplying two $n \times n$ matrices with three loops?
10. Give two real applications of 2D arrays.



14. Assessment Rubric

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Correctness	All tasks pass all tests	Most tasks correct	Some tasks correct	Mostly incorrect
Logic & pseudocode	Clear pseudocode and dry run for every task	Pseudocode for most tasks	Partial pseudocode	None

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Code quality	Meaningful names, functions, comments	Mostly clean	Hard to read	Copied or unreadable
Complexity & edge cases	States Big-O and handles all edge cases	Some edge cases / Big-O	Only basic cases	Not considered
Viva	Explains every line confidently	Explains most parts	Basic understanding	Cannot explain

Lab Rules

- Bring your own laptop with Colab or Jupyter working.
- Arrive on time; late arrival may result in no attendance for the lab.
- Copying another student's code or pasting AI output without understanding it will be awarded zero. You must be able to explain every line in the viva.



15. References and Further Learning

- NumPy documentation: numpy.org/doc
- Python list implementation notes: docs.python.org/3/faq/design.html (How are lists implemented?)
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, **Introduction to Algorithms** (CLRS), 4th ed., MIT Press, 2022.
- M. T. Goodrich, R. Tamassia, M. H. Goldwasser, **Data Structures and Algorithms in Python**, Wiley.
- MIT OpenCourseWare 6.006 Introduction to Algorithms; Harvard CS50x (free online courses).
- Visualise code step by step: pythontutor.com; animated data structures: visualgo.net.



About the Author

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

GitHub: github.com/RamzanPUCIT

LinkedIn: linkedin.com/in/ramzan-ai

ResearchGate: researchgate.net/profile/Muhammad-Ramzan-105

© 2026 Muhammad Ramzan. Prepared for the Department of Computer Science, University of Okara.

UNIVERSITY OF OKARA

Department of Computer Science

DATA STRUCTURES & ALGORITHMS

Lab Manual

LAB 04 | Searching Techniques: Linear Search, Binary Search, Two Pointers and Sliding Window

Week 4 of 16 • Duration: 3 hours (in-lab) + home practice

Prepared by

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

Applicable to: BSCS • BSIT • BSSE • BSAI • BSDS • BS Cyber Security and all programs offering DSA

Fall 2026 Edition

About This Lab Manual

This manual is part of a **16-week Data Structures & Algorithms (DSA) lab series**. It is designed so that a student starting with **zero programming knowledge** in Week 1 can confidently design, implement and analyse data structures by Week 16. Every lab builds on the previous one, so please do not skip labs.

Language policy: concepts are explained in a language-independent way (diagrams, pseudocode, dry runs). Full implementations are in **Python**, and every lab includes a **C++ Corner** so instructors teaching in C++ can use the same manual.

Every lab follows the same structure

Section	What you get
 Objectives & CLOs	What you will be able to do after this lab
 Motivation	Where this topic is used in real life, industry, research and the future (2026–2030)
 Concepts	Simple explanations with real examples, pseudocode and dry runs
 Code	Python implementation + C++ Corner
 Lab Tasks	Graded tasks:  Easy,  Medium,  Hard
 LeetCode	Real interview problems to practise
 Home Task &  Viva	Practice at home and prepare for oral questions
 Notebooks	Instructor notebook (complete code) and Student notebook (with TODO blanks and auto-check tests)

Lab 04 at a glance

Item	Details
Lab title	Searching Techniques: Linear Search, Binary Search, Two Pointers and Sliding Window
Week	4 of 16
Prerequisites	Lab 02 (Big-O), Lab 03 (Arrays)
Tools	Google Colab / Jupyter / VS Code (Python 3); any C++ compiler for the C++ Corner
Files	Lab04_Instructor.ipynb , Lab04_Student.ipynb
Next lab	Lab 05: Elementary Sorting (Bubble, Selection, Insertion)

1. Learning Objectives

By the end of this lab, you will be able to:

1. Implement **linear search** and explain its best, average and worst case.
2. Implement **binary search** iteratively and explain why it is **$O(\log n)$** .
3. Solve binary search **variants**: first/last occurrence, insert position, and searching on the answer.
4. Use Python's **bisect** module correctly.
5. Apply the **two-pointer** technique on sorted arrays and strings.
6. Apply the **sliding window** technique to subarray and substring problems.
7. Choose the right searching technique for a given problem.

Course Learning Outcomes (CLO) mapping

CLO	Description	Taxonomy level
CLO-2	Implement searching algorithms	Apply (C3)
CLO-3	Analyse and compare searching techniques	Analyse (C4)

2. Motivation: Finding Things Fast

Searching is the most frequent thing computers do. Every time you type in Google, look up a contact, check a result by roll number, or find a product on Daraz, a searching algorithm runs. The difference between a slow and a fast search is the difference between waiting minutes and getting results instantly.

2.1 Searching in Daily Life

Situation	Technique	Why
Finding your name on an unsorted attendance sheet	Linear search	No order, so check one by one
Finding a word in a dictionary book	Binary search	Words are sorted alphabetically
Guessing a number between 1 and 100 ("higher / lower")	Binary search	At most 7 guesses
Finding two items whose prices add up to your budget	Two pointers	Prices sorted, move from both ends
Best 7-day period of sales in a month	Sliding window	Slide a fixed window over days



2.2 Searching in Industry

- **Databases** use B-tree indexes, a generalisation of binary search, to find one record among billions in a few disk reads.
- **Git** uses `git bisect` (binary search over commits) to find which commit introduced a bug.
- **Network monitoring and finance** use sliding windows to compute moving averages, traffic per minute and fraud patterns in real time.
- Two pointers and sliding window are among the **most asked interview patterns** at top companies.



2.3 Searching in Research

- **Machine learning:** hyperparameter search and threshold tuning often use binary search on the answer.
- **Signal and sensor processing** (e.g., Human Activity Recognition from wearable sensors) splits data into **sliding windows** of fixed length before feeding them to LSTM or CNN models.
- **Bioinformatics:** searching DNA sequences uses windows (k-mers) and sorted indexes.



2.4 Why It Matters in 2026–2030



Search is everywhere in AI systems

Retrieval-Augmented Generation (RAG), the technique behind many AI assistants, first **searches** a large collection of documents and then gives the best ones to the language model. Fast, correct search is the backbone of these systems. The ideas you learn today (halving the search space, maintaining a window) appear again and again in advanced systems.



3. Part A: Linear Search

Linear (sequential) search checks every element one by one until it finds the target or reaches the end. It works on **any** array, sorted or not.

Pseudocode

```
ALGORITHM LinearSearch(A, target)
  FOR i ← 0 TO n - 1 DO
    IF A[i] = target THEN RETURN i
  RETURN -1
```

```
def linear_search(arr, target):
    for i, value in enumerate(arr):
        if value == target:
            return i
    return -1

roll_numbers = [1043, 1017, 1088, 1002, 1056]
print(linear_search(roll_numbers, 1088), linear_search(roll_numbers, 9999))
```

Case	Steps	Big-O
Best (first element)	1	O(1)
Average	$\approx n/2$	O(n)
Worst (last / absent)	n	O(n)
Extra space	-	O(1)

4. Part B: Binary Search

If the array is **sorted**, we can do much better. Look at the **middle** element: if it is the target, done; if the target is smaller, it must be in the **left half**; otherwise in the **right half**. Each step **throws away half** of the remaining elements.



Precondition

Binary search works **only on sorted data**. On unsorted data it gives wrong answers silently.

Pseudocode

```
ALGORITHM BinarySearch(A, target)
    low ← 0 ; high ← n - 1
    WHILE low ≤ high DO
        mid ← (low + high) // 2
        IF A[mid] = target THEN RETURN mid
        ELSE IF A[mid] < target THEN low ← mid + 1    // go right
        ELSE high ← mid - 1                          // go left
    RETURN -1
```

Dry run: search 23 in [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]

Step	low	high	mid	A[mid]	Decision
1	0	9	4	16	$16 < 23 \rightarrow$ go right, low = 5
2	5	9	7	56	$56 > 23 \rightarrow$ go left, high = 6
3	5	6	5	23	Found at index 5 

```
def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1

data = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
print(binary_search(data, 23), binary_search(data, 7))
```

4.1 Why $O(\log n)$?

After 1 step $n/2$ elements remain, after 2 steps $n/4$, after k steps $n/2^k$. We stop when one element remains: $n/2^k = 1$, so **$k = \log_2 n$** . For 1 crore (10^7) sorted roll numbers, binary search needs at most **24** comparisons; linear search may need 10,000,000.

```
import math
for n in [100, 10_000, 10_000_000, 8_000_000_000]:
    print(f"n = {n:>14,}: linear worst = {n:>14,}    binary worst = {math.floor(math.log2(n)) + 1}")
```

4.2 Binary Search Variants

Many problems need not just “is it present?” but **where** exactly. The trick is: when you find a match, **do not stop**; record it and keep searching in one direction.

```
def first_occurrence(arr, target):
    low, high, answer = 0, len(arr) - 1, -1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            answer = mid          # remember it...
            high = mid - 1        # ...but keep looking on the LEFT
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return answer

def lower_bound(arr, target):
    """First index where arr[i] >= target (= insert position)."""
    low, high = 0, len(arr)      # note: high = len(arr)
    while low < high:
        mid = (low + high) // 2
        if arr[mid] < target:
            low = mid + 1
        else:
            high = mid
    return low

grades = [50, 60, 60, 60, 75, 80, 95]
print(first_occurrence(grades, 60), lower_bound(grades, 70),
      lower_bound(grades, 100))
```

4.3 Python's Built-in `bisect` Module

```
import bisect
cutoffs = [50, 60, 70, 85]          # minimum marks for D, C, B, A
letters = ["F", "D", "C", "B", "A"]

def grade(marks):
    return letters[bisect.bisect_right(cutoffs, marks)]

print([grade(m) for m in [33, 50, 69, 70, 99]])

ranks = [10, 20, 30]
bisect.insort(ranks, 25)             # insert while keeping the list sorted
print(ranks)
```

4.4 Binary Search on the Answer

Sometimes we binary search over **possible answers** instead of an array. Example: the integer square root of n is the largest x with $x^2 \leq n$. Answers lie between 0 and n , and the condition " $x^2 \leq n$ " is True for small x and False for large x , so we can binary search it.

```
def int_sqrt(n):
    low, high, ans = 0, n, 0
    while low <= high:
        mid = (low + high) // 2
        if mid * mid <= n:
            ans = mid
            low = mid + 1
        else:
            high = mid - 1
    return ans

print(int_sqrt(10), int_sqrt(144), int_sqrt(2_000_000_000))
```

👉👉 5. Part C: Two Pointers

The **two-pointer** technique uses two indexes that move through the array, usually one from the **start** and one from the **end**, or both from the start at different speeds. It often turns an $O(n^2)$ “check all pairs” solution into **$O(n)$** .

Example: pair with a given sum in a sorted array

You have Rs. 1,000 and a **sorted** price list. Find two items that cost exactly Rs. 1,000. If the sum is too small, move the left pointer right (bigger price). If too large, move the right pointer left.

```
def pair_with_sum(prices, budget):
    left, right = 0, len(prices) - 1
    while left < right:
        s = prices[left] + prices[right]
        if s == budget:
            return prices[left], prices[right]
        elif s < budget:
            left += 1          # need a bigger sum
        else:
            right -= 1         # need a smaller sum
    return None

print(pair_with_sum([120, 250, 300, 450, 700, 880], 1000))
```

Step	left → price	right → price	Sum	Action
1	0 → 120	5 → 880	1000	Found ✓

Try budget 950 in the notebook and trace the pointers yourself.

🗖️ 6. Part D: Sliding Window

A **sliding window** is a range `[left, right]` that moves over the array. Instead of recomputing everything for every window, we **add the element entering** and **remove the element leaving**. This makes many $O(n \cdot k)$ problems **$O(n)$** .

6.1 Fixed-Size Window

Problem: daily sales of a shop for 10 days are given. Find the best total over any 3 consecutive days.

Idea

```
Sales:    [4, 2, 12, 3, 8, 5, 1, 9, 6, 2]    k = 3
Window 1: [4, 2, 12] = 18
Slide   : remove 4, add 3  -> [2, 12, 3] = 17
Slide   : remove 2, add 8  -> [12, 3, 8] = 23   <- best
...
```

```
def best_k_days(sales, k):
    window = sum(sales[:k])
    best = window
    for right in range(k, len(sales)):
        window += sales[right] - sales[right - k]    # add new, remove old
        best = max(best, window)
    return best

print(best_k_days([4, 2, 12, 3, 8, 5, 1, 9, 6, 2], 3))
```

6.2 Variable-Size Window

Problem: length of the longest substring without repeating characters. Expand **right** one character at a time; whenever a repeat appears, shrink from **left** until the window is valid again.

```
def longest_unique(s):
    seen = set()
    left = best = 0
    for right, ch in enumerate(s):
        while ch in seen:           # shrink until ch is no longer
            repeated
            seen.remove(s[left])
            left += 1
        seen.add(ch)
        best = max(best, right - left + 1)
    return best

print(longest_unique("abcabcbb"), longest_unique("pakistan"))
```

Each character enters the window once and leaves at most once, so the total work is **O(n)** even though there is a loop inside a loop.

7. C++ Corner

C++

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int binarySearch(const vector<int>& a, int target) {
    int low = 0, high = (int)a.size() - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;    // avoids integer overflow
        if (a[mid] == target) return mid;
        if (a[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

int main() {
    vector<int> a = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91};
    cout << binarySearch(a, 23) << endl;           // 5
    cout << binary_search(a.begin(), a.end(), 7) << endl;    // 0
    (false)
    cout << lower_bound(a.begin(), a.end(), 20) - a.begin() << endl;    // 5
}
```



A famous bug

In C++ and Java, $(low + high) / 2$ can **overflow** when low and high are very large integers. This bug existed in Java's standard library for about 9 years. Use $low + (high - low) / 2$. Python integers never overflow, so $(low + high) // 2$ is safe in Python.

8. Which Technique When?

Situation	Technique	Time
Unsorted data, search once	Linear search	$O(n)$
Sorted data, search many times	Binary search	$O(\log n)$
Find boundary / insert position	Binary search variant / bisect	$O(\log n)$
Answer is a number with a yes/no condition	Binary search on the answer	$O(\log(\text{range}))$

Situation	Technique	Time
Pairs in sorted data, reversing, palindromes	Two pointers	$O(n)$
Best contiguous subarray / substring	Sliding window	$O(n)$
Many lookups of exact values	Set / dictionary (Lab 12)	$O(1)$ average

9. Common Mistakes

Mistake	Effect	Fix
Binary search on unsorted data	Wrong answers	Sort first or use linear search
<code>while low < high</code> with <code>high = mid - 1</code>	Misses the last element	Match loop condition with updates
<code>low = mid</code> instead of <code>mid + 1</code>	Infinite loop	Always shrink the range
Recomputing window sum every time	$O(n \cdot k)$ instead of $O(n)$	Add new element, subtract old one
Forgetting the empty-array case	IndexError	Check <code>len(arr) == 0</code>

10. Lab Tasks

Solve all tasks in `Lab04_Student.ipynb`. Each task has a **test cell**: when it prints  your solution is correct. For each task, **write pseudocode first**, then code, then test with your own inputs including edge cases. Difficulty:  Easy  Medium  Hard.

Task 1 Count Comparisons

Modify linear search and binary search so that each returns `(index, comparisons)`. Search for the last element of `list(range(1, 1_000_001))` with both and print the comparison counts.

```
linear_count(list(range(1, 1001)), 1000) → (999, 1000)
```

Task 2 First and Last Position

Given a sorted list of marks, return `[first, last]` index of a target value, or `[-1, -1]` if absent. It must be **$O(\log n)$** (two binary searches).

```
search_range([50, 60, 60, 60, 75], 60) → [1, 3]
```

Task 3 Valid Palindrome (Two Pointers)

Return `True` if a sentence is a palindrome after keeping only letters and digits and ignoring case. Use two pointers that **skip** non-alphanumeric characters. Do not build a cleaned copy of the string.

```
"A man, a plan, a canal: Panama" → True
```

Task 4 Squares of a Sorted Array

Given a sorted array that may contain negative numbers, return the squares of each number, also sorted, in **$O(n)$** . Hint: the largest square is at one of the two ends; fill the result from the back using two pointers.

```
[-4, -1, 0, 3, 10] → [0, 1, 9, 16, 100]
```

Task 5 Maximum Average Sensor Window

A wearable sensor records heart-rate values every second. Find the **maximum average** over any window of exactly `k` consecutive readings, in $O(n)$. Round to 2 decimals.

This is exactly how activity-recognition systems split sensor data into windows.

```
max_avg([1, 12, -5, -6, 50, 3], 4) → 12.75
```

Task 6 ● Minimum Days to Make Bouquets (Search on Answer)

A nursery has flowers; `bloom[i]` is the day flower `i` blooms. You need `m` bouquets, each made of `k` **adjacent** bloomed flowers. Return the minimum day on which this is possible, or `-1`.

Hint: if it is possible on day `d`, it is also possible on any later day. Binary search the day between `min(bloom)` and `max(bloom)`, and write a helper `can_make(day)` that scans the array in $O(n)$.

```
min_days([1, 10, 3, 10, 2], m=3, k=1) → 3
min_days([1, 10, 3, 10, 2], m=3, k=2) → -1
```

Task 7 ● Longest Streak with at Most One Absence

Attendance is a list of 1 (present) and 0 (absent). Find the length of the **longest continuous period** that contains **at most one** absence. Use a variable-size sliding window in $O(n)$.

```
[1, 1, 0, 1, 1, 1, 0, 1] → 6      # [1, 1, 0, 1, 1, 1] or [1, 1, 1, 0, 1] ...
```



11. LeetCode Practice

Solve these on **leetcode.com** and upload screenshots of the “Accepted” result to your GitHub `DSA-Labs/Lab04` folder.

#	Problem	Level	Concept practised
704	Binary Search	Easy	Classic binary search
35	Search Insert Position	Easy	Lower bound
125	Valid Palindrome	Easy	Two pointers (Task 3)
643	Maximum Average Subarray I	Easy	Fixed window (Task 5)
34	Find First and Last Position of Element in Sorted Array	Medium	Binary search variants (Task 2)
167	Two Sum II – Input Array Is Sorted	Medium	Two pointers

#	Problem	Level	Concept practised
3	Longest Substring Without Repeating Characters	Medium	Variable window
1482	Minimum Number of Days to Make m Bouquets	Medium	Search on answer (Task 6)

12. Home Task

1. Finish all tasks and push `Lab04_Student.ipynb` to GitHub.
2. Write a **recursive** version of binary search and compare it with the iterative one (you will study recursion formally in Lab 06).
3. Implement **jump search** (jump $\sqrt{n}$ steps, then linear search) and compare its comparison count with linear and binary search.
4. **Mini-project: Smart Result Finder.** Store 10,000 generated student records sorted by roll number. Let the user search by roll number (binary search), find all students with the same marks (first/last occurrence), and find the best 5-subject streak of a student (sliding window).
5. Solve at least 3 LeetCode problems from the list.

13. Viva Questions

1. What is the main requirement for binary search?
2. Why is binary search $O(\log n)$? How many steps for 1,000,000 elements?
3. What are the best and worst cases of linear search?
4. When would you prefer linear search over binary search?
5. How do you find the first occurrence of a repeated value using binary search?
6. What does `bisect_left` return?
7. Explain the overflow bug in $(low + high) / 2$ and how to avoid it.
8. What is the two-pointer technique? Give an example.
9. What is a sliding window? Why is it $O(n)$ even with a nested while loop?
10. What is “binary search on the answer”?



14. Assessment Rubric

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Correctness	All tasks pass all tests	Most tasks correct	Some tasks correct	Mostly incorrect
Logic & pseudocode	Clear pseudocode and dry run for every task	Pseudocode for most tasks	Partial pseudocode	None
Code quality	Meaningful names, functions, comments	Mostly clean	Hard to read	Copied or unreadable
Complexity & edge cases	States Big-O and handles all edge cases	Some edge cases / Big-O	Only basic cases	Not considered
Viva	Explains every line confidently	Explains most parts	Basic understanding	Cannot explain



Lab Rules

- Bring your own laptop with Colab or Jupyter working.
- Arrive on time; late arrival may result in no attendance for the lab.
- Copying another student's code or pasting AI output without understanding it will be awarded zero. You must be able to explain every line in the viva.



15. References and Further Learning

- Python `bisect` module: docs.python.org/3/library/bisect.html
- J. Bloch, “Extra, Extra – Read All About It: Nearly All Binary Searches and Mergesorts are Broken”, Google Research Blog, 2006.
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, **Introduction to Algorithms** (CLRS), 4th ed., MIT Press, 2022.
- M. T. Goodrich, R. Tamassia, M. H. Goldwasser, **Data Structures and Algorithms in Python**, Wiley.
- MIT OpenCourseWare 6.006 Introduction to Algorithms; Harvard CS50x (free online courses).

- Visualise code step by step: **pythontutor.com**; animated data structures: **visualgo.net**.



About the Author

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

GitHub: github.com/RamzanPUCIT

LinkedIn: linkedin.com/in/ramzan-ai

ResearchGate: researchgate.net/profile/Muhammad-Ramzan-105

© 2026 Muhammad Ramzan. Prepared for the Department of Computer Science, University of Okara.

UNIVERSITY OF OKARA

Department of Computer Science

DATA STRUCTURES & ALGORITHMS

Lab Manual

LAB 05 | Elementary Sorting: Bubble Sort, Selection Sort and Insertion Sort

Week 5 of 16 • Duration: 3 hours (in-lab) + home practice

Prepared by

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

Applicable to: BSCS • BSIT • BSSE • BSAI • BSDS • BS Cyber Security and all programs offering DSA

Fall 2026 Edition

About This Lab Manual

This manual is part of a **16-week Data Structures & Algorithms (DSA) lab series**. It is designed so that a student starting with **zero programming knowledge** in Week 1 can confidently design, implement and analyse data structures by Week 16. Every lab builds on the previous one, so please do not skip labs.

Language policy: concepts are explained in a language-independent way (diagrams, pseudocode, dry runs). Full implementations are in **Python**, and every lab includes a **C++ Corner** so instructors teaching in C++ can use the same manual.

Every lab follows the same structure

Section	What you get
 Objectives & CLOs	What you will be able to do after this lab
 Motivation	Where this topic is used in real life, industry, research and the future (2026–2030)
 Concepts	Simple explanations with real examples, pseudocode and dry runs
 Code	Python implementation + C++ Corner
 Lab Tasks	Graded tasks:  Easy,  Medium,  Hard
 LeetCode	Real interview problems to practise
 Home Task &  Viva	Practice at home and prepare for oral questions
 Notebooks	Instructor notebook (complete code) and Student notebook (with TODO blanks and auto-check tests)

Lab 05 at a glance

Item	Details
Lab title	Elementary Sorting: Bubble Sort, Selection Sort and Insertion Sort
Week	5 of 16
Prerequisites	Lab 03 (Arrays), Lab 04 (Searching)
Tools	Google Colab / Jupyter / VS Code (Python 3); any C++ compiler for the C++ Corner
Files	Lab05_Instructor.ipynb , Lab05_Student.ipynb
Next lab	Lab 06: Recursion and Divide & Conquer

1. Learning Objectives

By the end of this lab, you will be able to:

1. Explain why sorting is important and how it enables fast searching.
2. Implement **Bubble Sort** (with early stopping), **Selection Sort** and **Insertion Sort**.
3. Trace each algorithm by hand with a **pass-by-pass dry run**.
4. Count comparisons and swaps and derive the **best, average and worst case** complexity.
5. Explain **stability** and **in-place** sorting with examples.
6. Sort records (e.g., students) by one or more keys using Python's `sorted()` and `key=`.
7. Choose a suitable elementary sort for small or nearly-sorted data.

Course Learning Outcomes (CLO) mapping

CLO	Description	Taxonomy level
CLO-2	Implement elementary sorting algorithms	Apply (C3)
CLO-3	Analyse and compare sorting algorithms	Analyse (C4)

2. Motivation: Order Makes Everything Faster

In Lab 04 you saw that binary search needs only 20 steps for a million items, **but only if the data is sorted**. Sorting is the step that makes fast searching, ranking and analysis possible. It is so important that some estimates say a noticeable share of all computing time in history has been spent on sorting.

2.1 Sorting in Daily Life

Situation	Sorted by
University merit list	Aggregate marks (descending)
Cricket batting averages / PSL points table	Points, then net run rate
Daraz: "Price: Low to High"	Price
Contacts in your phone	Name (alphabetical)
Playing cards in your hand	You naturally use Insertion Sort!
Students standing in a line by height (PT period)	Height

2.2 Sorting in Industry

- Databases sort data for **ORDER BY**, for building indexes and for joining tables.
- E-commerce and search engines **rank** results; ranking is sorting by relevance score.
- Python's built-in sort (**Timsort**) and C++'s **std::sort** (**Introsort**) both use **Insertion Sort** for small pieces of data, because it is very fast on small or nearly-sorted inputs. So the simple algorithm you learn today runs inside real industrial software.

2.3 Sorting in Research

- **Machine learning:** decision trees sort feature values to find the best split; evaluation metrics like ROC-AUC sort predictions by score.
- **Information retrieval:** “learning to rank” research is about sorting documents by relevance.
- **GIS:** spatial data is often sorted along space-filling curves (e.g., Z-order) for fast queries.

2.4 Why It Matters in 2026–2030

Learning to think like an algorithm designer

You will rarely write bubble sort in a real job; you will call `sorted()`. But elementary sorts teach the core skills every engineer needs: **loop invariants**, **dry runs**, **counting operations** and **comparing trade-offs**. These skills let you understand faster algorithms (Lab 07) and evaluate any code, whether written by a human or an AI.

3. Part A: Basic Terms

Term	Meaning	Example
Ascending / descending	Smallest first / largest first	[1, 3, 7] / [7, 3, 1]
Comparison	Checking which of two elements is bigger	<code>a[j] > a[j+1]</code>
Swap	Exchanging two elements	<code>a[i], a[j] = a[j], a[i]</code>

Term	Meaning	Example
Pass	One complete scan over the (unsorted part of the) array	-
In-place	Uses only $O(1)$ extra memory	Bubble, Selection, Insertion
Stable	Equal elements keep their original relative order	Two students with 80 marks stay in roll-number order

4. Part B: Bubble Sort

Idea: repeatedly compare **adjacent** elements and swap them if they are in the wrong order. After each pass, the **largest remaining element “bubbles up”** to its correct position at the end.

Pseudocode

```

ALGORITHM BubbleSort(A)
  FOR i ← 0 TO n - 2 DO                                // pass number
    swapped ← FALSE
    FOR j ← 0 TO n - 2 - i DO                          // last i elements already
      sorted
        IF A[j] > A[j + 1] THEN
          swap A[j], A[j + 1] ; swapped ← TRUE
    IF swapped = FALSE THEN BREAK                      // already sorted: stop early

```

Dry run on [5, 1, 4, 2, 8]

Pass	Array after the pass	Fixed at the end
1	[1, 4, 2, 5, 8]	8
2	[1, 2, 4, 5, 8]	5, 8
3	[1, 2, 4, 5, 8] (no swaps)	Stop early 

```
def bubble_sort(arr):
    a = arr[:]                # work on a copy
    n = len(a)
    for i in range(n - 1):
        swapped = False
        for j in range(n - 1 - i):
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swapped = True
        print(f"Pass {i + 1}: {a}")
        if not swapped:
            break
    return a

bubble_sort([5, 1, 4, 2, 8])
```

5. Part C: Selection Sort

Idea: find the **minimum** element in the unsorted part and **put it at the front** (swap it with the first unsorted element). Repeat for the rest. Like choosing the shortest student and sending them to the front of the line, again and again.

Pseudocode

```
ALGORITHM SelectionSort(A)
  FOR i ← 0 TO n - 2 DO
    min_index ← i
    FOR j ← i + 1 TO n - 1 DO
      IF A[j] < A[min_index] THEN min_index ← j
    swap A[i], A[min_index]
```

Dry run on [64, 25, 12, 22, 11]

i	Minimum found	Array after swap
0	11	[11, 25, 12, 22, 64]
1	12	[11, 12, 25, 22, 64]
2	22	[11, 12, 22, 25, 64]
3	25	[11, 12, 22, 25, 64]

```
def selection_sort(arr):
    a = arr[:]
    n = len(a)
    for i in range(n - 1):
        min_index = i
        for j in range(i + 1, n):
            if a[j] < a[min_index]:
                min_index = j
        a[i], a[min_index] = a[min_index], a[i]
        print(f"i = {i}: {a}")
    return a

selection_sort([64, 25, 12, 22, 11])
```



Selection sort's special property

It makes **at most $n - 1$ swaps**, the fewest of all three. This is useful when writing to memory is expensive (e.g., flash memory or EEPROM in embedded devices).



6. Part D: Insertion Sort

Idea: build the sorted part one element at a time. Take the next element (the **key**), **shift** bigger elements of the sorted part one place to the right, and **insert** the key into the gap. This is exactly how you arrange playing cards in your hand.

Pseudocode

```
ALGORITHM InsertionSort(A)
  FOR i ← 1 TO n - 1 DO
    key ← A[i] ; j ← i - 1
    WHILE j ≥ 0 AND A[j] > key DO
      A[j + 1] ← A[j]           // shift right
      j ← j - 1
    A[j + 1] ← key              // insert
```

Dry run on [12, 11, 13, 5, 6]

i	key	Array after inserting key	Sorted part
1	11	[11, 12, 13, 5, 6]	[11, 12]
2	13	[11, 12, 13, 5, 6]	[11, 12, 13]
3	5	[5, 11, 12, 13, 6]	[5, 11, 12, 13]
4	6	[5, 6, 11, 12, 13]	all

```
def insertion_sort(arr):  
    a = arr[:]  # Create a copy of the array  
    for i in range(1, len(a)):  
        key = a[i]  
        j = i - 1  
        while j >= 0 and a[j] > key:  
            a[j + 1] = a[j]      # shift  
            j -= 1  
        a[j + 1] = key          # insert  
        print(f"i = {i}, key = {key:>2}: {a}")  
    return a  
  
insertion_sort([12, 11, 13, 5, 6])
```



7. Part E: Analysis and Comparison

7.1 Counting the Work

For bubble and selection sort, the inner loop runs $(n-1) + (n-2) + \dots + 1 = \mathbf{n(n-1)/2}$ times in the worst case, which is $\mathbf{O(n^2)}$. For insertion sort, the worst case (reverse-sorted input) also shifts $n(n-1)/2$ times, but on **already sorted** input the while loop never runs, so it is $\mathbf{O(n)}$.

Algorithm	Best	Average	Worst	Space	Stable?	Swaps (worst)
Bubble (early stop)	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	$O(n^2)$
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	$O(n)$
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	$O(n^2)$ shifts

```
import random

def count_ops(sort_name, arr):
    a = arr[:]; n = len(a); comps = writes = 0
    if sort_name == "bubble":
        for i in range(n - 1):
            swapped = False
            for j in range(n - 1 - i):
                comps += 1
                if a[j] > a[j + 1]:
                    a[j], a[j + 1] = a[j + 1], a[j]; writes += 1; swapped = True
            if not swapped:
                break
    elif sort_name == "selection":
        for i in range(n - 1):
            m = i
            for j in range(i + 1, n):
                comps += 1
                if a[j] < a[m]:
                    m = j
            if m != i:
                a[i], a[m] = a[m], a[i]; writes += 1
    else:
        for i in range(1, n):
            key, j = a[i], i - 1
            while j >= 0:
                comps += 1
                if a[j] <= key:
                    break
                a[j + 1] = a[j]; writes += 1; j -= 1
            a[j + 1] = key
    assert a == sorted(arr)
    return comps, writes

n = 500
inputs = {"random": random.sample(range(n), n), "sorted": list(range(n)), "reversed": list(range(n, 0, -1))}
print(f"{'input':<9} {'algorithm':<10} {'comparisons':>12} {'swaps/shifts':>13}")
for name, data in inputs.items():
    for alg in ["bubble", "selection", "insertion"]:
        c, w = count_ops(alg, data)
        print(f"{'name':<9} {'alg':<10} {'c':>12,} {'w':>13,}")
```

Look at the “sorted” rows: bubble and insertion do only $n - 1$ comparisons, while selection still does $n(n-1)/2$. That is why insertion sort is the champion for nearly-sorted data.

7.2 Stability: Why It Matters

Suppose the student list is already in roll-number order, and we sort by marks. A **stable** sort keeps students with equal marks in roll-number order; an unstable sort may mix them up.

```
students = [("Ali", 80), ("Sara", 95), ("Hamza", 80), ("Zain", 70)]

def selection_sort_by_marks(recs):
    a = recs[:]
    for i in range(len(a) - 1):
        m = i
        for j in range(i + 1, len(a)):
            if a[j][1] < a[m][1]:
                m = j
        a[i], a[m] = a[m], a[i]
    return a

print("Selection (unstable):", selection_sort_by_marks(students))
print("sorted() (stable) :", sorted(students, key=lambda s: s[1]))
```

Selection sort moved Hamza **before** Ali (the long swap jumped over Ali), even though Ali came first originally. Python’s `sorted()` is guaranteed stable.



8. Part F: Sorting in Real Python Code

In real projects, use the built-in tools. `sorted(list)` returns a new sorted list; `list.sort()` sorts in place. Both are **$O(n \log n)$** and stable. The `key` parameter tells Python **what to sort by**.

```
students = [
    {"name": "Ali", "cgpa": 3.4, "age": 20},
    {"name": "Sara", "cgpa": 3.9, "age": 21},
    {"name": "Hamza", "cgpa": 3.4, "age": 19},
]
by_cgpa = sorted(students, key=lambda s: s["cgpa"], reverse=True)
print([s["name"] for s in by_cgpa])

# Multiple keys: CGPA high to low, then age low to high
ranked = sorted(students, key=lambda s: (-s["cgpa"], s["age"]))
print([s["name"] for s in ranked])

words = ["banana", "Apple", "cherry"]
print(sorted(words), sorted(words, key=str.lower), sorted(words, key=len))
```

9. C++ Corner

C++

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

void insertionSort(vector<int>& a) {
    for (size_t i = 1; i < a.size(); i++) {
        int key = a[i];
        int j = (int)i - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }
}

int main() {
    vector<int> a = {12, 11, 13, 5, 6};
    insertionSort(a);
    for (int x : a) cout << x << " ";
    cout << endl;

    vector<pair<string,int>> s = {"Ali",80},{"Sara",95},{"Hamza",80}};
    stable_sort(s.begin(), s.end(),
        [](auto& x, auto& y){ return x.second > y.second; });
    for (auto& p : s) cout << p.first << " " << p.second << endl;
}
```

10. Common Mistakes

Mistake	Effect	Fix
Inner loop <code>range(n)</code> in bubble sort	<code>a[j+1]</code> IndexError	Use <code>range(n - 1 - i)</code>
Swapping inside selection sort's inner loop	Becomes a slower, different algorithm	Swap once after finding the minimum
<code>while a[j] > key and j >= 0</code>	Checks <code>a[-1]</code> when <code>j = -1</code>	Check <code>j >= 0</code> first
<code>result = list.sort()</code>	result is <code>None</code>	Use <code>sorted(list)</code> or sort then use the list

Mistake	Effect	Fix
Assuming every sort is stable	Wrong order of equal records	Use a stable sort or add a tie-break key

11. Lab Tasks

Solve all tasks in `Lab05_Student.ipynb`. Each task has a **test cell**: when it prints  your solution is correct. For each task, **write pseudocode first**, then code, then test with your own inputs including edge cases. Difficulty:  Easy  Medium  Hard.

Task 1 Bubble Sort with Statistics

Write `bubble_sort_stats(arr)` that returns `(sorted_list, passes, swaps)` using bubble sort with early stopping. Do not modify the input list.

```
bubble_sort_stats([5, 1, 4, 2, 8]) → ([1, 2, 4, 5, 8], 3, 4)
```

Task 2 Merit List with Selection Sort

Write `merit_list(records)` that sorts `(name, marks)` tuples by marks in **descending** order using **your own selection sort** (no `sorted/sort`). Return the list of names.

```
[("Ali", 72), ("Sara", 91), ("Usman", 85)] → ["Sara", "Usman", "Ali"]
```

Task 3 Stable Insertion Sort for Records

Write `insertion_sort_by(records, key)` that sorts a list of dictionaries by `record[key]` in ascending order using insertion sort. It must be **stable**. Return a new list.

```
by "marks": [{"n": "Ali", "marks": 80}, {"n": "Sara", "marks": 70},
{"n": "Hamza", "marks": 80}]
→ Sara(70), Ali(80), Hamza(80)    # Ali stays before Hamza
```

Task 4 Sort Colours (Dutch National Flag)

An array contains only 0s, 1s and 2s (e.g., red, white, blue balls). Sort it **in one pass** and **in place** without any sorting function.

Hint: keep three pointers `low`, `mid`, `high`. 0 → swap with low; 1 → move on; 2 → swap with high.

```
[2, 0, 2, 1, 1, 0] → [0, 0, 1, 1, 2, 2]
```

Task 5 🟡 Rank Students by Multiple Keys

Using Python's `sorted()` with a **key**, rank students by **CGPA (high → low)**, then **attendance (high → low)**, then **name (A → Z)**. Return the list of names.

```
[("Bilal",3.5,90), ("Ayesha",3.8,85), ("Ahmad",3.5,95), ("Zara",3.5,90)]
→ ["Ayesha", "Ahmad", "Bilal", "Zara"]
```

Task 6 🔴 Count Inversions (How Unsorted Is It?)

An **inversion** is a pair (i, j) with $i < j$ and $a[i] > a[j]$. The number of inversions measures how far an array is from sorted, and it is exactly the number of **shifts insertion sort performs**.

Write `count_inversions(arr)` by instrumenting insertion sort (count the shifts). Then verify it against a brute-force $O(n^2)$ double loop.

```
[2, 4, 1, 3, 5] → 3    # (2,1), (4,1), (4,3)
```

Task 7 🔴 Nearly Sorted Data

A sensor log is **almost sorted** by time: each reading is at most **k** positions away from its correct position. Show experimentally that insertion sort is fast here.

Write `make_nearly_sorted(n, k)` that returns a list of $0..n-1$ where elements are shuffled only within blocks of size **k**, and `insertion_shifts(arr)` that returns the number of shifts. Show that $\text{shifts} \leq n \cdot k$, far less than $n^2/4$ for random data.

🏆 12. LeetCode Practice

Solve these on **leetcode.com** and upload screenshots of the “Accepted” result to your GitHub **DSA-Labs/Lab05** folder.

#	Problem	Level	Concept practised
1051	Height Checker	Easy	Compare with sorted order
977	Squares of a Sorted Array	Easy	Sorting / two pointers
242	Valid Anagram	Easy	Sort characters
1122	Relative Sort Array	Easy	Custom sort key
75	Sort Colors	Medium	Dutch national flag (Task 4)
912	Sort an Array	Medium	Implement a sort (try insertion, see the time limit!)



13. Home Task

1. Finish all tasks and push `Lab05_Student.ipynb` to GitHub.
2. Implement **Cocktail Shaker Sort** (bubble sort that goes left-to-right then right-to-left) and compare its passes with bubble sort.
3. Modify insertion sort to use **binary search** to find the insert position. Does it reduce comparisons? Does it reduce shifts? Explain.
4. **Mini-project: PSL Points Table.** Store at least 6 teams with points, net run rate and wins. Sort by points, then NRR, then wins using (a) your own insertion sort with a comparison function and (b) `sorted()` with a tuple key. Print a formatted table.
5. Solve at least 3 LeetCode problems from the list.

? 14. Viva Questions

1. Explain bubble sort in your own words. Why is it called “bubble”?
2. How does early stopping make bubble sort $O(n)$ in the best case?
3. Why does selection sort always take $O(n^2)$ comparisons?
4. Which elementary sort makes the fewest swaps? When is that useful?
5. Why is insertion sort good for nearly-sorted data?
6. What is a stable sort? Give an example where stability matters.
7. Is selection sort stable? Show a counter-example.
8. What does “in-place” mean?
9. What is the difference between `sorted(a)` and `a.sort()`?
10. Why do Timsort and Introsort use insertion sort internally?



15. Assessment Rubric

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Correctness	All tasks pass all tests	Most tasks correct	Some tasks correct	Mostly incorrect
Logic & pseudocode	Clear pseudocode and dry run for every task	Pseudocode for most tasks	Partial pseudocode	None

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Code quality	Meaningful names, functions, comments	Mostly clean	Hard to read	Copied or unreadable
Complexity & edge cases	States Big-O and handles all edge cases	Some edge cases / Big-O	Only basic cases	Not considered
Viva	Explains every line confidently	Explains most parts	Basic understanding	Cannot explain

Lab Rules

- Bring your own laptop with Colab or Jupyter working.
- Arrive on time; late arrival may result in no attendance for the lab.
- Copying another student's code or pasting AI output without understanding it will be awarded zero. You must be able to explain every line in the viva.

16. References and Further Learning

- Sorting HOW TO (Python docs): docs.python.org/3/howto/sorting.html
- Animated sorting algorithms: visualgo.net/en/sorting
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, **Introduction to Algorithms** (CLRS), 4th ed., MIT Press, 2022.
- M. T. Goodrich, R. Tamassia, M. H. Goldwasser, **Data Structures and Algorithms in Python**, Wiley.
- MIT OpenCourseWare 6.006 Introduction to Algorithms; Harvard CS50x (free online courses).
- Visualise code step by step: pythontutor.com; animated data structures: visualgo.net.



About the Author

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

GitHub: github.com/RamzanPUCIT

LinkedIn: linkedin.com/in/ramzan-ai

ResearchGate: researchgate.net/profile/Muhammad-Ramzan-105

© 2026 Muhammad Ramzan. Prepared for the Department of Computer Science, University of Okara.

UNIVERSITY OF OKARA

Department of Computer Science

DATA STRUCTURES & ALGORITHMS

Lab Manual

LAB 06 | Recursion, Backtracking and Divide & Conquer

Week 6 of 16 • Duration: 3 hours (in-lab) + home practice

Prepared by

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

Applicable to: BSCS • BSIT • BSSE • BSAI • BSDS • BS Cyber Security and all programs offering DSA

Fall 2026 Edition

About This Lab Manual

This manual is part of a **16-week Data Structures & Algorithms (DSA) lab series**. It is designed so that a student starting with **zero programming knowledge** in Week 1 can confidently design, implement and analyse data structures by Week 16. Every lab builds on the previous one, so please do not skip labs.

Language policy: concepts are explained in a language-independent way (diagrams, pseudocode, dry runs). Full implementations are in **Python**, and every lab includes a **C++ Corner** so instructors teaching in C++ can use the same manual.

Every lab follows the same structure

Section	What you get
 Objectives & CLOs	What you will be able to do after this lab
 Motivation	Where this topic is used in real life, industry, research and the future (2026–2030)
 Concepts	Simple explanations with real examples, pseudocode and dry runs
 Code	Python implementation + C++ Corner
 Lab Tasks	Graded tasks:  Easy,  Medium,  Hard
 LeetCode	Real interview problems to practise
 Home Task &  Viva	Practice at home and prepare for oral questions
 Notebooks	Instructor notebook (complete code) and Student notebook (with TODO blanks and auto-check tests)

Lab 06 at a glance

Item	Details
Lab title	Recursion, Backtracking and Divide & Conquer
Week	6 of 16
Prerequisites	Lab 02 (Big-O), Lab 04 (Binary Search)
Tools	Google Colab / Jupyter / VS Code (Python 3); any C++ compiler for the C++ Corner
Files	Lab06_Instructor.ipynb , Lab06_Student.ipynb
Next lab	Lab 07: Efficient Sorting (Merge Sort, Quick Sort, Counting Sort)

1. Learning Objectives

By the end of this lab, you will be able to:

1. Define recursion and identify the **base case** and the **recursive case** of a problem.
2. Trace recursive calls using the **call stack** and a **recursion tree**.
3. Write recursive solutions for classic problems: factorial, power, sum of digits, binary search, Tower of Hanoi.
4. Explain why naive recursive Fibonacci is **$O(2^n)$** and fix it with **memoization**.
5. Convert between recursion and iteration, and know Python's recursion limit.
6. Use **backtracking** to generate subsets and permutations.
7. Explain the **divide & conquer** strategy that powers merge sort and quick sort.

Course Learning Outcomes (CLO) mapping

CLO	Description	Taxonomy level
CLO-2	Implement recursive and divide-and-conquer algorithms	Apply (C3)
CLO-3	Analyse recursive algorithms using recursion trees	Analyse (C4)

2. Motivation: Solving Big Problems with Smaller Copies

Some problems are naturally made of **smaller versions of themselves**. A folder contains files and other folders, which contain files and other folders. A family tree has parents, who have parents. **Recursion** is a way to solve such problems by letting a function **call itself** on a smaller input.

Analogy: Russian dolls (Matryoshka)

To find the smallest doll, you open the big doll and repeat the **same action** on the doll inside. You stop when a doll cannot be opened (the **base case**). That is recursion.

2.1 Recursion in Daily Life and Computing

Example	Recursive structure
Folders on your computer	A folder contains folders → search all files recursively
Family tree / organisation chart	Each person has children who have children

Example	Recursive structure
Two mirrors facing each other	An image inside an image inside an image...
Fractals (ferns, snowflakes, coastlines)	Each part looks like the whole
HTML pages	Tags inside tags (a <code>div</code> inside a <code>div</code>)
Mathematics: $n! = n \times (n-1)!$	Defined using a smaller version of itself

2.2 Recursion in Industry

- **File systems and compilers** walk tree structures recursively (e.g., `os.walk`, parsing code into syntax trees).
- **Fast algorithms** such as Merge Sort, Quick Sort, Binary Search and the Fast Fourier Transform are built on divide & conquer.
- **Games and puzzles** (chess engines, Sudoku solvers) use recursive search with backtracking.
- **Web crawlers** visit a page, then recursively visit the pages it links to.

2.3 Recursion in Research

- **AI planning and game playing:** Minimax and Monte Carlo Tree Search (used in AlphaGo) explore game trees recursively.
- **Dynamic programming**, central to bioinformatics (DNA alignment) and reinforcement learning, starts from recursive formulas plus memoization.
- **Recursive neural networks** process sentences as trees in NLP.

2.4 Why It Matters in 2026–2030

Thinking recursively is a superpower

Trees, graphs, parsing and search, the backbone of AI systems, databases and compilers, are recursive by nature. Once you learn to **trust the recursion** (assume the smaller call works and focus on one step), complex problems become short, elegant programs. The rest of this course (trees, BST, heaps, graphs) depends heavily on recursion.

3. Part A: Anatomy of a Recursive Function

Every correct recursive function has **two parts**:

Part	Role	Factorial example
Base case	The smallest input, answered directly; it stops the recursion	$0! = 1$
Recursive case	Reduces the problem and calls itself on a smaller input	$n! = n \times (n - 1)!$

```
def factorial(n):
    if n == 0:                # base case
        return 1
    return n * factorial(n - 1) # recursive case (smaller input)

print(factorial(5))
```

⚠ No base case = infinite recursion

If the base case is missing or never reached, the function calls itself forever until Python stops it with `RecursionError: maximum recursion depth exceeded`.

3.1 The Call Stack

Each function call gets a **stack frame** that stores its own local variables. Calls are **pushed** onto the call stack and **popped** when they return (Last In, First Out, like the Stack you will build in Lab 10).

Winding (going down) and unwinding (coming back)

```
factorial(4)
-> 4 * factorial(3)
    -> 3 * factorial(2)
        -> 2 * factorial(1)
            -> 1 * factorial(0)
                -> 1                (base case)
            <- 1 * 1 = 1
        <- 2 * 1 = 2
    <- 3 * 2 = 6
<- 4 * 6 = 24
```

See the call stack in action

```
def factorial_trace(n, depth=0):
    indent = "    " * depth
    print(f"{indent}factorial({n}) called")
    if n == 0:
        print(f"{indent}base case -> 1")
        return 1
    result = n * factorial_trace(n - 1, depth + 1)
    print(f"{indent}factorial({n}) returns {result}")
    return result

factorial_trace(4)
```

Space complexity of recursion includes the call stack: `factorial(n)` keeps n frames at the deepest point, so it uses $O(n)$ space.

3.2 How to Think Recursively (The Leap of Faith)

1. **Define** exactly what the function returns, e.g., “`sum_list(a)` returns the sum of list `a`”.
2. **Base case:** what is the answer for the smallest input (empty list $\rightarrow 0$)?
3. **Trust** that the function already works for a smaller input (`sum_list(a[1:])`).
4. **Combine:** use that smaller answer to build the full answer (`a[0] + sum_list(a[1:])`).

```
def sum_list(a):
    if not a:
        # base case: empty list
        return 0
    return a[0] + sum_list(a[1:]) # first element + sum of the rest

def count_down(n):
    if n == 0:
        print("Go! 🚀")
        return
    print(n)
    count_down(n - 1)

print(sum_list([4, 8, 15, 16, 23, 42]))
count_down(3)
```



4. Part B: Classic Recursive Problems

4.1 Power: x^n

```
def power(x, n):                                # O(n)
    if n == 0:
        return 1
    return x * power(x, n - 1)

def fast_power(x, n):                            # O(log n): halve the exponent
    if n == 0:
        return 1
    half = fast_power(x, n // 2)
    return half * half if n % 2 == 0 else half * half * x

print(power(2, 10), fast_power(2, 10), fast_power(3, 13) == 3 ** 13)
```

4.2 Recursive Binary Search

```
def binary_search_rec(arr, target, low, high):
    if low > high:                                # base case: empty range
        return -1
    mid = (low + high) // 2
    if arr[mid] == target:
        return mid
    if arr[mid] < target:
        return binary_search_rec(arr, target, mid + 1, high)
    return binary_search_rec(arr, target, low, mid - 1)

data = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
print(binary_search_rec(data, 72, 0, len(data) - 1))
```

4.3 Tower of Hanoi

Move n discs from peg A to peg C using peg B, moving one disc at a time and never placing a bigger disc on a smaller one. The recursive insight: **move $n-1$ discs out of the way, move the biggest disc, then move the $n-1$ discs on top of it.**

```
def hanoi(n, source, helper, target):
    if n == 0:
        return 0
    moves = hanoi(n - 1, source, target, helper) # move n-1 to helper
    print(f"Move disc {n} from {source} to {target}")
    moves += 1
    moves += hanoi(n - 1, helper, source, target) # move n-1 onto target
    return moves

print("Total moves:", hanoi(3, "A", "B", "C"))
```

The number of moves is $2^n - 1$. For 64 discs (the legend), that is about 1.8×10^{19} moves: at one move per second it would take around 585 billion years. A perfect example of **exponential** growth.



5. Part C: Recursion Trees and Memoization

5.1 Fibonacci: The Danger of Repeated Work

Fibonacci numbers: $F(0) = 0$, $F(1) = 1$, $F(n) = F(n-1) + F(n-2)$. The direct recursive code is beautiful but **extremely slow**.

Recursion tree for fib(5)

```

fib(5)
├── fib(4)
│   ├── fib(3)
│   │   ├── fib(2) — fib(1), fib(0)
│   │   └── fib(1)
│   └── fib(2) — fib(1), fib(0)
└── fib(3)
    ├── fib(2) — fib(1), fib(0)
    └── fib(1)

```

<- fib(2) computed again
 <- whole fib(3) computed again!

```

calls = 0
def fib_naive(n):
    global calls
    calls += 1
    if n < 2:
        return n
    return fib_naive(n - 1) + fib_naive(n - 2)

for n in [10, 20, 25]:
    calls = 0
    value = fib_naive(n)
    print(f"fib({n}) = {value:>6}  ->  {calls:>7,} function calls")

```

The number of calls roughly **doubles** with each increase of n : the complexity is $O(2^n)$. `fib_naive(50)` would need about 40 billion calls.

5.2 Memoization: Remember What You Already Computed

Memoization stores the result of each call in a dictionary. Before computing, check whether the answer is already known. Each $F(k)$ is computed **once**, so the complexity drops to $O(n)$.

```
def fib_memo(n, memo=None):
    if memo is None:
        memo = {}
    if n in memo:
        return memo[n]          # already computed: O(1)
    if n < 2:
        return n
    memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
    return memo[n]

from functools import lru_cache

@lru_cache(maxsize=None)      # Python's built-in memoization
def fib_cached(n):
    return n if n < 2 else fib_cached(n - 1) + fib_cached(n - 2)

print(fib_memo(80), fib_cached(80))
```

5.3 Recursion vs Iteration

```
def fib_iter(n):                # O(n) time, O(1) space, no recursion
    limit
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a

import sys
print(fib_iter(80), "| Python recursion limit:", sys.getrecursionlimit())
```

	Recursion	Iteration (loops)
Code	Short, close to the mathematical definition	Sometimes longer
Memory	O(depth) for the call stack	Usually O(1)
Risk	Stack overflow / RecursionError (~1000 deep in Python)	Infinite loop
Best for	Trees, graphs, divide & conquer, backtracking	Simple repetition, very deep processes



6. Part D: Backtracking

Backtracking builds a solution step by step, explores one choice, and **undoes** (backtracks) that choice to try the next one. It systematically explores all possibilities.

Generate all subsets (e.g., all possible combinations of toppings)

```
def subsets(items):
    result = []
    current = []

    def explore(i):
        if i == len(items):                # decided for every item
            result.append(current[:])
            return
        current.append(items[i])           # choice 1: include items[i]
        explore(i + 1)
        current.pop()                      # undo (backtrack)
        explore(i + 1)                     # choice 2: exclude items[i]

    explore(0)
    return result

print(subsets(["cheese", "olives", "chicken"]))
```

Each item has 2 choices, so n items give 2^n subsets. Backtracking is exponential, but it is often the only exact method, and it can be sped up by **pruning** branches that cannot lead to a solution (e.g., in Sudoku).



7. Part E: Divide & Conquer

Divide & conquer solves a problem in three steps:

1. **Divide** the problem into smaller sub-problems (usually halves).
2. **Conquer** each sub-problem recursively.
3. **Combine** the sub-answers into the final answer.

```
def max_dc(a, low, high):
    if low == high:                        # one element
        return a[low]
    mid = (low + high) // 2
    left_max = max_dc(a, low, mid)         # conquer left half
    right_max = max_dc(a, mid + 1, high)   # conquer right half
    return left_max if left_max > right_max else right_max  # combine

marks = [67, 89, 45, 92, 78, 55, 81]
print(max_dc(marks, 0, len(marks) - 1))
```

Algorithm	Divide	Combine	Complexity
Binary search	Pick one half	Nothing	$O(\log n)$
Max by D&C	Two halves	Compare two values	$O(n)$
Merge sort (Lab 07)	Two halves	Merge two sorted halves: $O(n)$	$O(n \log n)$
Quick sort (Lab 07)	Partition around a pivot	Nothing	$O(n \log n)$ average

8. C++ Corner

C++

```
#include <iostream>
#include <unordered_map>
using namespace std;

long long factorial(int n) {
    if (n == 0) return 1;           // base case
    return n * factorial(n - 1);    // recursive case
}

unordered_map<int, long long> memo;
long long fib(int n) {
    if (n < 2) return n;
    if (memo.count(n)) return memo[n];
    return memo[n] = fib(n - 1) + fib(n - 2);
}

void hanoi(int n, char from, char via, char to) {
    if (n == 0) return;
    hanoi(n - 1, from, to, via);
    cout << "Move disc " << n << " from " << from << " to " << to << endl;
    hanoi(n - 1, via, from, to);
}

int main() {
    cout << factorial(10) << " " << fib(80) << endl;
    hanoi(3, 'A', 'B', 'C');
}
```

9. Common Mistakes

Mistake	Effect	Fix
Missing base case	RecursionError	Always write the base case first
Input does not get smaller	Infinite recursion	Make sure each call moves towards the base case
Forgetting <code>return</code> before the recursive call	Function returns <code>None</code>	<code>return f(n - 1)</code>
Repeated sub-problems without memo	Exponential time	Use a dictionary or <code>@lru_cache</code>
Using a mutable default <code>memo={}</code>	Memo shared between unrelated calls	Use <code>memo=None</code> then create it
Very deep recursion in Python	Crashes around depth 1000	Use iteration for long linear processes

10. Lab Tasks

Solve all tasks in `Lab06_Student.ipynb`. Each task has a **test cell**: when it prints  your solution is correct. For each task, **write pseudocode first**, then code, then test with your own inputs including edge cases. Difficulty:  Easy  Medium  Hard.

Task 1 Sum of Digits

Write a **recursive** function `digit_sum(n)` that returns the sum of the digits of a non-negative integer. No loops, no `str()`.

```
digit_sum(2026) → 10
```

Task 2 Reverse a String Recursively

Write `reverse(s)` recursively, without slicing tricks like `[::-1]` and without loops. Hint: reverse of `s` = reverse of `s[1:]` + `s[0]`.

```
reverse("Okara") → "arakO"
```

Task 3 Count Files in a Folder Tree

A folder is represented as a dictionary: keys are names; a value of `None` means a file, and a dictionary means a sub-folder. Write `count_files(folder)` and `total_depth(folder)` (the maximum nesting depth; a folder with only files has depth 1).

This is exactly how your operating system walks directories.

```
{"notes.txt": None, "labs": {"lab1.py": None, "data": {"a.csv": None}}}
count_files → 3, total_depth → 3
```

Task 4 Grid Paths with Memoization

A delivery rider starts at the top-left of an $r \times c$ grid of streets and can move only **right** or **down**. Some cells are blocked (`1`). Count the number of paths to the bottom-right.

Write the recursive solution `paths(grid)`, then add memoization so that a 15×15 grid finishes instantly.

```
grid = [[0,0,0],
        [0,1,0],
        [0,0,0]] → 2
```

Task 5 Tower of Hanoi Move List

Write `hanoi_moves(n, a, b, c)` that **returns** the list of moves as tuples (`disc`, `from`, `to`) instead of printing. Verify that the length is $2^n - 1$ and that no move ever places a bigger disc on a smaller one (write a simulator).

```
hanoi_moves(2, "A", "B", "C") → [(1, "A", "B"), (2, "A", "C"), (1, "B", "C")]
```

Task 6 Permutations (Backtracking)

Write `permutations(items)` that returns all orderings of a list using backtracking with a `used` list. Do not use `itertools`. The result order should be lexicographic if the input is sorted.

```
permutations([1, 2, 3]) → [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]
```

Task 7 Flood Fill: Count Lakes on a Map

A satellite map is a grid where `1` is water and `0` is land. Water cells connected **up, down, left or right** form one lake. Write `count_lakes(grid)` using a recursive `fill(i, j)` that marks a whole lake as visited.

This is the same algorithm used by the paint-bucket tool in MS Paint and by region detection in remote-sensing images.

```
[[1,1,0,0],
 [1,0,0,1],
 [0,0,1,1],
 [0,0,0,0]] → 2
```

11. LeetCode Practice

Solve these on **leetcode.com** and upload screenshots of the “Accepted” result to your GitHub `DSA-Labs/Lab06` folder.

#	Problem	Level	Concept practised
509	Fibonacci Number	Easy	Recursion + memoization
344	Reverse String	Easy	Recursion with two pointers
231	Power of Two	Easy	Recursive halving
70	Climbing Stairs	Easy	Recursion → memoization

#	Problem	Level	Concept practised
50	Pow(x, n)	Medium	Fast power, divide & conquer
78	Subsets	Medium	Backtracking
46	Permutations	Medium	Backtracking (Task 6)
200	Number of Islands	Medium	Flood fill (Task 7)

12. Home Task

1. Finish all tasks and push `Lab06_Student.ipynb` to GitHub.
2. Write recursive functions for: palindrome check, GCD (Euclid's algorithm), and printing all binary strings of length n.
3. Draw the recursion tree of `fib_naive(6)` on paper and count the calls. Then explain how memoization changes the tree.
4. **Mini-project: Sudoku Solver.** Write a backtracking solver for a 9×9 Sudoku. Count how many recursive calls it makes. Try to reduce them with pruning (check row, column and box before placing a digit).
5. Solve at least 3 LeetCode problems from the list.

13. Viva Questions

1. What is recursion? What are the base case and recursive case?
2. What happens if a recursive function has no base case?
3. Explain the call stack using `factorial(3)`.
4. What is the space complexity of a recursive function with depth n?
5. Why is naive recursive Fibonacci $O(2^n)$?
6. What is memoization? How does it change Fibonacci's complexity?
7. When would you choose iteration over recursion?
8. How many moves does Tower of Hanoi need for n discs?
9. What is backtracking? How many subsets does a set of n items have?
10. Explain the three steps of divide & conquer with an example.



14. Assessment Rubric

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Correctness	All tasks pass all tests	Most tasks correct	Some tasks correct	Mostly incorrect
Logic & pseudocode	Clear pseudocode and dry run for every task	Pseudocode for most tasks	Partial pseudocode	None
Code quality	Meaningful names, functions, comments	Mostly clean	Hard to read	Copied or unreadable
Complexity & edge cases	States Big-O and handles all edge cases	Some edge cases / Big-O	Only basic cases	Not considered
Viva	Explains every line confidently	Explains most parts	Basic understanding	Cannot explain

Lab Rules

- Bring your own laptop with Colab or Jupyter working.
- Arrive on time; late arrival may result in no attendance for the lab.
- Copying another student's code or pasting AI output without understanding it will be awarded zero. You must be able to explain every line in the viva.



15. References and Further Learning

- `functools.lru_cache`: docs.python.org/3/library/functools.html
- Step through recursive calls visually at pythontutor.com
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, **Introduction to Algorithms** (CLRS), 4th ed., MIT Press, 2022.
- M. T. Goodrich, R. Tamassia, M. H. Goldwasser, **Data Structures and Algorithms in Python**, Wiley.
- MIT OpenCourseWare 6.006 Introduction to Algorithms; Harvard CS50x (free online courses).
- Visualise code step by step: pythontutor.com; animated data structures: visualgo.net.



About the Author

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

GitHub: github.com/RamzanPUCIT

LinkedIn: linkedin.com/in/ramzan-ai

ResearchGate: researchgate.net/profile/Muhammad-Ramzan-105

© 2026 Muhammad Ramzan. Prepared for the Department of Computer Science, University of Okara.

UNIVERSITY OF OKARA

Department of Computer Science

DATA STRUCTURES & ALGORITHMS

Lab Manual

LAB 07 | Efficient Sorting: Merge Sort, Quick Sort and Counting Sort

Week 7 of 16 • Duration: 3 hours (in-lab) + home practice

Prepared by

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

Applicable to: BSCS • BSIT • BSSE • BSAI • BSDS • BS Cyber Security and all programs offering DSA

Fall 2026 Edition

About This Lab Manual

This manual is part of a **16-week Data Structures & Algorithms (DSA) lab series**. It is designed so that a student starting with **zero programming knowledge** in Week 1 can confidently design, implement and analyse data structures by Week 16. Every lab builds on the previous one, so please do not skip labs.

Language policy: concepts are explained in a language-independent way (diagrams, pseudocode, dry runs). Full implementations are in **Python**, and every lab includes a **C++ Corner** so instructors teaching in C++ can use the same manual.

Every lab follows the same structure

Section	What you get
 Objectives & CLOs	What you will be able to do after this lab
 Motivation	Where this topic is used in real life, industry, research and the future (2026–2030)
 Concepts	Simple explanations with real examples, pseudocode and dry runs
 Code	Python implementation + C++ Corner
 Lab Tasks	Graded tasks:  Easy,  Medium,  Hard
 LeetCode	Real interview problems to practise
 Home Task &  Viva	Practice at home and prepare for oral questions
 Notebooks	Instructor notebook (complete code) and Student notebook (with TODO blanks and auto-check tests)

Lab 07 at a glance

Item	Details
Lab title	Efficient Sorting: Merge Sort, Quick Sort and Counting Sort
Week	7 of 16
Prerequisites	Lab 05 (Elementary Sorting), Lab 06 (Recursion & Divide and Conquer)
Tools	Google Colab / Jupyter / VS Code (Python 3); any C++ compiler for the C++ Corner
Files	Lab07_Instructor.ipynb , Lab07_Student.ipynb
Next lab	Lab 08: Singly Linked List

1. Learning Objectives

By the end of this lab, you will be able to:

1. Implement **Merge Sort** using divide & conquer and the merge procedure.
2. Explain why merge sort is **$O(n \log n)$** using a recursion tree.
3. Implement **Quick Sort** with the **Lomuto partition** and explain pivot choice.
4. Explain quick sort's **average $O(n \log n)$** and **worst $O(n^2)$** cases and how randomisation helps.
5. Implement **Counting Sort**, a non-comparison sort that runs in **$O(n + k)$** .
6. Use **Quickselect** to find the k-th smallest element in average $O(n)$.
7. Compare all sorting algorithms and choose the right one for a situation.

Course Learning Outcomes (CLO) mapping

CLO	Description	Taxonomy level
CLO-2	Implement efficient sorting algorithms	Apply (C3)
CLO-3	Analyse divide-and-conquer sorting algorithms	Analyse (C4)



2. Motivation: From Hours to Seconds

In Lab 05, sorting 2,000 numbers with bubble sort already took noticeable time. Imagine sorting the **crores of CNIC records** at NADRA, or all transactions of a bank in a day. $O(n^2)$ algorithms would take **days**. The algorithms in this lab, which are $O(n \log n)$, do it in **seconds**.

n	n^2 (bubble/insertion)	$n \log_2 n$ (merge/quick)	Speed-up
1,000	1,000,000	$\approx 10,000$	100×
1,000,000	10^{12}	$\approx 2 \times 10^7$	50,000×
100,000,000 (10 crore)	10^{16}	$\approx 2.7 \times 10^9$	≈ 3.7 million×



2.1 Efficient Sorting in Daily Life

- **Sorting exam papers:** teachers often split the pile among helpers, each sorts a small pile, then they **merge** piles. That is merge sort!
- **Organising a library:** pick a letter (pivot), put earlier books on the left and later books on the right, then repeat for each side. That is quick sort.

- **Counting votes:** count how many votes each candidate got, then list them. That is counting sort.



2.2 In Industry

- **Python's `sorted()` (Timsort)** is a hybrid of merge sort and insertion sort. Java uses Timsort for objects and a dual-pivot quick sort for primitive types.
- **C++ `std::sort` (Introsort)** starts with quick sort, switches to heap sort if recursion gets too deep, and uses insertion sort for small pieces.
- **External sorting:** when data is bigger than RAM (e.g., terabytes of logs), databases use merge sort on disk chunks.
- **Big data frameworks** (Hadoop MapReduce, Spark) sort data across many machines using merge-based ideas.



2.3 In Research

- **Parallel and GPU sorting** (e.g., radix sort on GPUs) is an active research area for AI and scientific computing.
- **Learned sorting:** recent research uses machine-learning models to predict where each element belongs, beating classic algorithms on some data.
- Sorting is a building block in **computational geometry** (convex hull) and **GIS** (spatial indexes).



2.4 Why It Matters in 2026–2030



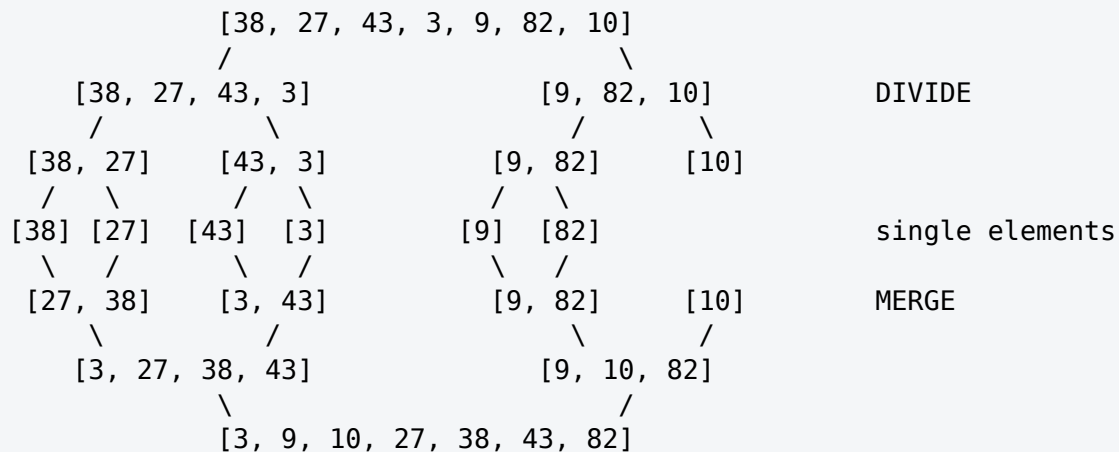
Divide & conquer scales to the cloud

Merge sort's idea, **split the work, solve pieces independently, then combine**, is exactly how modern distributed systems process huge datasets across thousands of computers. Learning it now prepares you for parallel programming, MapReduce and GPU computing.



3. Part A: Merge Sort

Idea: an array of size 1 is already sorted. So: **divide** the array into two halves, **sort each half recursively**, then **merge** the two sorted halves into one sorted array.

Merge sort on [38, 27, 43, 3, 9, 82, 10]

3.1 The Merge Step

You already wrote `merge_sorted` in Lab 03: compare the fronts of two sorted lists and repeatedly take the smaller one. It takes **$O(n)$** time for a total of n elements.

```

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:          # <= keeps merge sort STABLE
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

def merge_sort(arr):
    if len(arr) <= 1:                  # base case
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])       # sort left half
    right = merge_sort(arr[mid:])      # sort right half
    return merge(left, right)         # combine

print(merge_sort([38, 27, 43, 3, 9, 82, 10]))

```

3.2 Why $O(n \log n)$?

Look at the recursion tree. At **every level**, all n elements are merged once, which costs **$O(n)$** per level. The array is halved at each level, so there are **$\log_2 n$ levels**. Total: **$n \times \log n$** . This holds in the best, average and worst case.

Level	Sub-arrays	Size of each	Merge work at this level
0	1	n	n
1	2	n/2	n
2	4	n/4	n
...	...	...	...
log n	n	1	n

Property	Merge Sort
Time (all cases)	$O(n \log n)$
Extra space	$O(n)$ for merging
Stable?	Yes
In-place?	No
Great for	Linked lists, external (disk) sorting, when stability is needed

⚡ 4. Part B: Quick Sort

Idea: choose a **pivot** element. **Partition** the array so that everything smaller than the pivot is on its left and everything larger is on its right. Now the pivot is in its **final position**. Recursively sort the left and right parts. No merge step is needed.

4.1 Lomuto Partition

Pseudocode

```

ALGORITHM Partition(A, low, high)           // pivot = A[high]
    pivot ← A[high] ; i ← low - 1          // i = end of the "smaller"
    region
    FOR j ← low TO high - 1 DO
        IF A[j] ≤ pivot THEN
            i ← i + 1 ; swap A[i], A[j]
    swap A[i + 1], A[high]                  // put pivot in its final place
    RETURN i + 1
  
```

Dry run: partition [10, 80, 30, 90, 40, 50, 70] with pivot 70

j	A[j]	A[j] ≤ 70?	i	Array
0	10	Yes → swap A[0], A[0]	0	[10, 80, 30, 90, 40, 50, 70]
1	80	No	0	[10, 80, 30, 90, 40, 50, 70]
2	30	Yes → swap A[1], A[2]	1	[10, 30, 80, 90, 40, 50, 70]
3	90	No	1	[10, 30, 80, 90, 40, 50, 70]
4	40	Yes → swap A[2], A[4]	2	[10, 30, 40, 90, 80, 50, 70]
5	50	Yes → swap A[3], A[5]	3	[10, 30, 40, 50, 80, 90, 70]
end	-	swap A[4], pivot	-	[10, 30, 40, 50, 70 , 90, 80]

```
def partition(a, low, high):
    pivot = a[high]
    i = low - 1
    for j in range(low, high):
        if a[j] <= pivot:
            i += 1
            a[i], a[j] = a[j], a[i]
    a[i + 1], a[high] = a[high], a[i + 1]
    return i + 1

def quick_sort(a, low=0, high=None):
    if high is None:
        high = len(a) - 1
    if low < high:
        p = partition(a, low, high)          # pivot is now at index p
        quick_sort(a, low, p - 1)           # sort left of pivot
        quick_sort(a, p + 1, high)          # sort right of pivot
    return a

print(quick_sort([10, 80, 30, 90, 40, 50, 70]))
```

4.2 Best, Average and Worst Case

Case	When	Recursion depth	Time
Best / average	Pivot splits the array roughly in half	$\log n$	$O(n \log n)$
Worst	Pivot is always the smallest or largest (e.g., already sorted input with last-element pivot)	n	$O(n^2)$

Fix: choose a **random pivot** (or the median of three). Then the bad case becomes extremely unlikely for any input.

```
import random, sys

def quick_sort_random(a, low=0, high=None):
    if high is None:
        high = len(a) - 1
    if low < high:
        r = random.randint(low, high)      # random pivot
        a[r], a[high] = a[high], a[r]
        p = partition(a, low, high)
        quick_sort_random(a, low, p - 1)
        quick_sort_random(a, p + 1, high)
    return a

sys.setrecursionlimit(10_000)
sorted_input = list(range(2000))
print(quick_sort_random(sorted_input[:])[ :10], "... works fast even on sorted input")
```

Property	Quick Sort
Time	$O(n \log n)$ average, $O(n^2)$ worst
Extra space	$O(\log n)$ recursion stack (average)
Stable?	No
In-place?	Yes
Great for	General-purpose in-memory sorting; very fast in practice (cache-friendly)

5. Part C: Counting Sort (Non-Comparison)

Any algorithm that sorts only by **comparing** elements needs at least about $n \log n$ comparisons in the worst case. But if the values are **small integers in a known range** (e.g., marks 0–100, ages 0–120), we can avoid comparisons: just **count** how many times each value occurs.

```
def counting_sort(arr, max_value):
    count = [0] * (max_value + 1)
    for x in arr:
        count[x] += 1
    result = []
    for value in range(max_value + 1):
        result.extend([value] * count[value])
    return result

quiz_marks = [7, 3, 10, 3, 8, 7, 7, 0, 10, 5]
print(counting_sort(quiz_marks, 10))
```

count occurrences: $O(n)$
rebuild: $O(k)$

Time $O(n + k)$ where k is the range of values. Space $O(k)$. It beats $O(n \log n)$ when k is not much larger than n . Counting sort is the building block of **Radix Sort**, which sorts large numbers digit by digit.

6. Part D: Quickselect: The k-th Smallest Without Full Sorting

To find the median or the 3rd highest mark, sorting costs $O(n \log n)$. **Quickselect** uses partition but recurses into **only one side**, giving $O(n)$ on average.

```
def quickselect(a, k):
    # k-th smallest, k starts at 1
    a = a[:]
    low, high = 0, len(a) - 1
    while True:
        r = random.randint(low, high)
        a[r], a[high] = a[high], a[r]
        p = partition(a, low, high)
        if p == k - 1:
            return a[p]
        elif p < k - 1:
            low = p + 1
            # answer is on the right
        else:
            high = p - 1
            # answer is on the left

marks = [67, 89, 45, 92, 78, 55, 81]
print("median:", quickselect(marks, 4), "| 2nd smallest:",
      quickselect(marks, 2))
```



7. Part E: The Complete Sorting Comparison

Algorithm	Best	Average	Worst	Space	Stable	Notes
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Teaching only
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Fewest swaps
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Small / nearly sorted
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	Guaranteed, linked lists
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Fastest in practice
Heap (Lab 15)	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No	Guaranteed + in-place
Counting	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(k)$	Yes*	Small integer range
Timsort (Python)	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	Real-world default

* Stable when implemented with prefix sums (see the Home Task).

Race on 50,000 random integers between 0 and 100,000

```
import time
data = [random.randint(0, 100_000) for _ in range(50_000)]
for name, f in [("merge_sort", lambda d: merge_sort(d)),
                ("quick_sort_random", lambda d: quick_sort_random(d[:])),
                ("counting_sort", lambda d: counting_sort(d, 100_000)),
                ("sorted (Timsort)", lambda d: sorted(d))]:
    t = time.perf_counter(); out = f(data); e1 = time.perf_counter() - t
    assert out == sorted(data)
    print(f"{name:<18} {e1:.4f} s")
```

8. C++ Corner

C++

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

void mergeSort(vector<int>& a, int l, int r) {
    if (l >= r) return;
    int m = l + (r - l) / 2;
    mergeSort(a, l, m);
    mergeSort(a, m + 1, r);
    vector<int> tmp;
    int i = l, j = m + 1;
    while (i <= m && j <= r) tmp.push_back(a[i] <= a[j] ? a[i++] : a[j++]);
    while (i <= m) tmp.push_back(a[i++]);
    while (j <= r) tmp.push_back(a[j++]);
    for (int k = 0; k < (int)tmp.size(); k++) a[l + k] = tmp[k];
}

int partition(vector<int>& a, int low, int high) {
    int pivot = a[high], i = low - 1;
    for (int j = low; j < high; j++)
        if (a[j] <= pivot) swap(a[++i], a[j]);
    swap(a[i + 1], a[high]);
    return i + 1;
}

void quickSort(vector<int>& a, int low, int high) {
    if (low < high) {
        int p = partition(a, low, high);
        quickSort(a, low, p - 1);
        quickSort(a, p + 1, high);
    }
}

int main() {
    vector<int> a = {38, 27, 43, 3, 9, 82, 10}, b = a;
    mergeSort(a, 0, a.size() - 1);
    quickSort(b, 0, b.size() - 1);
    sort(b.begin(), b.end()); // the library version (Introsort)
    for (int x : a) cout << x << " ";
}
```

9. Common Mistakes

Mistake	Effect	Fix
Using <code><</code> instead of <code><=</code> in merge	Merge sort becomes unstable	Take from the left on ties
Forgetting to copy leftovers after merging	Missing elements	<code>extend(left[i:])</code> and <code>extend(right[j:])</code>
Quick sort with fixed last pivot on sorted data	$O(n^2)$ and RecursionError	Random or median-of-three pivot
Off-by-one in partition range	Infinite recursion	Recurse on <code>p - 1</code> and <code>p + 1</code> , never <code>p</code>
Lomuto partition on data with many duplicates	Degrades towards $O(n^2)$	Use three-way partition (<code><</code> , <code>=</code> , <code>></code> pivot)
Counting sort on huge ranges (e.g., CNIC numbers)	Huge memory	Use radix sort or comparison sort

10. Lab Tasks

Solve all tasks in `Lab07_Student.ipynb`. Each task has a **test cell**: when it prints  your solution is correct. For each task, **write pseudocode first**, then code, then test with your own inputs including edge cases. Difficulty:  Easy  Medium  Hard.

Task 1 Merge Sort with Merge Counting

Write `merge_sort_count(arr)` that returns `(sorted_list, merges)` where `merges` is the number of times the merge step is called. For n elements ($n \geq 1$) it should be exactly $n - 1$. Do not modify the input.

```
merge_sort_count([5, 2, 4, 1]) → ([1, 2, 4, 5], 3)
```

Task 2 Quick Sort in Descending Order

Modify `partition` and `quick_sort` into `partition_desc` and `quick_sort_desc` so that the array is sorted from **largest to smallest**, in place.

```
[3, 9, 1, 7] → [9, 7, 3, 1]
```

Task 3 Stable Counting Sort for Student Records

Sort `(name, marks)` records by marks (0–100) using counting sort with **prefix sums** so that it is **stable**.

Steps: count each mark; convert counts to prefix sums (positions); walk the input **from right to left**, placing each record at `count[mark] - 1` and decrementing.

```
[("Ali",80),("Sara",70),("Hamza",80),("Zoya",70)] → Sara, Zoya, Ali, Hamza
```

Task 4 Merge Overlapping Time Slots

The lab booking system has time slots as `[start, end]` (in minutes). Merge all overlapping slots and return them sorted by start time.

Hint: sort by start time ($O(n \log n)$), then scan once and merge.

```
[[1, 3], [8, 10], [2, 6], [15, 18]] → [[1, 6], [8, 10], [15, 18]]
```

Task 5 🟡 **k-th Highest Marks with Quickselect**

Write `kth_highest(marks, k)` that returns the k-th **highest** value in average $O(n)$ using the quickselect idea (do not sort the whole list). Use it to find the 3rd position holder's marks.

```
kth_highest([67, 89, 45, 92, 78], 2) → 89
```

Task 6 🔴 **Count Inversions in $O(n \log n)$**

In Lab 05 you counted inversions in $O(n^2)$. Do it in **$O(n \log n)$** by modifying merge sort: while merging, whenever you take an element from the **right** half, it forms an inversion with **every remaining element of the left half**.

Return `(sorted_list, inversions)`. Test on 100,000 elements.

```
[2, 4, 1, 3, 5] → ([1, 2, 3, 4, 5], 3)
```

Task 7 🔴 **Largest Number from Digits**

Given non-negative integers, arrange them to form the **largest possible number** and return it as a string.

Hint: a custom comparison: put x before y if `str(x) + str(y) > str(y) + str(x)`. Use `functools.cmp_to_key` with `sorted`, or adapt your merge sort to use this comparison.

```
[3, 30, 34, 5, 9] → "9534330"
```

**11. LeetCode Practice**

Solve these on **leetcode.com** and upload screenshots of the “Accepted” result to your GitHub `DSA-Labs/Lab07` folder.

#	Problem	Level	Concept practised
912	Sort an Array	Medium	Merge sort / quick sort
56	Merge Intervals	Medium	Sort + scan (Task 4)
215	Kth Largest Element in an Array	Medium	Quickselect (Task 5)
179	Largest Number	Medium	Custom comparator (Task 7)
1636	Sort Array by Increasing Frequency	Easy	Sorting with keys

#	Problem	Level	Concept practised
148	Sort List	Medium	Merge sort on a linked list (after Lab 08)



12. Home Task

1. Finish all tasks and push `Lab07_Student.ipynb` to GitHub.
2. Implement **bottom-up (iterative) merge sort** that merges sub-arrays of size 1, 2, 4, 8 ... without recursion.
3. Implement **Hoare's partition** scheme and compare the number of swaps with Lomuto on random data.
4. Implement **Radix Sort** for 6-digit roll numbers using stable counting sort on each digit.
5. **Mini-project: Sorting Visualiser.** Using matplotlib, draw the array as bars after every merge (merge sort) or every partition (quick sort) and save the frames (or animate them in Colab).



13. Viva Questions

1. Explain the three steps of merge sort.
2. Why is merge sort $O(n \log n)$ in all cases?
3. Why does merge sort need $O(n)$ extra space?
4. What is a pivot? What does partition guarantee?
5. When does quick sort take $O(n^2)$? How do we avoid it?
6. Why is quick sort often faster than merge sort in practice even though its worst case is worse?
7. Which of merge sort and quick sort is stable?
8. How does counting sort beat the $O(n \log n)$ lower bound?
9. What is quickselect and what is its average complexity?
10. What algorithm does Python's `sorted()` use?



14. Assessment Rubric

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Correctness	All tasks pass all tests	Most tasks correct	Some tasks correct	Mostly incorrect
Logic & pseudocode	Clear pseudocode and dry run for every task	Pseudocode for most tasks	Partial pseudocode	None
Code quality	Meaningful names, functions, comments	Mostly clean	Hard to read	Copied or unreadable
Complexity & edge cases	States Big-O and handles all edge cases	Some edge cases / Big-O	Only basic cases	Not considered
Viva	Explains every line confidently	Explains most parts	Basic understanding	Cannot explain



Lab Rules

- Bring your own laptop with Colab or Jupyter working.
- Arrive on time; late arrival may result in no attendance for the lab.
- Copying another student's code or pasting AI output without understanding it will be awarded zero. You must be able to explain every line in the viva.



15. References and Further Learning

- T. Peters, "Timsort" description:
github.com/python/cpython/blob/main/Objects/listsort.txt
- Sorting animations: visualgo.net/en/sorting
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, **Introduction to Algorithms** (CLRS), 4th ed., MIT Press, 2022.
- M. T. Goodrich, R. Tamassia, M. H. Goldwasser, **Data Structures and Algorithms in Python**, Wiley.
- MIT OpenCourseWare 6.006 Introduction to Algorithms; Harvard CS50x (free online courses).

- Visualise code step by step: **pythontutor.com**; animated data structures: **visualgo.net**.



About the Author

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

GitHub: github.com/RamzanPUCIT

LinkedIn: linkedin.com/in/ramzan-ai

ResearchGate: researchgate.net/profile/Muhammad-Ramzan-105

© 2026 Muhammad Ramzan. Prepared for the Department of Computer Science, University of Okara.

UNIVERSITY OF OKARA

Department of Computer Science

DATA STRUCTURES & ALGORITHMS

Lab Manual

LAB 08 | Singly Linked List: Nodes, Pointers and Dynamic Memory

Week 8 of 16 • Duration: 3 hours (in-lab) + home practice

Prepared by

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

Applicable to: BSCS • BSIT • BSSE • BSAI • BSDS • BS Cyber Security and all programs offering DSA

Fall 2026 Edition

About This Lab Manual

This manual is part of a **16-week Data Structures & Algorithms (DSA) lab series**. It is designed so that a student starting with **zero programming knowledge** in Week 1 can confidently design, implement and analyse data structures by Week 16. Every lab builds on the previous one, so please do not skip labs.

Language policy: concepts are explained in a language-independent way (diagrams, pseudocode, dry runs). Full implementations are in **Python**, and every lab includes a **C++ Corner** so instructors teaching in C++ can use the same manual.

Every lab follows the same structure

Section	What you get
 Objectives & CLOs	What you will be able to do after this lab
 Motivation	Where this topic is used in real life, industry, research and the future (2026–2030)
 Concepts	Simple explanations with real examples, pseudocode and dry runs
 Code	Python implementation + C++ Corner
 Lab Tasks	Graded tasks:  Easy,  Medium,  Hard
 LeetCode	Real interview problems to practise
 Home Task &  Viva	Practice at home and prepare for oral questions
 Notebooks	Instructor notebook (complete code) and Student notebook (with TODO blanks and auto-check tests)

Lab 08 at a glance

Item	Details
Lab title	Singly Linked List: Nodes, Pointers and Dynamic Memory
Week	8 of 16
Prerequisites	Lab 01 (references in memory), Lab 03 (arrays, classes)
Tools	Google Colab / Jupyter / VS Code (Python 3); any C++ compiler for the C++ Corner
Files	Lab08_Instructor.ipynb , Lab08_Student.ipynb
Next lab	Lab 09: Doubly and Circular Linked Lists

1. Learning Objectives

By the end of this lab, you will be able to:

1. Explain how a linked list stores data in **nodes connected by references (pointers)**.
2. Compare linked lists with arrays in terms of memory and operation costs.
3. Implement a **Node** class and a **LinkedList** class with **head**, **tail** and **size**.
4. Implement **traversal**, **search**, **insertion** (head, tail, position) and **deletion** (head, value, position).
5. **Reverse** a linked list in place by re-linking pointers.
6. Use the **slow/fast pointer** technique to find the middle and to detect a cycle.
7. Solve classic interview problems on linked lists.

Course Learning Outcomes (CLO) mapping

CLO	Description	Taxonomy level
CLO-1	Explain the structure of linked lists	Understand (C2)
CLO-2	Implement singly linked list operations	Apply (C3)
CLO-3	Compare arrays and linked lists	Analyse (C4)

2. Motivation: When Arrays Are Not Enough

Arrays are fast for access, but they have two big problems: their size is fixed (or resizing needs copying), and **inserting or deleting in the middle requires shifting** many elements. A **linked list** solves this by not storing elements side by side at all. Each element simply **remembers where the next one is**.



Analogy: a train

A train is a chain of coaches. Each coach is connected to the next one. To add a coach in the middle, you **unhook** two coaches and hook the new one in between; no other coach has to move. To reach coach 7, however, you must walk through coaches 1 to 6. That is a linked list.

2.1 Linked Lists in Daily Life

Example	Why it is like a linked list
Treasure hunt / clue chain	Each clue tells you where the next clue is

Example	Why it is like a linked list
Music playlist (next song)	Each song points to the next song
Image viewer (next image)	Each image knows the next image
Round-robin turns in a Ludo game	Each player knows who plays next
A queue of people where everyone only knows who is behind them	Follow the chain from the first person



2.2 Linked Lists in Industry

- **Operating systems** keep lists of processes, free memory blocks and open files as linked lists.
- **Undo/redo, browser history, and music players** use linked lists (often doubly linked, Lab 09).
- **Hash tables** (Lab 12) use linked lists to handle collisions (separate chaining).
- **Stacks, queues and graphs** (adjacency lists) are commonly built from linked lists.
- **Blockchain** is essentially a linked list where each block stores the hash (reference) of the previous block.



2.3 Linked Lists in Research

- **Memory management and garbage collection** research (e.g., in Python and Java runtimes) is built around linked structures.
- **Lock-free linked lists** are studied for high-performance multi-core and database systems.
- **Sparse data:** polynomials and sparse matrices with mostly zeros are stored efficiently as linked lists of non-zero terms.



2.4 Why It Matters in 2026–2030



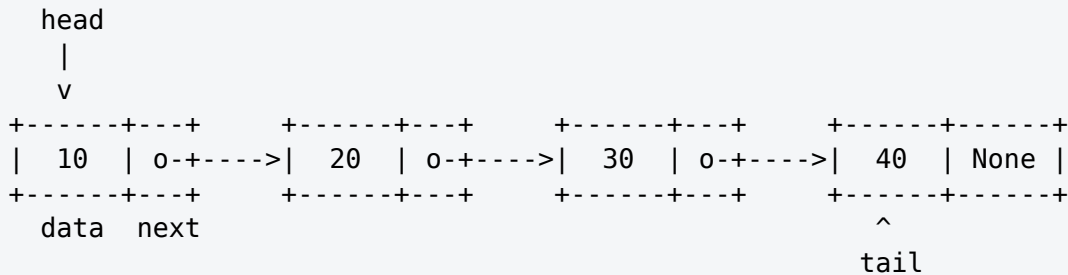
Pointers are the language of complex data structures

Trees, graphs, tries and many AI data structures (e.g., knowledge graphs, computation graphs in PyTorch) are just **nodes with references**. If you can confidently create, re-link and traverse nodes in a linked list, you have mastered the core skill for everything that comes next in this course and in advanced systems.

3. Part A: Structure of a Linked List

A **singly linked list** is a sequence of **nodes**. Each node has two parts: the **data** and a reference called **next** that points to the following node. The first node is the **head**; the last node's **next** is **None** (NULL).

A singly linked list



The nodes can be anywhere in memory, e.g. addresses 5000, 1200, 8800, 3400

	Array	Linked List
Memory	Contiguous block	Nodes scattered anywhere
Size	Fixed or resized by copying	Grows one node at a time
Access k-th element	$O(1)$	$O(n)$, must walk from head
Insert / delete at front	$O(n)$, shifting	$O(1)$
Insert / delete at end	$O(1)$ amortised	$O(1)$ with a tail pointer (insert)
Insert after a known node	$O(n)$	$O(1)$
Extra memory	None per element	One reference per node
Cache performance	Excellent	Poorer (nodes are far apart)



4. Part B: The Node and LinkedList Classes

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None          # reference to the next node

# Building a list by hand: 10 -> 20 -> 30
a = Node(10); b = Node(20); c = Node(30)
a.next = b
b.next = c
head = a

current = head                    # traversal
while current is not None:
    print(current.data, end=" -> ")
    current = current.next
print("None")
```

Now we wrap this in a **LinkedList** class that keeps track of **head**, **tail** and **size**, so users of our list never handle nodes directly.

```
class LinkedList:
    def __init__(self):
        self.head = None
        self.tail = None
        self.size = 0

    def __len__(self):
        return self.size

    def __str__(self):
        parts, cur = [], self.head
        while cur:
            parts.append(str(cur.data))
            cur = cur.next
        return " -> ".join(parts + ["None"])

    def is_empty(self):
        return self.head is None
```

+ 5. Part C: Insertion

5.1 Insert at the Head: $O(1)$

Steps (order matters!)

Before: head -> [20] -> [30] -> None

1. new = Node(10)

2. new.next = head [10] -> [20] -> [30] -> None

3. head = new head -> [10] -> [20] -> [30] -> None

5.2 Insert at the Tail: $O(1)$ with a tail pointer

Without a tail pointer we would have to walk the whole list to find the last node ($O(n)$). With **tail**, we simply do **tail.next = node** and then **tail = node**. If the list is empty, the new node is both head and tail.

5.3 Insert at a Position: $O(n)$

Walk to the node **before** the position (index - 1), then link the new node in between. Always connect the new node to the rest of the list **before** changing the previous node's **next**, or you will lose the rest of the list.

```

def push_front(self, data):
    node = Node(data)
    node.next = self.head
    self.head = node
    if self.tail is None:          # list was empty
        self.tail = node
    self.size += 1

def push_back(self, data):
    node = Node(data)
    if self.head is None:
        self.head = self.tail = node
    else:
        self.tail.next = node
        self.tail = node
    self.size += 1

def insert_at(self, index, data):
    if index < 0 or index > self.size:
        raise IndexError("index out of range")
    if index == 0:
        return self.push_front(data)
    if index == self.size:
        return self.push_back(data)
    prev = self.head
    for _ in range(index - 1):      # walk to the node before index
        prev = prev.next
    node = Node(data)
    node.next = prev.next          # 1. connect new node to the rest
    prev.next = node              # 2. connect previous node to new node
    self.size += 1

LinkedList.push_front = push_front
LinkedList.push_back = push_back
LinkedList.insert_at = insert_at

ll = LinkedList()
for x in [20, 30, 50]:
    ll.push_back(x)
ll.push_front(10)
ll.insert_at(3, 40)
print(ll, "| size =", len(ll))

```

We attach methods to the class one part at a time so each idea stays short. In your own code, write all methods inside the class.

6. Part D: Search and Access

```
def find(self, target):
    cur, index = self.head, 0
    while cur:
        if cur.data == target:
            return index
        cur = cur.next
        index += 1
    return -1

def get(self, index):
    if not 0 <= index < self.size:
        raise IndexError("index out of range")
    cur = self.head
    for _ in range(index):
        cur = cur.next
    return cur.data

LinkedList.find = find
LinkedList.get = get
print(ll.find(40), ll.find(99), ll.get(2))
```

— 7. Part E: Deletion

To delete a node, make the **previous node skip over it**: `prev.next = prev.next.next`. Python's garbage collector frees the removed node automatically (in C++ you must **delete** it yourself). Special cases: deleting the head, deleting the tail (update **tail**), and an empty list.

Skipping a node

```
Delete 30:   head -> [10] -> [20] -> [30] -> [40] -> None
              prev   target
prev.next = target.next
              head -> [10] -> [20] -----> [40] -> None
```

```
def pop_front(self):
    if self.head is None:
        raise IndexError("pop from empty list")
    data = self.head.data
    self.head = self.head.next
    if self.head is None:
        self.tail = None
    self.size -= 1
    return data

def remove(self, target):
    """Remove the first node containing target. Return True if removed."""
    if self.head is None:
        return False
    if self.head.data == target:
        self.pop_front()
        return True
    prev = self.head
    while prev.next and prev.next.data != target:
        prev = prev.next
    if prev.next is None:
        return False # not found
    if prev.next is self.tail:
        self.tail = prev # deleting the last node
    prev.next = prev.next.next
    self.size -= 1
    return True

LinkedList.pop_front = pop_front
LinkedList.remove = remove
print(ll)
ll.remove(30); ll.remove(50); print(ll, "| tail =", ll.tail.data)
print("popped", ll.pop_front(), "->", ll)
```

8. Part F: Reversing a Linked List

Reversing is the most famous linked-list interview question. We walk through the list once and **flip each next pointer** to point backwards. We need three references: **prev**, **cur** and **nxt**.

Dry run

```
Start:   None   [10] -> [20] -> [30] -> None
         prev   cur
Step 1:  None <- [10]   [20] -> [30] -> None
         prev   cur
Step 2:  None <- [10] <- [20]   [30] -> None
         prev   cur
Step 3:  None <- [10] <- [20] <- [30]         head = prev
```

```
def reverse(self):
    prev, cur = None, self.head
    self.tail = self.head
    while cur:
        nxt = cur.next           # 1. save the rest of the list
        cur.next = prev         # 2. flip the pointer
        prev = cur              # 3. move prev forward
        cur = nxt               # 4. move cur forward
    self.head = prev

LinkedList.reverse = reverse
demo = LinkedList()
for x in [10, 20, 30, 40]:
    demo.push_back(x)
demo.reverse()
print(demo)
```

Time $O(n)$, extra space $O(1)$. Notice how step 1 is essential: if you flip the pointer first, you lose the rest of the list forever.

9. Part G: Slow and Fast Pointers

9.1 Find the Middle in One Pass

Move **slow** one step and **fast** two steps at a time. When **fast** reaches the end, **slow** is at the middle.

```
def middle(self):
    slow = fast = self.head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow.data if slow else None

LinkedList.middle = middle
m = LinkedList()
for x in [1, 2, 3, 4, 5]:
    m.push_back(x)
print("middle of", m, "is", m.middle())
```

9.2 Detect a Cycle (Floyd's Tortoise and Hare)

If some node's **next** points back to an earlier node, traversal would loop forever. With slow and fast pointers, if there is a cycle the fast pointer will eventually **meet** the slow one, like a faster runner lapping a slower runner on a circular track.

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:
            return True
    return False

nodes = [Node(i) for i in range(5)]
for i in range(4):
    nodes[i].next = nodes[i + 1]
print("cycle?", has_cycle(nodes[0]))
nodes[4].next = nodes[2]          # create a cycle 4 -> 2
print("cycle?", has_cycle(nodes[0]))
```

10. C++ Corner

In C++, nodes are created with **new** and must be freed with **delete**. Forgetting to free them causes **memory leaks**.

C++

```

#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int d) : data(d), next(nullptr) {}
};

class LinkedList {
    Node* head = nullptr;
public:
    void pushFront(int d) { Node* n = new Node(d); n->next = head; head = n; }
    bool remove(int target) {
        Node** cur = &head;           // pointer to the link we may
change
        while (*cur && (*cur)->data != target) cur = &((*cur)->next);
        if (!*cur) return false;
        Node* dead = *cur;
        *cur = dead->next;
        delete dead;                   // free memory
        return true;
    }
    void reverse() {
        Node *prev = nullptr, *cur = head;
        while (cur) { Node* nxt = cur->next; cur->next = prev; prev = cur; cur
= nxt; }
        head = prev;
    }
    void print() {
        for (Node* c = head; c; c = c->next) cout << c->data << " -> ";
        cout << "NULL" << endl;
    }
    ~LinkedList() { while (head) { Node* t = head; head = head->next; delete
t; } }
};

int main() {
    LinkedList ll;
    for (int x : {40, 30, 20, 10}) ll.pushFront(x);
    ll.print(); ll.remove(30); ll.reverse(); ll.print();
}

```

11. Complexity Summary

Operation	Singly linked list (head + tail)
push_front / pop_front	O(1)
push_back	O(1)
pop_back (remove last)	O(n), need the node before tail

Operation	Singly linked list (head + tail)
get(i) / find(x)	O(n)
insert_at / remove(x)	O(n) to find the spot, O(1) to re-link
reverse / middle / has_cycle	O(n) time, O(1) space

🚫 12. Common Mistakes

Mistake	Effect	Fix
Moving head while traversing	Lose the list	Use a separate cur variable
Changing prev.next before linking the new node	Rest of the list is lost	Link node.next first
Not handling an empty list	AttributeError: 'NoneType'...	Check if self.head is None
Forgetting to update tail	push_back attaches to a deleted node	Update tail when deleting the last node
while cur.next: when cur may be None	Crash on empty list	Check cur first or use while cur:
Forgetting size updates	Wrong length	Change size in every insert/delete



13. Lab Tasks

Solve all tasks in `Lab08_Student.ipynb`. Each task has a **test cell**: when it prints  your solution is correct. For each task, **write pseudocode first**, then code, then test with your own inputs including edge cases. Difficulty:  Easy  Medium  Hard.

Task 1 Build and Print

The notebook gives helper functions `from_list(values)` (builds a list and returns its head) and `to_list(head)`.

Write `length(head)` and `sum_values(head)` iteratively, and `length_rec(head)` recursively.

```
head = from_list([4, 8, 15]) → length 3, sum 27
```

Task 2 Remove All Occurrences

Write `remove_all(head, value)` that deletes **every** node containing `value` and returns the (possibly new) head. Hint: a **dummy node** placed before the head removes the special case of deleting the head.

```
[7, 1, 7, 2, 7] remove 7 → [1, 2]
```

Task 3 Reverse (Iterative and Recursive)

Write `reverse_iter(head)` and `reverse_rec(head)`, each returning the new head. For the recursive version: reverse the rest, then make the second node point back to the first.

```
[1, 2, 3, 4] → [4, 3, 2, 1]
```

Task 4 Merge Two Sorted Lists

Two sorted linked lists of roll numbers must be merged into one sorted list by **re-linking the existing nodes** (do not create new nodes except one dummy). Return the head.

```
[1, 4, 9] + [2, 3, 10] → [1, 2, 3, 4, 9, 10]
```

Task 5 **Remove the N-th Node from the End**

In **one pass**, remove the n-th node from the end and return the head. Hint: move a **fast** pointer n steps ahead, then move **fast** and **slow** together; use a dummy node.

```
[1, 2, 3, 4, 5], n = 2 → [1, 2, 3, 5]
```

Task 6 **Palindrome Linked List**

Check whether a linked list reads the same forwards and backwards in **O(n) time and O(1) extra space**: find the middle (slow/fast), reverse the second half, compare the two halves, and (good practice) restore the list.

```
[1, 2, 2, 1] → True, [1, 2, 3] → False
```

Task 7 **Add Two Big Numbers**

Very large numbers (bigger than any built-in integer type in C++) can be stored as linked lists of digits in **reverse order**. Add two such numbers and return the result as a linked list.

This is how arbitrary-precision arithmetic (used in cryptography and Python's own big integers) works.

```
342 + 465: [2, 4, 3] + [5, 6, 4] → [7, 0, 8] (= 807)
```

**14. LeetCode Practice**

Solve these on **leetcode.com** and upload screenshots of the “Accepted” result to your GitHub **DSA-Labs/Lab08** folder.

#	Problem	Level	Concept practised
206	Reverse Linked List	Easy	Pointer re-linking (Task 3)
21	Merge Two Sorted Lists	Easy	Dummy node (Task 4)
876	Middle of the Linked List	Easy	Slow / fast pointers
203	Remove Linked List Elements	Easy	Dummy node (Task 2)
141	Linked List Cycle	Easy	Floyd's algorithm
234	Palindrome Linked List	Easy	Middle + reverse (Task 6)
19	Remove Nth Node From End of List	Medium	Two pointers (Task 5)

#	Problem	Level	Concept practised
2	Add Two Numbers	Medium	Digit lists (Task 7)



15. Home Task

1. Finish all tasks and push `Lab08_Student.ipynb` to GitHub.
2. Add `pop_back()`, `insert_sorted(x)` and `__iter__` (so `for x in ll:` works) to your `LinkedList` class.
3. Find the **start node** of a cycle using Floyd's algorithm (after the meeting point, move one pointer to head and advance both by one).
4. **Mini-project: Polynomial Calculator.** Store polynomials such as $5x^3 + 4x^2 + 2$ as linked lists of (coefficient, power) nodes sorted by power. Implement addition, multiplication and evaluation at a given x .
5. Solve at least **3** LeetCode problems from the list.



16. Viva Questions

1. What is a node? What are its two parts?
2. What does the head point to? What does the last node's next contain?
3. Why is accessing the k -th element $O(n)$ in a linked list?
4. Why is inserting at the front $O(1)$ in a linked list but $O(n)$ in an array?
5. Why do we keep a tail pointer?
6. Explain the order of steps when inserting a node in the middle.
7. How do you delete a node? What special cases must you handle?
8. Explain how to reverse a linked list in place.
9. How does the slow/fast pointer technique find the middle?
10. How does Floyd's algorithm detect a cycle?



17. Assessment Rubric

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Correctness	All tasks pass all tests	Most tasks correct	Some tasks correct	Mostly incorrect

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Logic & pseudocode	Clear pseudocode and dry run for every task	Pseudocode for most tasks	Partial pseudocode	None
Code quality	Meaningful names, functions, comments	Mostly clean	Hard to read	Copied or unreadable
Complexity & edge cases	States Big-O and handles all edge cases	Some edge cases / Big-O	Only basic cases	Not considered
Viva	Explains every line confidently	Explains most parts	Basic understanding	Cannot explain

Lab Rules

- Bring your own laptop with Colab or Jupyter working.
- Arrive on time; late arrival may result in no attendance for the lab.
- Copying another student's code or pasting AI output without understanding it will be awarded zero. You must be able to explain every line in the viva.

18. References and Further Learning

- Linked list animations: **visualgo.net/en/list**
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, **Introduction to Algorithms** (CLRS), 4th ed., MIT Press, 2022.
- M. T. Goodrich, R. Tamassia, M. H. Goldwasser, **Data Structures and Algorithms in Python**, Wiley.
- MIT OpenCourseWare 6.006 Introduction to Algorithms; Harvard CS50x (free online courses).
- Visualise code step by step: **pythontutor.com**; animated data structures: **visualgo.net**.



About the Author

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

GitHub: github.com/RamzanPUCIT

LinkedIn: linkedin.com/in/ramzan-ai

ResearchGate: researchgate.net/profile/Muhammad-Ramzan-105

© 2026 Muhammad Ramzan. Prepared for the Department of Computer Science, University of Okara.

UNIVERSITY OF OKARA

Department of Computer Science

DATA STRUCTURES & ALGORITHMS

Lab Manual

LAB 09 | Doubly Linked Lists, Circular Linked Lists and Their Applications

Week 9 of 16 • Duration: 3 hours (in-lab) + home practice

Prepared by

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

Applicable to: BSCS • BSIT • BSSE • BSAI • BSDS • BS Cyber Security and all programs offering DSA

Fall 2026 Edition

About This Lab Manual

This manual is part of a **16-week Data Structures & Algorithms (DSA) lab series**. It is designed so that a student starting with **zero programming knowledge** in Week 1 can confidently design, implement and analyse data structures by Week 16. Every lab builds on the previous one, so please do not skip labs.

Language policy: concepts are explained in a language-independent way (diagrams, pseudocode, dry runs). Full implementations are in **Python**, and every lab includes a **C++ Corner** so instructors teaching in C++ can use the same manual.

Every lab follows the same structure

Section	What you get
 Objectives & CLOs	What you will be able to do after this lab
 Motivation	Where this topic is used in real life, industry, research and the future (2026–2030)
 Concepts	Simple explanations with real examples, pseudocode and dry runs
 Code	Python implementation + C++ Corner
 Lab Tasks	Graded tasks:  Easy,  Medium,  Hard
 LeetCode	Real interview problems to practise
 Home Task &  Viva	Practice at home and prepare for oral questions
 Notebooks	Instructor notebook (complete code) and Student notebook (with TODO blanks and auto-check tests)

Lab 09 at a glance

Item	Details
Lab title	Doubly Linked Lists, Circular Linked Lists and Their Applications
Week	9 of 16
Prerequisites	Lab 08 (Singly Linked List)
Tools	Google Colab / Jupyter / VS Code (Python 3); any C++ compiler for the C++ Corner
Files	Lab09_Instructor.ipynb , Lab09_Student.ipynb
Next lab	Lab 10: Stacks

1. Learning Objectives

By the end of this lab, you will be able to:

1. Explain the structure of a **doubly linked list** (prev and next references) and its advantages.
2. Implement a doubly linked list with **sentinel (dummy) head and tail** nodes.
3. Insert and delete at both ends and **delete a node in $O(1)$** when you have a reference to it.
4. Traverse a doubly linked list forwards and backwards.
5. Implement a **circular linked list** and traverse it safely.
6. Solve the **Josephus problem** and simulate **round-robin scheduling** with a circular list.
7. Build real applications: **browser history**, **music playlist** and an **LRU cache**.

Course Learning Outcomes (CLO) mapping

CLO	Description	Taxonomy level
CLO-2	Implement doubly and circular linked lists	Apply (C3)
CLO-3	Select an appropriate list variant for an application	Analyse (C4)

2. Motivation: Going Back and Going Round

A singly linked list can only move **forward**. You cannot go back to the previous node, and deleting the last node takes $O(n)$. Many real systems need to move **both ways** (back and forward in a browser) or **go round in circles** (players taking turns). Doubly and circular linked lists solve exactly these problems.

2.1 In Daily Life

Application	Structure	Why
Browser Back / Forward buttons	Doubly linked list	Move to previous and next page
Music player: previous / next song, repeat all	Circular doubly linked list	Both directions and wrap around
Image gallery on your phone	Doubly linked list	Swipe left and right
Ludo / card game turns	Circular linked list	After the last player, the first plays again

Application	Structure	Why
Ctrl+Z / Ctrl+Y (undo / redo)	Doubly linked list (or two stacks)	Step back and forward through edits
Traffic signal cycle (red → green → yellow → red)	Circular list	Repeats forever

2.2 In Industry

- **LRU (Least Recently Used) caches** in web browsers, databases, Redis and operating systems use a **hash map + doubly linked list** to evict old data in $O(1)$.
- **Operating systems** use circular queues of processes for **round-robin CPU scheduling**.
- **Text editors** store lines in doubly linked lists so the cursor can move up and down efficiently.
- Java's `LinkedList` and Python's `collections.deque` are implemented with doubly linked structures.

2.3 In Research

- **Cache replacement policies** (LRU, LFU, ARC) are actively researched for CDNs, databases and AI model serving (e.g., caching attention keys/values in large language models).
- **Real-time systems and networking** use circular buffers to process continuous sensor and network streams.

2.4 Why It Matters in 2026–2030

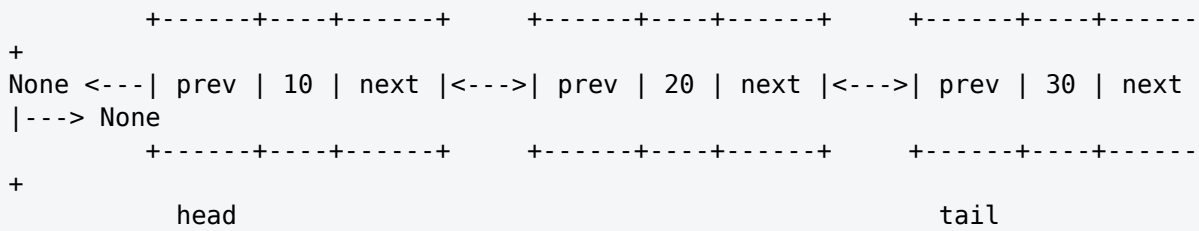
Caching is everywhere

Every fast app you use, from YouTube to ChatGPT, relies on **caching**: keeping recently used data close so it can be served instantly. The LRU cache you build today (Task 7) is the exact design asked in interviews at top companies and used in production systems.

3. Part A: Doubly Linked List

In a **doubly linked list (DLL)**, each node has **three parts**: **prev** (reference to the previous node), **data**, and **next**.

A doubly linked list



	Singly linked list	Doubly linked list
References per node	1 (next)	2 (prev, next)
Traverse backwards	Not possible	Yes
Delete last node	$O(n)$	$O(1)$
Delete a given node (with reference)	$O(n)$, need previous node	$O(1)$
Memory	Less	More (one extra reference per node)
Code complexity	Simpler	More pointers to update

3.1 Sentinel Nodes: A Professional Trick

Handling empty lists and the head/tail special cases makes linked-list code messy. Professionals add two **dummy (sentinel) nodes** that never hold real data: **head** sentinel at the start and **tail** sentinel at the end. Every real node is always **between two nodes**, so insertion and deletion have **no special cases**.

DLL with sentinels

```

[HEAD] <-> [10] <-> [20] <-> [30] <-> [TAIL]      (HEAD and TAIL hold no data)
Empty list:  [HEAD] <-> [TAIL]
    
```

```

class DNode:
    def __init__(self, data=None):
        self.data = data
        self.prev = None
        self.next = None

class DoublyLinkedList:
    def __init__(self):
        self.head = DNode()      # sentinel
        self.tail = DNode()      # sentinel
        self.head.next = self.tail
        self.tail.prev = self.head
    
```

```

        self.size = 0

    def _insert_between(self, data, before, after):
        node = DNode(data)
        node.prev, node.next = before, after
        before.next = node
        after.prev = node
        self.size += 1
        return node

    def _unlink(self, node):          # 0(1) delete of a known node
        node.prev.next = node.next
        node.next.prev = node.prev
        self.size -= 1
        return node.data

    def push_front(self, data):
        return self._insert_between(data, self.head, self.head.next)

    def push_back(self, data):
        return self._insert_between(data, self.tail.prev, self.tail)

    def pop_front(self):
        if self.size == 0:
            raise IndexError("empty list")
        return self._unlink(self.head.next)

    def pop_back(self):
        if self.size == 0:
            raise IndexError("empty list")
        return self._unlink(self.tail.prev)

    def forward(self):
        out, cur = [], self.head.next
        while cur is not self.tail:
            out.append(cur.data); cur = cur.next
        return out

    def backward(self):
        out, cur = [], self.tail.prev
        while cur is not self.head:
            out.append(cur.data); cur = cur.prev
        return out

d = DoublyLinkedList()
for x in [20, 30, 40]:
    d.push_back(x)
d.push_front(10)
print("forward :", d.forward())
print("backward:", d.backward())
print("pop_back ->", d.pop_back(), "| pop_front ->", d.pop_front(), "|",
      d.forward())

```

3.2 Inserting in the Middle: Four Links

To insert node N between nodes A and B, **four** references change: **N.prev = A**, **N.next = B**, **A.next = N**, **B.prev = N**. Set the new node's links first, then update the neighbours.

```
Before:  [A] <-> [B]
1. N.prev = A      2. N.next = B
3. A.next = N      4. B.prev = N
After:   [A] <-> [N] <-> [B]
```

3.3 Deleting a Node in O(1)

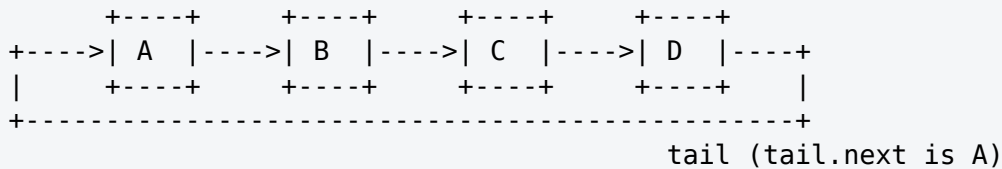
If we hold a reference to a node (for example from a dictionary), we can remove it instantly: its previous node skips forward over it and its next node skips backward over it. This is impossible in O(1) with a singly linked list, and it is the key to the LRU cache.

```
d2 = DoublyLinkedList()
handles = {name: d2.push_back(name) for name in ["Ali", "Sara", "Hamza",
"Zoya"]}
d2._unlink(handles["Hamza"])          # O(1): no searching
print(d2.forward())
```

4. Part B: Circular Linked List

In a **circular linked list**, the last node points back to the **first** node instead of None. There is no end, so you can keep going round. It is convenient to keep a reference to the **tail**: then **tail.next** is the head, giving $O(1)$ access to both ends.

Circular singly linked list



! Careful with traversal

`while cur is not None` loops **forever** on a circular list. Instead, stop when you come back to the starting node.

```

class CNode:
    def __init__(self, data):
        self.data = data
        self.next = None

class CircularList:
    def __init__(self):
        self.tail = None
        self.size = 0

    def append(self, data):
        node = CNode(data)
        if self.tail is None:
            node.next = node           # single node points to itself
        else:
            node.next = self.tail.next # new node -> head
            self.tail.next = node      # old tail -> new node
        self.tail = node
        self.size += 1

    def to_list(self):
        if self.tail is None:
            return []
        out, cur = [], self.tail.next # start at head
        while True:
            out.append(cur.data)
            cur = cur.next
            if cur is self.tail.next: # back at the start
                break
        return out

    def rotate(self):                  # move head to the end in O(1)
        if self.tail:
            self.tail = self.tail.next

players = CircularList()
for p in ["Red", "Green", "Yellow", "Blue"]:
    players.append(p)
turn = players.tail.next
for _ in range(6):                   # six turns of Ludo
    print("Turn:", turn.data)
    turn = turn.next

```

4.1 Round-Robin Scheduling

An operating system gives each process a fixed **time slice** (quantum). If a process is not finished, it goes to the back of the circle. This is fair: no process waits forever.

```
def round_robin(jobs, quantum):
    """jobs: list of (name, burst_time). Returns finishing order and
    times."""
    ring = CircularList()
    for name, burst in jobs:
        ring.append([name, burst])
    time, finished = 0, []
    prev, cur = ring.tail, ring.tail.next
    while ring.size:
        run = min(quantum, cur.data[1])
        time += run
        cur.data[1] -= run
        if cur.data[1] == 0:                # job done: remove it from
the ring
            finished.append((cur.data[0], time))
            ring.size -= 1
            if ring.size == 0:
                break
            prev.next = cur.next
            if cur is ring.tail:
                ring.tail = prev
            cur = cur.next
        else:
            prev, cur = cur, cur.next
    return finished

print(round_robin([("P1", 5), ("P2", 3), ("P3", 8)], quantum=2))
```

5. Part C: Choosing the Right List

Need	Best choice
Simple stack or forward-only sequence	Singly linked list
Move both ways, delete known nodes in $O(1)$	Doubly linked list
Repeat in a cycle (turns, playlists, buffers)	Circular linked list
Both directions + cycle (repeat-all playlist)	Circular doubly linked list
Fast access by index	Array / Python list (not a linked list!)
Queue with fast operations at both ends	<code>collections.deque</code> (built on a doubly linked structure)

Python's built-in deque

```

from collections import deque
history = deque(maxlen=3) # keeps only the last 3 items
for page in ["google.com", "uo.edu.pk", "github.com", "leetcode.com"]:
    history.append(page)
print(history)
history.appendleft("start"); print(history.pop(), history)

```

6. C++ Corner

C++

```

#include <iostream>
#include <list>
using namespace std;

struct DNode {
    int data; DNode* prev; DNode* next;
    DNode(int d) : data(d), prev(nullptr), next(nullptr) {}
};

int main() {
    // Manual doubly linked list: 10 <-> 20 <-> 30
    DNode* a = new DNode(10); DNode* b = new DNode(20); DNode* c = new
DNode(30);
    a->next = b; b->prev = a; b->next = c; c->prev = b;
    for (DNode* p = c; p; p = p->prev) cout << p->data << " "; // backwards
    cout << endl;
    // delete b in O(1)
    b->prev->next = b->next; b->next->prev = b->prev; delete b;
    delete a; delete c;

    // The standard library doubly linked list
    list<string> playlist = {"Song A", "Song B", "Song C"};
    playlist.push_front("Intro");
    playlist.pop_back();
    for (auto it = playlist.rbegin(); it != playlist.rend(); ++it) cout << *it
<< " | ";
}

```

7. Complexity Summary

Operation	Singly (head+tail)	Doubly (sentinels)	Circular (tail ref)
Insert front / back	O(1) / O(1)	O(1) / O(1)	O(1) / O(1)
Delete front	O(1)	O(1)	O(1)
Delete back	O(n)	O(1)	O(n) (singly circular)

Operation	Singly (head+tail)	Doubly (sentinels)	Circular (tail ref)
Delete a known node	O(n)	O(1)	O(n)
Search / access by index	O(n)	O(n)	O(n)
Reverse traversal	-	O(n)	-

8. Common Mistakes

Mistake	Effect	Fix
Updating only next , forgetting prev	Backward traversal breaks	Always update both directions
Changing neighbours before setting the new node's links	Lost references	Set node.prev/next first
while cur: on a circular list	Infinite loop	Stop when back at the start node
Removing the tail of a circular list without updating tail	Ring points to a removed node	Update tail to the previous node
Printing sentinel nodes as data	None values in output	Start at head.next , stop at tail

9. Lab Tasks

Solve all tasks in `Lab09_Student.ipynb`. Each task has a **test cell**: when it prints  your solution is correct. For each task, **write pseudocode first**, then code, then test with your own inputs including edge cases. Difficulty:  Easy  Medium  Hard.

Task 1 Insert After a Value

Add a method `insert_after(target, data)` to `DoublyLinkedList` (the notebook re-defines the class) that inserts `data` right after the first node containing `target`. Return `True` if inserted, `False` if `target` is not found.

```
[10, 20, 30].insert_after(20, 25) → [10, 20, 25, 30]
```

Task 2 Reverse a Doubly Linked List

Add `reverse()` to `DoublyLinkedList`: swap `prev` and `next` in **every** node (including the sentinels) and then swap the head and tail sentinels. Both `forward()` and `backward()` must still work.

```
[1, 2, 3] → [3, 2, 1]
```

Task 3 Browser History

Implement `BrowserHistory(homepage)` with methods `visit(url)`, `back(steps)` and `forward(steps)` using a doubly linked list of pages and a `current` reference.

`visit` clears all forward history. `back/forward` move at most as far as possible and return the current URL.

```
b = BrowserHistory("uo.edu.pk"); visit("lms"); visit("results")
back(1) → "lms"; back(5) → "uo.edu.pk"; forward(1) → "lms"
visit("timetable"); forward(2) → "timetable"
```

Task 4 Josephus Problem

n people stand in a circle numbered 1 to n . Starting from person 1, every k -th person is eliminated, and counting continues from the next person. Return the **order of elimination** and the **survivor**.

Simulate it with a circular linked list ($O(n \cdot k)$). The story: the historian Josephus used this to find the safe position.

```
josephus(7, 3) → ([3, 6, 2, 7, 5, 1], 4)
```

Task 5 🟡 Playlist with Repeat-All

Build a **Playlist** class on a **circular doubly linked list** with: `add(song)`, `next()`, `prev()` (both wrap around), `current()` and `remove_current()` (removes the playing song and moves to the next one).

Return **None** from `current()` when the playlist is empty.

```
add A, B, C → current A; prev() → C; next() → A; next() → B; remove_current()
→ now C
```

Task 6 🔴 Delete Duplicates from a Sorted DLL

Write `dedupe(dll)` that removes duplicate values from a **sorted DoublyLinkedList** in a single pass using `_unlink`, keeping the first occurrence. Make sure `backward()` still gives the correct result.

```
[1, 1, 2, 3, 3, 3, 4] → [1, 2, 3, 4]
```

Task 7 🔴 LRU Cache (Interview Classic)

Design an `LRUCache(capacity)` with `get(key)` and `put(key, value)`, **both in $O(1)$** . When the cache is full, `put` evicts the **least recently used** key. Any `get` or `put` makes a key the most recently used.

Design: a dictionary maps `key` → DLL node; the DLL keeps keys in order of use (most recent at the front). Use `DNode` with `data = (key, value)`.

```
c = LRUCache(2); put(1,1); put(2,2); get(1) → 1; put(3,3) # evicts 2
get(2) → -1; put(4,4) # evicts 1
get(1) → -1; get(3) → 3; get(4) → 4
```



10. LeetCode Practice

Solve these on **leetcode.com** and upload screenshots of the “Accepted” result to your GitHub **DSA-Labs/Lab09** folder.

#	Problem	Level	Concept practised
707	Design Linked List	Medium	Build a full linked list
1472	Design Browser History	Medium	Doubly linked list (Task 3)
146	LRU Cache	Medium	Hash map + DLL (Task 7)

#	Problem	Level	Concept practised
641	Design Circular Deque	Medium	Circular structure
430	Flatten a Multilevel Doubly Linked List	Medium	DLL pointer manipulation
1823	Find the Winner of the Circular Game	Medium	Josephus (Task 4)



11. Home Task

1. Finish all tasks and push `Lab09_Student.ipynb` to GitHub.
2. Add `__iter__`, `__reversed__` and `__contains__` to `DoublyLinkedList` so it works with `for`, `reversed()` and `in`.
3. Implement a **circular buffer** of fixed size for sensor readings: new readings overwrite the oldest when full. Compute the moving average of the last 10 readings.
4. **Mini-project: Text Editor Line Buffer.** Store lines of text in a doubly linked list with a cursor. Support: move up/down, insert a line, delete the current line, undo the last delete, and print the document.
5. Solve at least 3 LeetCode problems from the list.

? 12. Viva Questions

1. What are the three parts of a doubly linked list node?
2. Give two advantages and one disadvantage of a DLL compared to a singly linked list.
3. Why can a DLL delete a known node in $O(1)$?
4. What are sentinel nodes and why are they useful?
5. How many references change when inserting a node in the middle of a DLL?
6. What is a circular linked list? How do you traverse it without an infinite loop?
7. Why keep a reference to the tail instead of the head in a circular list?
8. Explain round-robin scheduling.
9. Explain the Josephus problem.
10. How does an LRU cache achieve $O(1)$ get and put?



13. Assessment Rubric

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Correctness	All tasks pass all tests	Most tasks correct	Some tasks correct	Mostly incorrect
Logic & pseudocode	Clear pseudocode and dry run for every task	Pseudocode for most tasks	Partial pseudocode	None
Code quality	Meaningful names, functions, comments	Mostly clean	Hard to read	Copied or unreadable
Complexity & edge cases	States Big-O and handles all edge cases	Some edge cases / Big-O	Only basic cases	Not considered
Viva	Explains every line confidently	Explains most parts	Basic understanding	Cannot explain



Lab Rules

- Bring your own laptop with Colab or Jupyter working.
- Arrive on time; late arrival may result in no attendance for the lab.
- Copying another student's code or pasting AI output without understanding it will be awarded zero. You must be able to explain every line in the viva.



14. References and Further Learning

- Python `collections.deque`: docs.python.org/3/library/collections.html#collections.deque
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, **Introduction to Algorithms** (CLRS), 4th ed., MIT Press, 2022.
- M. T. Goodrich, R. Tamassia, M. H. Goldwasser, **Data Structures and Algorithms in Python**, Wiley.
- MIT OpenCourseWare 6.006 Introduction to Algorithms; Harvard CS50x (free online courses).
- Visualise code step by step: pythontutor.com; animated data structures: visualgo.net.



About the Author

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

GitHub: github.com/RamzanPUCIT

LinkedIn: linkedin.com/in/ramzan-ai

ResearchGate: researchgate.net/profile/Muhammad-Ramzan-105

© 2026 Muhammad Ramzan. Prepared for the Department of Computer Science, University of Okara.

UNIVERSITY OF OKARA

Department of Computer Science

DATA STRUCTURES & ALGORITHMS

Lab Manual

LAB 10 | Stacks: LIFO, Implementations and Applications

Week 10 of 16 • Duration: 3 hours (in-lab) + home practice

Prepared by

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

Applicable to: BSCS • BSIT • BSSE • BSAI • BSDS • BS Cyber Security and all programs offering DSA

Fall 2026 Edition

About This Lab Manual

This manual is part of a **16-week Data Structures & Algorithms (DSA) lab series**. It is designed so that a student starting with **zero programming knowledge** in Week 1 can confidently design, implement and analyse data structures by Week 16. Every lab builds on the previous one, so please do not skip labs.

Language policy: concepts are explained in a language-independent way (diagrams, pseudocode, dry runs). Full implementations are in **Python**, and every lab includes a **C++ Corner** so instructors teaching in C++ can use the same manual.

Every lab follows the same structure

Section	What you get
 Objectives & CLOs	What you will be able to do after this lab
 Motivation	Where this topic is used in real life, industry, research and the future (2026–2030)
 Concepts	Simple explanations with real examples, pseudocode and dry runs
 Code	Python implementation + C++ Corner
 Lab Tasks	Graded tasks:  Easy,  Medium,  Hard
 LeetCode	Real interview problems to practise
 Home Task &  Viva	Practice at home and prepare for oral questions
 Notebooks	Instructor notebook (complete code) and Student notebook (with TODO blanks and auto-check tests)

Lab 10 at a glance

Item	Details
Lab title	Stacks: LIFO, Implementations and Applications
Week	10 of 16
Prerequisites	Lab 03 (Arrays), Lab 08 (Linked Lists)
Tools	Google Colab / Jupyter / VS Code (Python 3); any C++ compiler for the C++ Corner
Files	Lab10_Instructor.ipynb , Lab10_Student.ipynb
Next lab	Lab 11: Queues, Circular Queues and Deques

1. Learning Objectives

By the end of this lab, you will be able to:

1. Explain the **stack ADT** and the **LIFO** principle with real examples.
2. Implement the stack operations **push**, **pop**, **peek/top**, **size**, **is_empty** in **O(1)**.
3. Implement a stack using an **array (Python list)**, a **fixed-size array** (with overflow/underflow) and a **linked list**.
4. Check **balanced brackets** using a stack.
5. Convert **infix to postfix** and **evaluate postfix** expressions.
6. Use a **monotonic stack** to solve “next greater element” problems in **O(n)**.
7. Explain how the **function call stack** and **undo/redo** work.

Course Learning Outcomes (CLO) mapping

CLO	Description	Taxonomy level
CLO-1	Explain stack as an abstract data type	Understand (C2)
CLO-2	Implement stacks and stack-based algorithms	Apply (C3)
CLO-3	Apply stacks to solve problems	Apply / Analyse (C3–C4)

2. Motivation: The Most Recent Thing First

Many problems share one pattern: **the last thing that came in is the first thing that must go out**. When you press Ctrl+Z, the **last** change is undone first. When a function finishes, control returns to the function that called it **most recently**. A **stack** is the data structure for this pattern.

Analogy: a stack of plates

At a wedding buffet, clean plates are stacked. The waiter puts a new plate **on top**, and guests take a plate **from the top**. Nobody pulls a plate from the middle. Last In, First Out: **LIFO**.

2.1 Stacks in Daily Life

Example	Push	Pop
Undo in MS Word / Photoshop	Every edit you make	Ctrl+Z removes the latest edit
Browser Back button	Every page you visit	Back goes to the most recent page
Pile of books / exam copies on a table	Put one on top	Take from the top
Bangles on a wrist	Last bangle put on	Is the first to come off
Chips in a Pringles can	Filled from the top	Eaten from the top

2.2 Stacks in Industry

- **Every running program** uses a call stack to manage function calls and local variables. A “stack overflow” error (and the famous website) is named after it.
- **Compilers and interpreters** use stacks to check syntax (matching brackets, HTML/XML tags) and to evaluate expressions.
- **Calculators** convert your expression to postfix and evaluate it with a stack.
- **Code editors (VS Code)** highlight matching brackets and detect missing ones using a stack.
- **Depth-First Search** in graphs (Lab 16) and backtracking (maze solving) use stacks.

2.3 Stacks in Research

- **Programming languages and compilers** research: parsing (LL/LR parsers), virtual machines (the Python and Java VMs are stack machines).
- **NLP**: classic dependency parsers (shift-reduce parsers) use a stack and a buffer to build sentence trees.
- **Computational geometry**: the Graham scan algorithm finds the convex hull of points with a stack.

2.4 Why It Matters in 2026–2030



A small structure with a big footprint

Stacks appear in every layer of computing: hardware, operating systems, compilers, browsers, AI agents that plan and backtrack. Monotonic stacks are now a standard interview topic. Mastering the stack also prepares you to understand recursion deeply, because **every recursive algorithm can be rewritten with an explicit stack**.

3. Part A: The Stack ADT

An **Abstract Data Type (ADT)** describes **what operations** a structure supports, not **how** it is implemented. The stack ADT has these operations:

Operation	Meaning	Time
<code>push(x)</code>	Add x on the top	O(1)
<code>pop()</code>	Remove and return the top element	O(1)
<code>peek() / top()</code>	Return the top element without removing it	O(1)
<code>is_empty()</code>	Is the stack empty?	O(1)
<code>size()</code>	Number of elements	O(1)

Error	When
Stack underflow	<code>pop()</code> or <code>peek()</code> on an empty stack
Stack overflow	<code>push()</code> on a full fixed-size stack

Stack operations step by step

push(10)	push(20)	push(30)	pop() -> 30	peek() -> 20
		30 <-top		
	20 <-top	20	20 <-top	20 <-top
10 <-top	10	10	10	10
+-----+	+-----+	+-----+	+-----+	+-----+



4. Part B: Implementations

4.1 1. Using a Python List (Dynamic Array)

The **end** of a Python list is the top of the stack: **append** and **pop()** at the end are $O(1)$ amortised. Never use the front (**insert(0, x)**, **pop(0)**), which is $O(n)$.

```
class Stack:
    def __init__(self):
        self._items = []

    def push(self, x):
        self._items.append(x)

    def pop(self):
        if self.is_empty():
            raise IndexError("stack underflow")
        return self._items.pop()

    def peek(self):
        if self.is_empty():
            raise IndexError("stack is empty")
        return self._items[-1]

    def is_empty(self):
        return len(self._items) == 0

    def size(self):
        return len(self._items)

    def __str__(self):
        return "bottom " + str(self._items) + " top"

s = Stack()
for x in [10, 20, 30]:
    s.push(x)
print(s, "| pop:", s.pop(), "| peek:", s.peek(), "| size:", s.size())
```

4.2 2. Fixed-Size Array Stack (Overflow and Underflow)

In C and embedded systems, the stack has a fixed capacity. We keep an index **top** (-1 when empty).

```
class ArrayStack:
    def __init__(self, capacity):
        self.data = [None] * capacity
        self.top = -1

    def push(self, x):
        if self.top == len(self.data) - 1:
            raise OverflowError("stack overflow")
        self.top += 1
        self.data[self.top] = x

    def pop(self):
        if self.top == -1:
            raise IndexError("stack underflow")
        x = self.data[self.top]
        self.data[self.top] = None
        self.top -= 1
        return x

a = ArrayStack(2)
a.push("A"); a.push("B")
try:
    a.push("C")
except OverflowError as e:
    print("Error:", e)
print(a.pop(), a.pop())
```

4.3 3. Linked-List Stack

The **head** of a singly linked list is the top: **push_front** and **pop_front** are $O(1)$ and the stack never needs resizing.

```

class LinkedStack:
    class _Node:
        def __init__(self, data, next):
            self.data, self.next = data, next

    def __init__(self):
        self.head = None
        self.count = 0

    def push(self, x):
        self.head = self._Node(x, self.head)
        self.count += 1

    def pop(self):
        if self.head is None:
            raise IndexError("stack underflow")
        x = self.head.data
        self.head = self.head.next
        self.count -= 1
        return x

    def peek(self):
        return None if self.head is None else self.head.data

ls = LinkedStack()
for ch in "DSA":
    ls.push(ch)
print(ls.pop() + ls.pop() + ls.pop())    # reversed: ASD

```

Implementation	push/pop	Pros	Cons
Python list	$O(1)$ amortised	Simplest, fast, cache-friendly	Occasional resize
Fixed array	$O(1)$	No resizing, predictable memory	Overflow when full
Linked list	$O(1)$	Never full, no resize	Extra memory per node

5. Part C: Classic Applications

5.1 Balanced Brackets

Scan the expression. Push every **opening** bracket. For a **closing** bracket, the top of the stack must be the matching opening bracket; pop it. At the end, the stack must be empty.

```
def is_balanced(expr):
    pairs = {")": "(", "]" : "[", "}": "{"}
    stack = []
    for ch in expr:
        if ch in "([{":
            stack.append(ch)
        elif ch in ")]}":
            if not stack or stack.pop() != pairs[ch]:
                return False
    return not stack

for e in ["{[a + b] * (c - d)}", "(a + b)", "((x)", "print(arr[i])"] :
    print(f"{e:<22} -> {is_balanced(e)}")
```

5.2 Infix, Prefix and Postfix

Notation	Example	Used by
Infix	A + B * C	Humans; needs precedence rules and brackets
Prefix (Polish)	+ A * B C	Some compilers, LISP
Postfix (Reverse Polish)	A B C * +	Calculators, stack machines; no brackets needed

5.3 Evaluating a Postfix Expression

Read tokens left to right. **Number** → push. **Operator** → pop two numbers (the first pop is the **right** operand), apply the operator, push the result. The final value on the stack is the answer.

Token	Action	Stack
5	push	[5]
3	push	[5, 3]
2	push	[5, 3, 2]
*	$3 \times 2 = 6$	[5, 6]

Token	Action	Stack
+	$5 + 6 = 11$	[11]
4	push	[11, 4]
-	$11 - 4 = 7$	[7]

```
def eval_postfix(tokens):
    stack = []
    for t in tokens.split():
        if t in "+-*/":
            right = stack.pop()
            left = stack.pop()
            if t == "+": stack.append(left + right)
            elif t == "-": stack.append(left - right)
            elif t == "*": stack.append(left * right)
            else: stack.append(left / right)
        else:
            stack.append(float(t))
    return stack.pop()

print(eval_postfix("5 3 2 * + 4 -"))      # 5 + 3*2 - 4 = 7
```

5.4 Infix to Postfix (Shunting-Yard Algorithm)

Rules: operands go straight to the output. (is pushed.) pops operators to the output until (. For an operator, first pop operators of **higher or equal precedence** (for left-associative operators), then push it. At the end, pop everything.

```

PREC = {"+": 1, "-": 1, "*": 2, "/": 2, "^": 3}

def infix_to_postfix(expr):
    out, stack = [], []
    for t in expr.split():
        if t == "(":
            stack.append(t)
        elif t == ")":
            while stack[-1] != "(":
                out.append(stack.pop())
            stack.pop() # discard "("
        elif t in PREC:
            while (stack and stack[-1] != "(" and
                   (PREC[stack[-1]] > PREC[t] or (PREC[stack[-1]] == PREC[t] and t
            != "^"))):
                out.append(stack.pop())
            stack.append(t)
        else:
            out.append(t) # operand
    while stack:
        out.append(stack.pop())
    return " ".join(out)

print(infix_to_postfix("A + B * C"))
print(infix_to_postfix("( A + B ) * C - D / E"))
print(infix_to_postfix("2 ^ 3 ^ 2")) # ^ is right-associative

```



6. Part D: Monotonic Stack

A **monotonic stack** keeps its elements in increasing or decreasing order. It solves “**next greater / smaller element**” problems in **O(n)** instead of $O(n^2)$, because each element is pushed and popped at most once.

Problem: for each day’s share price, find the next day with a **higher** price (–1 if none).

```

def next_greater(prices):
    result = [-1] * len(prices)
    stack = [] # indexes of days still waiting for a
    higher price
    for i, p in enumerate(prices):
        while stack and prices[stack[-1]] < p:
            result[stack.pop()] = p # today is the answer for that earlier
            day
        stack.append(i)
    return result

prices = [100, 80, 120, 90, 95, 130]
print(next_greater(prices))

```

7. Part E: The Call Stack and Recursion

Python itself uses a stack to run functions. Any recursive function can be rewritten using your own stack, which avoids Python's recursion limit.

```
def factorial_with_stack(n):
    stack = []
    while n > 0:
        stack.append(n)
        n -= 1
    result = 1
    while stack:
        result *= stack.pop()
    return result

import traceback
def a(): b()
def b(): print("".join(traceback.format_stack(limit=3)[-1]))
print(factorial_with_stack(10))
a() # shows the call stack: a -> b
```

8. C++ Corner

```
C++
#include <iostream>
#include <stack>
#include <string>
using namespace std;

bool isBalanced(const string& s) {
    stack<char> st;
    for (char c : s) {
        if (c == '(' || c == '[' || c == '{') st.push(c);
        else if (c == ')' || c == ']' || c == '}') {
            if (st.empty()) return false;
            char t = st.top(); st.pop();
            if ((c == ')') && t != '(') || (c == ']') && t != '[') || (c == '}') && t
            != '{')
                return false;
        }
    }
    return st.empty();
}

int main() {
    stack<int> s; // std::stack: push, pop, top, empty, size
    s.push(10); s.push(20);
    cout << s.top() << endl; // 20
    s.pop(); // note: pop() returns void in C++
    cout << isBalanced("{[()]}") << isBalanced("]") << endl; // 10
}
```

9. Common Mistakes

Mistake	Effect	Fix
Using <code>list.pop(0)</code> / <code>insert(0, x)</code> as the top	$O(n)$ per operation	Use the end of the list
Popping without checking empty	Underflow / <code>IndexError</code>	Check <code>is_empty()</code> first
Wrong operand order in postfix	<code>5 3 -</code> gives -2 instead of 2	First pop = right operand
Forgetting the stack must be empty at the end	<code>((x)</code> reported as balanced	Return <code>not stack</code>
Treating <code>^</code> as left-associative	Wrong postfix for <code>2 ^ 3 ^ 2</code>	Do not pop equal-precedence <code>^</code>

10. Lab Tasks

Solve all tasks in `Lab10_Student.ipynb`. Each task has a **test cell**: when it prints  your solution is correct. For each task, **write pseudocode first**, then code, then test with your own inputs including edge cases. Difficulty:  Easy  Medium  Hard.

Task 1 Reverse Words with a Stack

Write `reverse_words(sentence)` that reverses the **order of words** using a stack (push every word, then pop). Do not use slicing or `reversed()`.

```
"Data Structures are fun" → "fun are Structures Data"
```

Task 2 HTML Tag Checker

Web browsers must check that HTML tags are properly nested. Given a list of tags like `"<p>"` and `"</p>"`, return `True` if every closing tag matches the most recent unclosed opening tag.

Hint: same idea as balanced brackets; a closing tag starts with `</`.

```
["<html>", "<body>", "<p>", "</p>", "</body>", "</html>"] → True
["<b>", "<i>", "</b>", "</i>"] → False
```

Task 3 Min Stack

Design `MinStack` with `push`, `pop`, `top` and `get_min`, all **O(1)**. Hint: keep a second stack that stores the current minimum at each level.

```
push(5), push(2), push(7) → get_min 2; pop() → get_min 2; pop() → get_min 5
```

Task 4 Evaluate Infix Expressions

Combine `infix_to_postfix` and a postfix evaluator to write `calculate(expr)` for space-separated infix expressions with numbers, `+`, `-`, `*`, `/`, `^` and brackets. Return a float.

```
calculate("( 2 + 3 ) * 4 - 6 / 2") → 17.0
```

Task 5 Daily Temperatures

For each day in a list of Lahore temperatures, return how many days you must wait for a **warmer** day (0 if none). Use a monotonic stack for **O(n)**.

```
[31, 33, 29, 30, 35, 28] → [1, 3, 1, 1, 0, 0]
```

Task 6 Text Editor with Undo and Redo

Build **Editor** with `type(text)` (append text), `delete(k)` (remove last `k` characters), `undo()` and `redo()`, and a `text` attribute.

Use **two stacks**: every change is pushed onto the undo stack; undo moves it to the redo stack; any new change clears the redo stack.

```
type("Hello"); type(" World"); delete(6) → "Hello"
undo() → "Hello World"; undo() → "Hello"; redo() → "Hello World"
```

Task 7 Largest Rectangle in a Histogram

A bar chart of daily rainfall has bar heights `h`. Find the area of the **largest rectangle** that fits entirely under the bars. Brute force is $O(n^2)$; use a monotonic increasing stack for **$O(n)$** .

Hint: when a bar is popped, it is the shortest bar of a rectangle that extends from the new stack top + 1 to the current index - 1.

```
[2, 1, 5, 6, 2, 3] → 10      # bars 5 and 6, width 2
```



11. LeetCode Practice

Solve these on **leetcode.com** and upload screenshots of the “Accepted” result to your GitHub **DSA-Labs/Lab10** folder.

#	Problem	Level	Concept practised
20	Valid Parentheses	Easy	Balanced brackets
1047	Remove All Adjacent Duplicates In String	Easy	Stack of characters
844	Backspace String Compare	Easy	Stack simulation
155	Min Stack	Medium	Auxiliary stack (Task 3)
150	Evaluate Reverse Polish Notation	Medium	Postfix evaluation
739	Daily Temperatures	Medium	Monotonic stack (Task 5)
496	Next Greater Element I	Easy	Monotonic stack
84	Largest Rectangle in Histogram	Hard	Monotonic stack (Task 7)



12. Home Task

1. Finish all tasks and push `Lab10_Student.ipynb` to GitHub.
2. Write `infix_to_prefix(expr)` (hint: reverse the expression, swap brackets, convert to postfix, reverse the result).
3. Implement **two stacks in one array** (one grows from the left, one from the right) so that no space is wasted.
4. **Mini-project: Scientific Calculator.** Build a calculator that accepts expressions without spaces (e.g., `3+4*(2-1)^2`), tokenises them, converts to postfix, evaluates, and reports errors such as unbalanced brackets or division by zero.
5. Solve at least **3** LeetCode problems from the list.



13. Viva Questions

1. What is LIFO? Give two real-life examples.
2. What is an abstract data type?
3. List the stack operations and their time complexity.
4. What are stack overflow and underflow?
5. Which end of a Python list should be the top of the stack? Why?
6. How do you check balanced brackets using a stack?
7. Convert `(A + B) * C` to postfix.
8. Evaluate the postfix expression `6 2 / 3 +`.
9. What is a monotonic stack and why is it $O(n)$?
10. How does the call stack work during recursion?



14. Assessment Rubric

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Correctness	All tasks pass all tests	Most tasks correct	Some tasks correct	Mostly incorrect
Logic & pseudocode	Clear pseudocode and dry run for every task	Pseudocode for most tasks	Partial pseudocode	None

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Code quality	Meaningful names, functions, comments	Mostly clean	Hard to read	Copied or unreadable
Complexity & edge cases	States Big-O and handles all edge cases	Some edge cases / Big-O	Only basic cases	Not considered
Viva	Explains every line confidently	Explains most parts	Basic understanding	Cannot explain

Lab Rules

- Bring your own laptop with Colab or Jupyter working.
- Arrive on time; late arrival may result in no attendance for the lab.
- Copying another student's code or pasting AI output without understanding it will be awarded zero. You must be able to explain every line in the viva.

15. References and Further Learning

- E. W. Dijkstra's Shunting-yard algorithm (1961); see the Wikipedia article "Shunting yard algorithm".
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, **Introduction to Algorithms** (CLRS), 4th ed., MIT Press, 2022.
- M. T. Goodrich, R. Tamassia, M. H. Goldwasser, **Data Structures and Algorithms in Python**, Wiley.
- MIT OpenCourseWare 6.006 Introduction to Algorithms; Harvard CS50x (free online courses).
- Visualise code step by step: **pythontutor.com**; animated data structures: **visualgo.net**.



About the Author

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

GitHub: github.com/RamzanPUCIT

LinkedIn: linkedin.com/in/ramzan-ai

ResearchGate: researchgate.net/profile/Muhammad-Ramzan-105

© 2026 Muhammad Ramzan. Prepared for the Department of Computer Science, University of Okara.

UNIVERSITY OF OKARA

Department of Computer Science

DATA STRUCTURES & ALGORITHMS

Lab Manual

LAB 11 | Queues: Linear Queue, Circular Queue, Deque and Priority Queue

Week 11 of 16 • Duration: 3 hours (in-lab) + home practice

Prepared by

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

Applicable to: BSCS • BSIT • BSSE • BSAI • BSDS • BS Cyber Security and all programs offering DSA

Fall 2026 Edition

About This Lab Manual

This manual is part of a **16-week Data Structures & Algorithms (DSA) lab series**. It is designed so that a student starting with **zero programming knowledge** in Week 1 can confidently design, implement and analyse data structures by Week 16. Every lab builds on the previous one, so please do not skip labs.

Language policy: concepts are explained in a language-independent way (diagrams, pseudocode, dry runs). Full implementations are in **Python**, and every lab includes a **C++ Corner** so instructors teaching in C++ can use the same manual.

Every lab follows the same structure

Section	What you get
 Objectives & CLOs	What you will be able to do after this lab
 Motivation	Where this topic is used in real life, industry, research and the future (2026–2030)
 Concepts	Simple explanations with real examples, pseudocode and dry runs
 Code	Python implementation + C++ Corner
 Lab Tasks	Graded tasks:  Easy,  Medium,  Hard
 LeetCode	Real interview problems to practise
 Home Task &  Viva	Practice at home and prepare for oral questions
 Notebooks	Instructor notebook (complete code) and Student notebook (with TODO blanks and auto-check tests)

Lab 11 at a glance

Item	Details
Lab title	Queues: Linear Queue, Circular Queue, Deque and Priority Queue
Week	11 of 16
Prerequisites	Lab 09 (Circular Linked Lists), Lab 10 (Stacks)
Tools	Google Colab / Jupyter / VS Code (Python 3); any C++ compiler for the C++ Corner
Files	Lab11_Instructor.ipynb , Lab11_Student.ipynb
Next lab	Lab 12: Hashing and Hash Tables

1. Learning Objectives

By the end of this lab, you will be able to:

1. Explain the **queue ADT** and the **FIFO** principle.
2. Implement `enqueue`, `dequeue`, `front/peek`, `rear`, `is_empty`, `is_full` in $O(1)$.
3. Explain why a simple array queue wastes space and fix it with a **circular queue** (modulo arithmetic).
4. Implement a **linked-list queue** and use Python's `collections.deque`.
5. Use a **deque** (double-ended queue) and solve the **sliding window maximum** problem.
6. Understand **priority queues** and use Python's `heapq` (details in Lab 15).
7. Simulate real systems: ticket counters, printers and CPU scheduling.

Course Learning Outcomes (CLO) mapping

CLO	Description	Taxonomy level
CLO-1	Explain queue variants as abstract data types	Understand (C2)
CLO-2	Implement queues, circular queues and deques	Apply (C3)
CLO-3	Apply queues to simulations and algorithms	Apply / Analyse (C3–C4)

2. Motivation: First Come, First Served

Whenever people or tasks wait for a resource, fairness demands that **whoever came first is served first**. Think of the line at a bank, a fee-submission counter or a petrol pump. Computers face the same situation constantly: print jobs, network packets, customer orders and messages all wait in **queues**.

Analogy: the fee counter at the university

Students join the line at the **rear** and are served at the **front**. Nobody jumps the line. First In, First Out: **FIFO**. The difference from a stack: a stack removes the **newest** item; a queue removes the **oldest**.

2.1 Queues in Daily Life

Example	Type of queue
Bank, ticket or fee counter line	Simple queue (FIFO)
Hospital emergency ward (serious patients first)	Priority queue
Call centre: “your call is number 5 in the queue”	Simple queue
Foodpanda / restaurant orders in the kitchen	Queue (sometimes priority)
Traffic at a signal	Queue
Tasbeeh counting beads going round	Circular queue

2.2 Queues in Industry

- **Operating systems:** ready queues for CPU scheduling, printer spooling, keyboard buffers.
- **Networking:** routers store packets in queues; video streaming uses buffers (circular queues).
- **Message queues** (RabbitMQ, Apache Kafka, AWS SQS) connect services in large systems like Daraz, Careem and banks; orders and payments are processed through queues.
- **Web servers** queue incoming requests; **task queues** (Celery) run background jobs.
- **Breadth-First Search** (Lab 16), used in maps and social networks, is driven by a queue.

2.3 Queues in Research

- **Queueing theory** (a branch of mathematics) models waiting times in hospitals, telecom networks and traffic systems.
- **Reinforcement learning:** experience replay buffers (fixed-size deques) store recent experiences for training agents.
- **Real-time AI on sensor streams** (e.g., activity recognition) keeps the last N readings in a circular buffer.

🌟 2.4 Why It Matters in 2026–2030

💡 The world runs on asynchronous systems

Modern apps are made of many services that talk through **queues**: an order is placed, queued, processed by payment, then by delivery. AI systems queue thousands of user requests for GPUs. Understanding queues, buffers and priorities is essential for cloud, distributed systems and AI engineering.

📖 3. Part A: The Queue ADT

Operation	Meaning	Time
<code>enqueue(x)</code>	Add x at the rear	$O(1)$
<code>dequeue()</code>	Remove and return the front element	$O(1)$
<code>front()</code> / <code>peek()</code>	Return the front element without removing it	$O(1)$
<code>rear()</code>	Return the last element without removing it	$O(1)$
<code>is_empty()</code> / <code>is_full()</code>	Check the state	$O(1)$
<code>size()</code>	Number of elements	$O(1)$

FIFO in action

```

enqueue(10) enqueue(20) enqueue(30)      dequeue() -> 10
front                rear                front      rear
|                    |                    |         |
[10] -> [10, 20] -> [10, 20, 30] -> [20, 30]

```

3.1 Why Not Just Use a Python List?

`list.append(x)` is $O(1)$, but `list.pop(0)` must **shift all remaining elements left**, so it is $O(n)$. For a queue with a million items, every dequeue would move a million elements.

```

import time
from collections import deque

n = 100_000
lst = list(range(n))
t = time.perf_counter()
while lst:
    lst.pop(0)          # O(n) each
t_list = time.perf_counter() - t

dq = deque(range(n))
t = time.perf_counter()
while dq:
    dq.popleft()        # O(1) each
t_deque = time.perf_counter() - t
print(f"list.pop(0): {t_list:.3f}s    deque.popleft(): {t_deque:.4f}s")

```

4. Part B: Circular Queue (Array-Based)

With a fixed array, a simple queue moves **front** forward on every dequeue. Soon **rear** reaches the end of the array even though there is **empty space at the beginning**. A **circular queue** solves this by **wrapping around**: after the last index comes index 0. We use the **modulo** operator: **next = (i + 1) % capacity**.

Wrap-around

capacity = 5, after enqueue A B C D, dequeue A B, enqueue E F

index:	0	1	2	3	4
	[F]	[]	[C]	[D]	[E]
	^		^		
	rear		front		

F wrapped around to index 0!

```

class CircularQueue:
    def __init__(self, capacity):
        self.data = [None] * capacity
        self.capacity = capacity
        self.front = 0                # index of the front element
        self.count = 0                # number of elements

    def is_empty(self):
        return self.count == 0

    def is_full(self):
        return self.count == self.capacity

    def enqueue(self, x):
        if self.is_full():
            raise OverflowError("queue is full")
        rear = (self.front + self.count) % self.capacity
        self.data[rear] = x
        self.count += 1

    def dequeue(self):
        if self.is_empty():
            raise IndexError("queue is empty")
        x = self.data[self.front]
        self.data[self.front] = None
        self.front = (self.front + 1) % self.capacity
        self.count -= 1
        return x

    def peek(self):
        return None if self.is_empty() else self.data[self.front]

    def __str__(self):
        return str([self.data[(self.front + i) % self.capacity] for i in
range(self.count)])

q = CircularQueue(5)
for x in "ABCD":
    q.enqueue(x)
q.dequeue(); q.dequeue()
q.enqueue("E"); q.enqueue("F")
print("queue:", q, "| raw array:", q.data, "| front index:", q.front)

```



Why keep `count`?

With only **front** and **rear** indexes, an empty queue and a full queue look the same ($\text{front} == \text{rear}$). Keeping a **count** (or leaving one slot always empty) removes this ambiguity.

5. Part C: Linked-List Queue

Enqueue at the **tail** and dequeue from the **head** of a singly linked list. Both are $O(1)$ and the queue never becomes full.

```
class LinkedQueue:
    class _Node:
        def __init__(self, data):
            self.data, self.next = data, None

    def __init__(self):
        self.head = self.tail = None
        self.count = 0

    def enqueue(self, x):
        node = self._Node(x)
        if self.tail is None:
            self.head = self.tail = node
        else:
            self.tail.next = node
            self.tail = node
        self.count += 1

    def dequeue(self):
        if self.head is None:
            raise IndexError("queue is empty")
        x = self.head.data
        self.head = self.head.next
        if self.head is None:
            self.tail = None
        self.count -= 1
        return x

printer = LinkedQueue()
for job in ["CV.pdf", "Assignment.docx", "Timetable.png"]:
    printer.enqueue(job)
while printer.count:
    print("Printing", printer.dequeue())
```

↔ 6. Part D: Deque (Double-Ended Queue)

A **deque** (pronounced “deck”) allows insertion and deletion at **both ends** in $O(1)$. It can act as both a stack and a queue. Python’s `collections.deque` is the standard tool and should be your default queue in Python.

Operation	deque method	Time
Add at rear / front	<code>append(x)</code> / <code>appendleft(x)</code>	$O(1)$
Remove from rear / front	<code>pop()</code> / <code>popleft()</code>	$O(1)$
Look at ends	<code>dq[-1]</code> / <code>dq[0]</code>	$O(1)$
Fixed-size buffer	<code>deque(maxlen=k)</code>	Old items drop automatically
Rotate	<code>rotate(k)</code>	$O(k)$

```
from collections import deque

dq = deque()
dq.append("B"); dq.append("C"); dq.appendleft("A")
print(dq, dq.popleft(), dq.pop(), dq)

last_5 = deque(maxlen=5)           # e.g. last 5 heart-rate readings
for bpm in [72, 75, 80, 78, 90, 95, 70]:
    last_5.append(bpm)
print("buffer:", list(last_5), "average:", sum(last_5) / len(last_5))
```

6.1 Sliding Window Maximum with a Deque

Problem: for every window of k consecutive values, report the maximum. Keep a deque of **indexes** whose values are in **decreasing order**. The front is always the maximum of the current window. Each index enters and leaves once, so the total is $O(n)$.

```
def window_max(nums, k):
    dq, result = deque(), []
    for i, x in enumerate(nums):
        if dq and dq[0] <= i - k:          # front index left the window
            dq.popleft()
        while dq and nums[dq[-1]] < x:     # smaller values can never be a
maximum
            dq.pop()
        dq.append(i)
        if i >= k - 1:
            result.append(nums[dq[0]])
    return result

print(window_max([1, 3, -1, -3, 5, 3, 6, 7], 3))
```

★ 7. Part E: Priority Queue (Preview)

In a **priority queue**, each element has a priority and the element with the **highest priority** (or lowest value) is served first, regardless of arrival time. It is usually implemented with a **heap** (Lab 15). Python provides **heapq**, a **min-heap**: the smallest item comes out first.

```
import heapq

er = []                                # emergency room: lower number =
more serious
heapq.heappush(er, (3, "Ali - fever"))
heapq.heappush(er, (1, "Sara - accident"))
heapq.heappush(er, (2, "Usman - fracture"))
heapq.heappush(er, (3, "Zoya - cough"))
while er:
    priority, patient = heapq.heappop(er)
    print(f"Treating (priority {priority}): {patient}")
```

Structure	Removes	Typical use
Stack	Newest item (LIFO)	Undo, recursion, DFS
Queue	Oldest item (FIFO)	Scheduling, buffering, BFS
Deque	Either end	Sliding windows, palindromes, both
Priority queue	Highest-priority item	Emergency rooms, Dijkstra, task schedulers

8. C++ Corner

C++

```
#include <iostream>
#include <queue>
#include <deque>
using namespace std;

class CircularQueue {
    int* data; int cap, front = 0, count = 0;
public:
    CircularQueue(int c) : cap(c) { data = new int[c]; }
    ~CircularQueue() { delete[] data; }
    bool enqueue(int x) {
        if (count == cap) return false;           // full
        data[(front + count) % cap] = x; count++; return true;
    }
    bool dequeue(int& out) {
        if (count == 0) return false;             // empty
        out = data[front]; front = (front + 1) % cap; count--; return
true;
    }
};

int main() {
    queue<string> q;                               // std::queue: push, pop, front,
back, empty
    q.push("A"); q.push("B");
    cout << q.front() << endl; q.pop();

    deque<int> d = {2, 3}; d.push_front(1); d.push_back(4);
    priority_queue<int> pq;                        // MAX-heap by default in C++
    for (int x : {5, 1, 9}) pq.push(x);
    cout << pq.top() << endl;                     // 9

    CircularQueue cq(3); int v;
    cq.enqueue(10); cq.enqueue(20); cq.dequeue(v); cout << v << endl;
}
```

9. Common Mistakes

Mistake	Effect	Fix
Using <code>list.pop(0)</code> for dequeue	O(n) per dequeue	Use <code>collections.deque</code>
Forgetting modulo in a circular queue	IndexError at the end of the array	<code>(i + 1) % capacity</code>

Mistake	Effect	Fix
Not resetting tail when the linked queue becomes empty	Next enqueue attaches to a dead node	if head is None: tail = None
Confusing front and rear	LIFO behaviour by accident	Enqueue at rear, dequeue at front
Assuming heapq is a max-heap	Smallest comes out first	Push negative priorities for max-first

10. Lab Tasks

Solve all tasks in `Lab11_Student.ipynb`. Each task has a **test cell**: when it prints  your solution is correct. For each task, **write pseudocode first**, then code, then test with your own inputs including edge cases. Difficulty:  Easy  Medium  Hard.

Task 1 Circular Queue with Rear and Resize

Extend `CircularQueue` into `GrowingQueue` that adds a `rear()` method and, instead of raising an error when full, **doubles its capacity** (copy the elements in queue order starting at index 0).

```
GrowingQueue(2): enqueue 1, 2, 3 → capacity 4, front 1, rear 3
```

Task 2 Time to Buy Tickets

People stand in a queue to buy cricket match tickets. `tickets[i]` is how many tickets person *i* wants; each person buys **one ticket per turn** (1 second) and then goes to the back of the line if they need more. Return the time taken for the person at position *k* to finish.

Simulate it with a `deque` of `[index, remaining]` pairs.

```
tickets = [2, 3, 2], k = 2 → 6
```

Task 3 Queue Using Two Stacks

Implement `StackQueue` with `enqueue`, `dequeue`, `peek` and `is_empty` using **only two Python lists used as stacks** (`append/pop()` only).

Hint: push into an `inbox` stack; when the `outbox` stack is empty, pour everything from `inbox` into `outbox`. Each element moves at most twice, so operations are **amortised $O(1)$** .

Task 4 Recent Requests Counter

A web server wants to know how many requests arrived in the **last 3000 milliseconds**. Implement `RecentCounter.ping(t)` which records a request at time *t* (strictly increasing) and returns the number of requests in `[t - 3000, t]`. Use a `deque` and drop old requests from the front.

```
ping(1) → 1, ping(100) → 2, ping(3001) → 3, ping(3002) → 3
```

Task 5 First Non-Repeating Character in a Stream

Characters of a chat message arrive one by one. After each character, report the **first character so far that has appeared exactly once** (or **#** if none). Use a queue of candidates and a count dictionary.

```
"aabc" → "a#bb"
```

Task 6 Bank Counter Simulation

Simulate a bank with **one cashier**. Customers arrive at given minutes with given service times: `customers = [(arrival, service), ...]` sorted by arrival. The cashier serves in FIFO order.

Return `(average_wait, max_queue_length)`, where `wait` = start of service – arrival (rounded to 2 decimals) and the queue length counts customers **waiting** (not the one being served). Keep a queue of the service start times of waiting customers.

```
[(0, 5), (1, 3), (2, 1), (10, 2)] → waits 0, 4, 6, 0 → (2.5, 2)
```

Task 7 Sliding Window Maximum (Traffic Monitoring)

A traffic camera records cars per minute. For each window of `k` minutes, report the **maximum**. Implement `window_max_k(counts, k)` in $O(n)$ with a deque of indexes, and verify it against a brute-force version on random data.

```
[1, 3, -1, -3, 5, 3, 6, 7], k = 3 → [3, 3, 5, 5, 6, 7]
```



11. LeetCode Practice

Solve these on **leetcode.com** and upload screenshots of the “Accepted” result to your GitHub `DSA-Labs/Lab11` folder.

#	Problem	Level	Concept practised
933	Number of Recent Calls	Easy	Deque window (Task 4)
2073	Time Needed to Buy Tickets	Easy	Queue simulation (Task 2)
232	Implement Queue using Stacks	Easy	Two stacks (Task 3)
225	Implement Stack using Queues	Easy	Reverse thinking
1700	Number of Students Unable to Eat Lunch	Easy	Queue simulation

#	Problem	Level	Concept practised
622	Design Circular Queue	Medium	Modulo arithmetic
239	Sliding Window Maximum	Hard	Monotonic deque (Task 7)



12. Home Task

1. Finish all tasks and push `Lab11_Student.ipynb` to GitHub.
2. Implement a **deque from scratch** using a circular array with `append`, `appendleft`, `pop` and `popleft`.
3. Extend the bank simulation to **two cashiers** and compare average waiting time for the same customers.
4. **Mini-project: Hospital Emergency System.** Patients arrive with a severity (1–5). Use `heapq` so that the most severe patient is treated first and, among equal severity, the one who arrived first (hint: push `(severity, arrival_number, name)`). Print the treatment order and the average waiting time.
5. Solve at least 3 LeetCode problems from the list.

? 13. Viva Questions

1. What is FIFO? How is a queue different from a stack?
2. List the queue operations and their time complexities.
3. Why is `list.pop(0)` a bad way to dequeue?
4. What problem does a circular queue solve?
5. How do you compute the next index in a circular queue?
6. How do you distinguish a full circular queue from an empty one?
7. What is a deque? Name two uses.
8. What is a priority queue? How is it different from a normal queue?
9. How can you build a queue from two stacks? What is the amortised cost?
10. Which graph algorithm uses a queue?



14. Assessment Rubric

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Correctness	All tasks pass all tests	Most tasks correct	Some tasks correct	Mostly incorrect
Logic & pseudocode	Clear pseudocode and dry run for every task	Pseudocode for most tasks	Partial pseudocode	None
Code quality	Meaningful names, functions, comments	Mostly clean	Hard to read	Copied or unreadable
Complexity & edge cases	States Big-O and handles all edge cases	Some edge cases / Big-O	Only basic cases	Not considered
Viva	Explains every line confidently	Explains most parts	Basic understanding	Cannot explain



Lab Rules

- Bring your own laptop with Colab or Jupyter working.
- Arrive on time; late arrival may result in no attendance for the lab.
- Copying another student's code or pasting AI output without understanding it will be awarded zero. You must be able to explain every line in the viva.



15. References and Further Learning

- Python `queue`, `collections.deque` and `heapq` documentation: docs.python.org/3/library
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, **Introduction to Algorithms** (CLRS), 4th ed., MIT Press, 2022.
- M. T. Goodrich, R. Tamassia, M. H. Goldwasser, **Data Structures and Algorithms in Python**, Wiley.
- MIT OpenCourseWare 6.006 Introduction to Algorithms; Harvard CS50x (free online courses).
- Visualise code step by step: pythontutor.com; animated data structures: visualgo.net.



About the Author

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

GitHub: github.com/RamzanPUCIT

LinkedIn: linkedin.com/in/ramzan-ai

ResearchGate: researchgate.net/profile/Muhammad-Ramzan-105

© 2026 Muhammad Ramzan. Prepared for the Department of Computer Science, University of Okara.

UNIVERSITY OF OKARA

Department of Computer Science

DATA STRUCTURES & ALGORITHMS

Lab Manual

LAB 12 | Hashing: Hash Functions, Collisions and Hash Tables

Week 12 of 16 • Duration: 3 hours (in-lab) + home practice

Prepared by

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

Applicable to: BSCS • BSIT • BSSE • BSAI • BSDS • BS Cyber Security and all programs offering DSA

Fall 2026 Edition

About This Lab Manual

This manual is part of a **16-week Data Structures & Algorithms (DSA) lab series**. It is designed so that a student starting with **zero programming knowledge** in Week 1 can confidently design, implement and analyse data structures by Week 16. Every lab builds on the previous one, so please do not skip labs.

Language policy: concepts are explained in a language-independent way (diagrams, pseudocode, dry runs). Full implementations are in **Python**, and every lab includes a **C++ Corner** so instructors teaching in C++ can use the same manual.

Every lab follows the same structure

Section	What you get
 Objectives & CLOs	What you will be able to do after this lab
 Motivation	Where this topic is used in real life, industry, research and the future (2026–2030)
 Concepts	Simple explanations with real examples, pseudocode and dry runs
 Code	Python implementation + C++ Corner
 Lab Tasks	Graded tasks:  Easy,  Medium,  Hard
 LeetCode	Real interview problems to practise
 Home Task &  Viva	Practice at home and prepare for oral questions
 Notebooks	Instructor notebook (complete code) and Student notebook (with TODO blanks and auto-check tests)

Lab 12 at a glance

Item	Details
Lab title	Hashing: Hash Functions, Collisions and Hash Tables
Week	12 of 16
Prerequisites	Lab 03 (Arrays), Lab 08 (Linked Lists)
Tools	Google Colab / Jupyter / VS Code (Python 3); any C++ compiler for the C++ Corner
Files	Lab12_Instructor.ipynb , Lab12_Student.ipynb
Next lab	Lab 13: Trees and Binary Trees

1. Learning Objectives

By the end of this lab, you will be able to:

1. Explain how hashing gives **O(1) average** search, insert and delete.
2. Design simple **hash functions** and explain the properties of a good one.
3. Explain **collisions** and resolve them with **separate chaining** and **open addressing (linear probing)**.
4. Explain **load factor** and **rehashing (resizing)**.
5. Implement your own **hash map** from scratch.
6. Use Python's **dict**, **set**, **Counter** and **defaultdict** professionally.
7. Solve classic problems with hashing: frequency counting, two-sum, anagrams and prefix sums.

Course Learning Outcomes (CLO) mapping

CLO	Description	Taxonomy level
CLO-1	Explain hash functions and collision resolution	Understand (C2)
CLO-2	Implement a hash table	Apply (C3)
CLO-3	Apply hashing to design efficient solutions	Analyse (C4)

2. Motivation: Finding Anything in One Step

Binary search finds an item among a million sorted items in about 20 steps. Can we do better? **Hashing** can find it in about **one step**, on average, without sorting at all. That is why Python's dictionary and set feel instant even with millions of items.

Analogy: student lockers by roll number

Imagine a university with 1,000 lockers. Instead of searching every locker for Ali's bag, the rule is: **locker number = last 3 digits of the roll number**. Ali (roll 2023-417) always goes straight to locker 417. The rule is the **hash function**; the lockers are the **hash table**. If two students have the same last 3 digits, we have a **collision** and need a plan.

2.1 Hashing in Daily Life

Example	Key → location
Phone contacts / WhatsApp search	Name → contact details

Example	Key → location
Dictionary app	Word → meaning
NADRA / bank systems	CNIC → citizen or account record
Daraz cart	Product ID → quantity
Password check on login	Password → hash stored in the database (never the password itself)
Spell checker	Is this word in the set of valid words?



2.2 Hashing in Industry

- **Databases** use hash indexes and hash joins; **caches** such as Redis and Memcached are giant hash tables.
- **Programming languages:** Python `dict/set`, Java `HashMap`, C++ `unordered_map`, JavaScript objects.
- **Security:** passwords are stored as cryptographic hashes (SHA-256, bcrypt); file downloads are verified with checksums.
- **Git** identifies every commit and file by its SHA hash; **blockchains** link blocks with hashes.
- **Load balancers** use consistent hashing to send each user to the same server.



2.3 Hashing in Research

- **Locality-Sensitive Hashing (LSH)** finds similar images, documents and audio quickly; it powers near-duplicate detection and recommendation systems.
- **Vector search for AI** (RAG, semantic search) uses hashing and graph-based indexes to search millions of embeddings.
- **Bloom filters** (probabilistic hash sets) are used in databases, networking and genomics to save memory.
- **Content-based image retrieval** often uses learned binary hash codes for fast search.



2.4 Why It Matters in 2026–2030



The default tool for fast lookup

In real-world coding interviews, the first improvement to a slow solution is often “**use a hash map**”. As data grows and AI systems search ever-larger collections, hashing (exact and similarity-based) remains one of the most important ideas in computer science.



3. Part A: Hash Functions

A **hash function** converts a **key** (number, string, tuple...) into an **index** of an array called the **hash table** (or bucket array). The table stores the value at that index.

Key → hash → index

key "Ali" --hash function--> 2 --> table[2] = ("Ali", 3.4)

index:	0	1	2	3	4
	[]	[]	[Ali: 3.4]	[]	[]

3.1 A Simple Hash Function for Integers

The **division method**: $h(x) = x \% m$, where m is the table size. With $m = 10$, the values 11, 12, 13, 14, 15 go to indexes 1, 2, 3, 4, 5 (exactly the example in your theory slides). A **prime** table size usually spreads keys better.

```
def h(x, m=10):
    return x % m

for key in [11, 12, 13, 14, 15, 25, 35]:
    print(key, "->", h(key))          # 15, 25, 35 all go to 5:
collisions!
```

3.2 Hashing Strings

```
def bad_hash(s, m):                                # sum of character codes
    return sum(ord(c) for c in s) % m

def poly_hash(s, m, base=31):                      # polynomial rolling hash (used in
    h = 0                                           Java)
    for c in s:
        h = (h * base + ord(c)) % m
    return h

for w in ["listen", "silent", "enlist"]:
    print(f"{w:<7} bad={bad_hash(w, 101):>3} poly={poly_hash(w,
101):>3}")
```

Anagrams have the same characters, so the sum-of-codes hash puts them all in the same bucket. The polynomial hash uses the **position** of each character too, so it spreads them out.

Property of a good hash function	Why
Deterministic	The same key must always give the same index
Fast to compute	Hashing must be $O(1)$ (or $O(\text{length of key})$)
Uniform distribution	Spreads keys evenly to minimise collisions
Uses the whole key	Small differences in keys should give different indexes

Python's built-in hash()

```
print(hash(42), hash("0kara") % 100, hash((1, 2)) % 100)
try:
    hash([1, 2])
except TypeError as e:
    print("Lists cannot be keys:", e)
```

Only **immutable** objects (int, str, tuple of immutables) can be dictionary keys or set items. If a key could change after insertion, its hash would change and it could never be found again.

4. Part B: Collisions and How to Resolve Them

A **collision** happens when two different keys hash to the same index. Collisions are **unavoidable** (there are more possible keys than indexes, the **pigeonhole principle**), so every hash table needs a strategy.

4.1 Strategy 1: Separate Chaining

Each slot holds a **list (chain)** of all key-value pairs that hash there. To search, hash the key and scan only that short chain.

Separate chaining

$m = 5$, keys: 15, 22, 7, 30, 12 ($h = \text{key} \% 5$)

0: [15] -> [30]

1:

2: [22] -> [7] -> [12]

3:

4:

4.2 Strategy 2: Open Addressing (Linear Probing)

All items live in the array itself. If the slot is taken, try the **next** slot ($((i + 1) \% m)$), and keep going until an empty slot is found. Search follows the same path. Deleting requires a special **tombstone** marker so later searches do not stop early.

Linear probing

$m = 7$, insert 10, 17, 24, 5 ($h = \text{key} \% 7$)

10 -> 3 [_, _, _, 10, _, _, _]

17 -> 3 taken, try 4 [_, _, _, 10, 17, _, _]

24 -> 3, 4 taken, try 5 [_, _, _, 10, 17, 24, _]

5 -> 5 taken, try 6 [_, _, _, 10, 17, 24, 5]

	Separate chaining	Linear probing
Where items live	Lists hanging from each slot	Inside the array
Load factor can exceed 1?	Yes	No (must stay below ~0.7)
Deletion	Easy	Needs tombstones
Cache performance	Poorer (pointers)	Excellent (contiguous)
Problem	Long chains if the hash is bad	Clustering: long runs of filled slots

4.3 Load Factor and Rehashing

The **load factor** $\alpha = n / m$ (items $\div$ slots). As α grows, chains get longer (or probe sequences get longer) and operations slow down. When α passes a threshold (e.g., 0.75), the table **doubles its size** and **re-inserts every item** with the new m . Like the dynamic array in Lab 03, this keeps operations **$O(1)$ amortised**.

Operation	Average	Worst (all keys collide)
Search	$O(1)$	$O(n)$
Insert	$O(1)$ amortised	$O(n)$
Delete	$O(1)$	$O(n)$

5. Part C: Build Your Own Hash Map (Separate Chaining)

```
class HashMap:
    def __init__(self, capacity=8):
        self.buckets = [[] for _ in range(capacity)]
        self.size = 0

    def _index(self, key):
        return hash(key) % len(self.buckets)

    def put(self, key, value):
        bucket = self.buckets[self._index(key)]
        for pair in bucket:
            if pair[0] == key:          # key exists: update
                pair[1] = value
                return
        bucket.append([key, value])     # new key
        self.size += 1
        if self.size / len(self.buckets) > 0.75:
            self._resize()

    def get(self, key, default=None):
        for k, v in self.buckets[self._index(key)]:
            if k == key:
                return v
        return default

    def remove(self, key):
        bucket = self.buckets[self._index(key)]
        for i, (k, v) in enumerate(bucket):
            if k == key:
                bucket.pop(i)
                self.size -= 1
                return v
        raise KeyError(key)

    def _resize(self):
        old = self.buckets
        self.buckets = [[] for _ in range(2 * len(old))]
        self.size = 0
        for bucket in old:
```

```

        for k, v in bucket:
            self.put(k, v)                # re-hash into the bigger table

    def __len__(self):
        return self.size

cgpa = HashMap()
for name, g in [("Ali", 3.4), ("Sara", 3.9), ("Hamza", 3.1), ("Zoya", 3.7), ("Bilal",
2.9),
                ("Iqra", 3.5), ("Omar", 3.2)]:
    cgpa.put(name, g)
cgpa.put("Ali", 3.6)                    # update
print(cgpa.get("Ali"), cgpa.get("Nobody", "not found"), len(cgpa), "buckets:",
len(cgpa.buckets))
print("longest chain:", max(len(b) for b in cgpa.buckets))

```



6. Part D: Hashing Tools in Python

```

from collections import Counter, defaultdict

votes = ["Ali", "Sara", "Hamza", "Ali", "Hamza", "Ali"] # class representative
election
tally = Counter(votes)                                # frequency counting in one line
print(tally, tally.most_common(1))

by_city = defaultdict(list)                           # missing keys get a default empty
list
for name, city in [("Ali", "Okara"), ("Sara", "Lahore"), ("Hamza", "Okara")]:
    by_city[city].append(name)
print(dict(by_city))

bscs, bsit = {"Ali", "Sara", "Hamza"}, {"Sara", "Usman"}
print("both:", bscs & bsit, "| only BSCS:", bscs - bsit, "| any:", bscs | bsit)

marks = {"Ali": 81, "Sara": 92}
print(marks.get("Zoya", 0), "Ali" in marks, list(marks.items()))

```

6.1 Cryptographic Hashes (Security)

```

import hashlib
pw = "okara123"
print(hashlib.sha256(pw.encode()).hexdigest())
print(hashlib.sha256("okara124".encode()).hexdigest()) # tiny change -> totally
different

```

Cryptographic hash functions are designed so that you cannot reverse them or find two inputs with the same hash. Websites store this hash (with a random **salt**), not your password. These are different from the fast hash functions used in hash tables.

7. C++ Corner

C++

```
#include <iostream>
#include <unordered_map>
#include <unordered_set>
#include <vector>
#include <list>
using namespace std;

int main() {
    unordered_map<string, double> cgpa;      // hash table (separate
chaining)
    cgpa["Ali"] = 3.4; cgpa["Sara"] = 3.9;
    cout << cgpa["Sara"] << " " << cgpa.count("Zoya") << endl;    // 3.9 0
    cout << "load factor: " << cgpa.load_factor() << endl;

    unordered_set<int> seen = {1, 2, 3};
    cout << (seen.find(2) != seen.end()) << endl;                // 1

    // A tiny chaining table by hand
    vector<list<int>> table(5);
    for (int k : {15, 22, 7, 30, 12}) table[k % 5].push_back(k);
    for (int i = 0; i < 5; i++) {
        cout << i << ": ";
        for (int k : table[i]) cout << k << " -> ";
        cout << "NULL" << endl;
    }
}
```

8. Common Mistakes

Mistake	Effect	Fix
Using a list as a dictionary key	TypeError: unhashable type	Use a tuple
<code>dict[key]</code> for a missing key	KeyError	Use <code>.get(key, default)</code> or <code>in</code>
Checking <code>x in list</code> inside a loop	Hidden $O(n^2)$	Convert the list to a set first
Assuming dict/set order means sorted	Insertion order, not sorted	Use <code>sorted()</code> when needed
Forgetting to resize a custom hash table	Long chains, $O(n)$ operations	Rehash when load factor > 0.75

Mistake	Effect	Fix
Deleting from a probing table by emptying the slot	Later keys become unreachable	Use a tombstone marker

9. Lab Tasks

Solve all tasks in `Lab12_Student.ipynb`. Each task has a **test cell**: when it prints  your solution is correct. For each task, **write pseudocode first**, then code, then test with your own inputs including edge cases. Difficulty:  Easy  Medium  Hard.

Task 1 Word Frequency Report

Write `top_words(text, k)` that returns the `k` most frequent words (lower-case, punctuation `.,!?` removed) as a list of `(word, count)` tuples, sorted by count descending and then alphabetically. Use a dictionary (do not use `Counter`).

```
top_words("Data is data. Data is power!", 2) → [("data", 3), ("is", 2)]
```

Task 2 Linear Probing Hash Set

Complete `ProbingSet(capacity)` with `add(key)`, `contains(key)` and `remove(key)` using **open addressing with linear probing** and a **DELETED** tombstone. Assume the table never becomes full (no resizing needed).

```
add 10, 17, 24 (capacity 7) → slots 3, 4, 5; remove 17; contains(24) → True
```

Task 3 Group Anagrams

Group words that are anagrams of each other. Use a dictionary whose key is a **canonical form** of the word (its sorted letters as a tuple or string). Return the groups sorted for easy checking: sort each group, then sort the list of groups.

```
["eat", "tea", "tan", "ate", "nat", "bat"] → [
  ["ate", "eat", "tea"], ["bat"],
  ["nat", "tan"]]

```

Task 4 Complete the Hash Map

Extend `HashMap` from Part C into `MyMap` with: `__contains__(key)` (so `key in m` works), `keys()` (list of all keys), `__getitem__`/`__setitem__` (so `m[k]` works; missing keys raise `KeyError`) and `load_factor()`.

Test it with 1,000 keys and check that the load factor stays at or below 0.75.

Task 5 Longest Consecutive Sequence

Given unsorted roll numbers, find the length of the **longest run of consecutive numbers** in $O(n)$ (sorting would be $O(n \log n)$).

Hint: put everything in a set. A number starts a run only if $x - 1$ is not in the set; from each start, count upwards.

```
[100, 4, 200, 1, 3, 2] → 4    (1, 2, 3, 4)
```

Task 6 ● Subarray Sum Equals K (Prefix Sums + Hashing)

A shop's daily profit/loss values are given (can be negative). Count how many **contiguous** periods have a total exactly equal to k .

Hint: keep a running prefix sum and a dictionary counting how many times each prefix sum has occurred. A period ending today has sum k if $\text{prefix} - k$ occurred before. This is $O(n)$ instead of $O(n^2)$.

```
[1, 1, 1], k = 2 → 2
[3, 4, 7, 2, -3, 1, 4, 2], k = 7 → 4
```

Task 7 ● Detect Plagiarism with Rolling Hashes

Two students submitted assignments. Find the **length of the longest common substring** of the two strings using **binary search on the length** and a **rolling (polynomial) hash** set of all substrings of that length (Rabin–Karp idea).

To keep things simple and safe, after finding hash matches, confirm by comparing the actual substrings.

```
longest_common_substring("datastructures", "structural") → 8
("structur")
```



10. LeetCode Practice

Solve these on **leetcode.com** and upload screenshots of the “Accepted” result to your GitHub [DSA-Labs/Lab12](#) folder.

#	Problem	Level	Concept practised
1	Two Sum	Easy	Hash map of complements
217	Contains Duplicate	Easy	Hash set
242	Valid Anagram	Easy	Frequency counting
387	First Unique Character in a String	Easy	Counting
705	Design HashSet	Easy	Build a hash table
706	Design HashMap	Easy	Chaining (Task 4)

#	Problem	Level	Concept practised
49	Group Anagrams	Medium	Canonical keys (Task 3)
128	Longest Consecutive Sequence	Medium	Set (Task 5)
560	Subarray Sum Equals K	Medium	Prefix sums (Task 6)



11. Home Task

1. Finish all tasks and push `Lab12_Student.ipynb` to GitHub.
2. Add **resizing** to `ProbingSet` (rehash when the load factor exceeds 0.6) and count how many probes are needed on average before and after.
3. Implement **quadratic probing** ($i, i+1^2, i+2^2, \dots$) and **double hashing**, and compare clustering with linear probing on 5,000 random keys.
4. **Mini-project: Student Record System.** Store at least 1,000 generated student records in your own `MyMap`, keyed by roll number. Support add, search, update, delete, and a report of the longest chain and load factor. Compare search time with a plain list.
5. Solve at least 3 LeetCode problems from the list.



12. Viva Questions

1. What is hashing? Why is lookup $O(1)$ on average?
2. What is a hash function? List three properties of a good one.
3. What is a collision? Why are collisions unavoidable?
4. Explain separate chaining with an example.
5. Explain linear probing. What is clustering?
6. Why does deletion in open addressing need a tombstone?
7. What is the load factor? What happens when it gets too high?
8. Why can a Python list not be a dictionary key?
9. What is the worst-case complexity of a hash table lookup and when does it happen?
10. How is a cryptographic hash different from a hash-table hash?



13. Assessment Rubric

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Correctness	All tasks pass all tests	Most tasks correct	Some tasks correct	Mostly incorrect
Logic & pseudocode	Clear pseudocode and dry run for every task	Pseudocode for most tasks	Partial pseudocode	None
Code quality	Meaningful names, functions, comments	Mostly clean	Hard to read	Copied or unreadable
Complexity & edge cases	States Big-O and handles all edge cases	Some edge cases / Big-O	Only basic cases	Not considered
Viva	Explains every line confidently	Explains most parts	Basic understanding	Cannot explain



Lab Rules

- Bring your own laptop with Colab or Jupyter working.
- Arrive on time; late arrival may result in no attendance for the lab.
- Copying another student's code or pasting AI output without understanding it will be awarded zero. You must be able to explain every line in the viva.



14. References and Further Learning

- Python **collections** (Counter, defaultdict): docs.python.org/3/library/collections.html
- Python **hashlib**: docs.python.org/3/library/hashlib.html
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, **Introduction to Algorithms** (CLRS), 4th ed., MIT Press, 2022.
- M. T. Goodrich, R. Tamassia, M. H. Goldwasser, **Data Structures and Algorithms in Python**, Wiley.
- MIT OpenCourseWare 6.006 Introduction to Algorithms; Harvard CS50x (free online courses).

- Visualise code step by step: **pythontutor.com**; animated data structures: **visualgo.net**.



About the Author

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

GitHub: github.com/RamzanPUCIT

LinkedIn: linkedin.com/in/ramzan-ai

ResearchGate: researchgate.net/profile/Muhammad-Ramzan-105

© 2026 Muhammad Ramzan. Prepared for the Department of Computer Science, University of Okara.

UNIVERSITY OF OKARA

Department of Computer Science

DATA STRUCTURES & ALGORITHMS

Lab Manual

LAB 13 | Trees and Binary Trees: Terminology, Representation and Traversals

Week 13 of 16 • Duration: 3 hours (in-lab) + home practice

Prepared by

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

Applicable to: BSCS • BSIT • BSSE • BSAI • BSDS • BS Cyber Security and all programs offering DSA

Fall 2026 Edition

About This Lab Manual

This manual is part of a **16-week Data Structures & Algorithms (DSA) lab series**. It is designed so that a student starting with **zero programming knowledge** in Week 1 can confidently design, implement and analyse data structures by Week 16. Every lab builds on the previous one, so please do not skip labs.

Language policy: concepts are explained in a language-independent way (diagrams, pseudocode, dry runs). Full implementations are in **Python**, and every lab includes a **C++ Corner** so instructors teaching in C++ can use the same manual.

Every lab follows the same structure

Section	What you get
 Objectives & CLOs	What you will be able to do after this lab
 Motivation	Where this topic is used in real life, industry, research and the future (2026–2030)
 Concepts	Simple explanations with real examples, pseudocode and dry runs
 Code	Python implementation + C++ Corner
 Lab Tasks	Graded tasks:  Easy,  Medium,  Hard
 LeetCode	Real interview problems to practise
 Home Task &  Viva	Practice at home and prepare for oral questions
 Notebooks	Instructor notebook (complete code) and Student notebook (with TODO blanks and auto-check tests)

Lab 13 at a glance

Item	Details
Lab title	Trees and Binary Trees: Terminology, Representation and Traversals
Week	13 of 16
Prerequisites	Lab 06 (Recursion), Lab 10 (Stacks), Lab 11 (Queues)
Tools	Google Colab / Jupyter / VS Code (Python 3); any C++ compiler for the C++ Corner
Files	Lab13_Instructor.ipynb , Lab13_Student.ipynb
Next lab	Lab 14: Binary Search Trees

1. Learning Objectives

By the end of this lab, you will be able to:

1. Explain why trees are **hierarchical (non-linear)** data structures and where they are used.
2. Use tree terminology correctly: **root, parent, child, leaf, sibling, edge, depth, height, level, subtree, degree**.
3. Identify **full, complete, perfect, balanced and degenerate** binary trees.
4. Represent a binary tree with **linked nodes** and with an **array** (index formulas).
5. Implement **preorder, inorder, postorder** (recursive and iterative) and **level-order** traversals.
6. Write recursive functions on trees: size, height, leaves, sum, mirror.
7. Build and evaluate an **expression tree**.

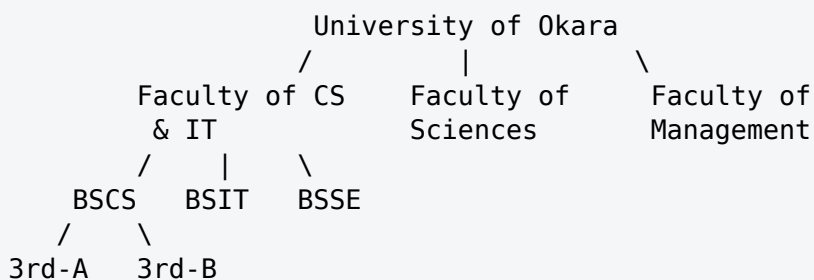
Course Learning Outcomes (CLO) mapping

CLO	Description	Taxonomy level
CLO-1	Explain tree terminology and binary tree types	Understand (C2)
CLO-2	Implement binary trees and traversals	Apply (C3)
CLO-3	Analyse recursive tree algorithms	Analyse (C4)

2. Motivation: Data with a Hierarchy

Arrays, linked lists, stacks and queues are **linear**: every element has one before and one after. But much real data is **hierarchical**: a university has faculties, which have departments, which have programs, which have sections. A **tree** represents such “one-to-many” relationships naturally.

A real hierarchy is a tree



2.1 Trees in Daily Life

Example	What forms the tree
Folders on your laptop (C: → Users → Documents ...)	Folders contain folders and files
Family tree (shajra)	Ancestors and descendants
Organisation chart	VC → Deans → HODs → Teachers
Cricket tournament knock-out bracket	Winners move up to the final (a binary tree!)
Website menus	Menu → sub-menu → page
Book contents	Chapters → sections → sub-sections

2.2 Trees in Industry

- **File systems** and the **HTML DOM** (every web page) are trees.
- **Databases** use B-trees and B+ trees for indexes; MySQL, PostgreSQL and Oracle all depend on them.
- **Compilers** turn your code into an **Abstract Syntax Tree** before running it.
- **Compression** (ZIP, JPEG) uses Huffman trees; **routing** and **auto-complete** use tries.
- **JSON and XML** data, used by every web API, are trees.

2.3 Trees in Research

- **Machine learning:** Decision Trees, Random Forests and Gradient Boosting (XGBoost, LightGBM) are among the most widely used models on tabular data.
- **AI search:** game trees and Monte Carlo Tree Search (AlphaGo) explore moves as trees.
- **NLP:** parse trees show the grammatical structure of sentences.
- **GIS & Remote Sensing:** quadtrees, k-d trees and R-trees index spatial data for fast map queries.
- **Biology:** phylogenetic trees show evolutionary relationships.

2.4 Why It Matters in 2026–2030



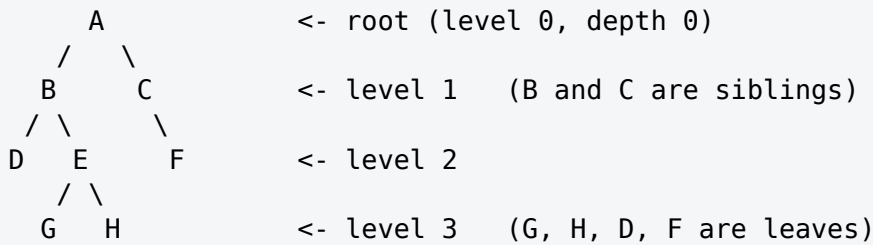
From data structures to AI models

Trees connect DSA directly to AI: a Decision Tree is literally a binary tree where each internal node asks a question about the data. Tree-based models still win many real-world data competitions. Understanding traversal and recursion on trees is the foundation for BSTs, heaps, tries, graph algorithms and these models.



3. Part A: Tree Terminology

Example tree used in this lab



Term	Meaning	In the example
Node	An element of the tree	A, B, ... H (8 nodes)
Edge	A link between parent and child	7 edges (always $n - 1$)
Root	The top node with no parent	A
Parent / Child	A node directly above / below another	B is parent of D and E
Siblings	Nodes with the same parent	D and E; G and H
Leaf	A node with no children	D, G, H, F
Internal node	A node with at least one child	A, B, C, E
Degree	Number of children of a node	$\text{deg}(B) = 2$, $\text{deg}(C) = 1$
Depth of a node	Edges from the root to the node	$\text{depth}(E) = 2$
Height of a node	Edges on the longest path down to a leaf	$\text{height}(B) = 2$
Height of the tree	Height of the root	3
Subtree	A node and all its descendants	B, D, E, G, H
Ancestor / Descendant	Nodes on the path up / below	A, B, E are ancestors of G

Some books define height in terms of nodes instead of edges (so a single node has height 1). In this manual, height counts edges: an empty tree has height -1 and a single node has height 0.

4. Part B: Binary Trees

A **binary tree** is a tree where every node has **at most two children**, called the **left child** and the **right child**.

Type	Definition	Example use
Full (strict)	Every node has 0 or 2 children	Expression trees, Huffman trees
Complete	All levels full except possibly the last, which is filled left to right	Heaps (Lab 15)
Perfect	All internal nodes have 2 children and all leaves are at the same level	Tournament brackets
Balanced	Left and right subtree heights differ by at most 1 at every node	AVL / Red-Black trees
Degenerate (skewed)	Every node has only one child; behaves like a linked list	Worst case for BSTs

4.1 Important Properties

- Maximum nodes at level $k = 2^k$.
- Maximum nodes in a binary tree of height $h = 2^{h+1} - 1$ (a perfect tree).
- Minimum height of a binary tree with n nodes = $\lceil \log_2 n \rceil$. This is why balanced trees give $O(\log n)$ operations.
- In a full binary tree, number of leaves = number of internal nodes + 1.

4.2 Representation 1: Linked Nodes

```
class TreeNode:
    def __init__(self, data, left=None, right=None):
        self.data = data
        self.left = left
        self.right = right

def build_tree(values):
    """Build a binary tree from a level-order list; None means 'no
    node'."""
    if not values or values[0] is None:
        return None
    from collections import deque
    root = TreeNode(values[0])
    q, i = deque([root]), 1
    while q and i < len(values):
        node = q.popleft()
        if i < len(values) and values[i] is not None:
            node.left = TreeNode(values[i]); q.append(node.left)
        i += 1
        if i < len(values) and values[i] is not None:
            node.right = TreeNode(values[i]); q.append(node.right)
        i += 1
    return root

# Build the example tree by hand
G, H = TreeNode("G"), TreeNode("H")
D, E = TreeNode("D"), TreeNode("E", G, H)
F = TreeNode("F")
B_, C_ = TreeNode("B", D, E), TreeNode("C", None, F)
root = TreeNode("A", B_, C_)

def show(node, indent="", label="root"):
    """Print the tree sideways: right subtree on top."""
    if node is None:
        return
    show(node.right, indent + "      ", "R")
    print(f"{indent}{label}: {node.data}")
    show(node.left, indent + "      ", "L")

show(root)
```

4.3 Representation 2: Array (for Complete Trees)

Store nodes level by level in an array. For the node at index **i** (0-based): left child at **2i + 1**, right child at **2i + 2**, parent at **(i - 1) // 2**. No pointers needed; this is exactly how heaps are stored in Lab 15.

```
tree = ["A", "B", "C", "D", "E", "F", "G"]    # a complete tree
i = 1                                         # node "B"
print("node:", tree[i], "| left:", tree[2 * i + 1], "| right:", tree[2 * i
+ 2],
      "| parent:", tree[(i - 1) // 2])
```



5. Part C: Tree Traversals

A **traversal** visits every node exactly once. Depth-first traversals differ only in **when** the current node is visited relative to its subtrees:

Traversal	Order	Result on the example tree	Typical use
Preorder	Node → Left → Right	A B D E G H C F	Copy a tree, print folder structure
Inorder	Left → Node → Right	D B G E H A C F	Sorted order in a BST (Lab 14)
Postorder	Left → Right → Node	D G H E B F C A	Delete a tree, compute folder sizes
Level order	Level by level, left to right	A B C D E F G H	Shortest paths, printing by levels



Memory trick

The name tells you where the **node** goes: **Pre** = node first, **In** = node in the middle, **Post** = node last. Left always comes before right.

5.1 Recursive Depth-First Traversals

```
def preorder(node, out):
    if node:
        out.append(node.data)          # visit
        preorder(node.left, out)
        preorder(node.right, out)
    return out

def inorder(node, out):
    if node:
        inorder(node.left, out)
        out.append(node.data)          # visit
        inorder(node.right, out)
    return out

def postorder(node, out):
    if node:
        postorder(node.left, out)
        postorder(node.right, out)
        out.append(node.data)          # visit
    return out

print("pre :", " ".join(preorder(root, [])))
print("in  :", " ".join(inorder(root, [])))
print("post:", " ".join(postorder(root, [])))
```

5.2 Level-Order Traversal with a Queue (Breadth-First)

```
from collections import deque

def level_order(root):
    if root is None:
        return []
    out, q = [], deque([root])
    while q:
        node = q.popleft()
        out.append(node.data)
        if node.left:
            q.append(node.left)
        if node.right:
            q.append(node.right)
    return out

print("level:", " ".join(level_order(root)))
```

5.3 Iterative Preorder with a Stack

Recursion uses the call stack; we can use our own stack instead. Push the right child **before** the left child so that the left one is processed first.

```
def preorder_iter(root):
    if root is None:
        return []
    out, stack = [], [root]
    while stack:
        node = stack.pop()
        out.append(node.data)
        if node.right:
            stack.append(node.right)
        if node.left:
            stack.append(node.left)
    return out

print("pre (stack):", " ".join(preorder_iter(root)))
```

Every traversal visits each node once, so all are **$O(n)$** time. Space is **$O(h)$** for DFS (the recursion/stack depth) and **$O(w)$** for BFS, where w is the maximum width of the tree.



6. Part D: Recursive Thinking on Trees

Almost every tree problem follows one pattern: **solve for the left subtree, solve for the right subtree, combine with the current node**. The base case is the empty tree (**None**).

```
def size(node):
    return 0 if node is None else 1 + size(node.left) + size(node.right)

def height(node):
    if node is None:
        return -1 # empty tree
    return 1 + max(height(node.left), height(node.right))

def count_leaves(node):
    if node is None:
        return 0
    if node.left is None and node.right is None:
        return 1
    return count_leaves(node.left) + count_leaves(node.right)

print("size:", size(root), "| height:", height(root), "| leaves:",
      count_leaves(root))
```



7. Part E: Expression Trees

An arithmetic expression can be stored as a binary tree: **leaves are numbers** and **internal nodes are operators**. Inorder traversal (with brackets) gives the infix form,

postorder gives the postfix form (Lab 10), and evaluating is a postorder computation.

Expression tree



```

expr = TreeNode("*", TreeNode("+", TreeNode(3), TreeNode(5)),
                  TreeNode("-", TreeNode(8), TreeNode(2)))

def evaluate(node):
    if node.left is None and node.right is None:
        return node.data # a number
    a, b = evaluate(node.left), evaluate(node.right)
    return {"+": a + b, "-": a - b, "*": a * b, "/": a / b}[node.data]

def to_infix(node):
    if node.left is None and node.right is None:
        return str(node.data)
    return f"({to_infix(node.left)} {node.data} {to_infix(node.right)})"

print(to_infix(expr), "=", evaluate(expr))
print("postfix:", " ".join(str(x) for x in postorder(expr, [])))
  
```

8. C++ Corner

C++

```
#include <iostream>
#include <queue>
#include <algorithm>
using namespace std;

struct Node {
    char data; Node* left; Node* right;
    Node(char d, Node* l = nullptr, Node* r = nullptr) : data(d), left(l), right(r) {}
};

void inorder(Node* n) { if (!n) return; inorder(n->left); cout << n->data << " "; inorder(n->right); }
int height(Node* n) { return n ? 1 + max(height(n->left), height(n->right)) : -1; }

void levelOrder(Node* root) {
    queue<Node*> q; if (root) q.push(root);
    while (!q.empty()) {
        Node* n = q.front(); q.pop();
        cout << n->data << " ";
        if (n->left) q.push(n->left);
        if (n->right) q.push(n->right);
    }
}

void destroy(Node* n) { if (!n) return; destroy(n->left); destroy(n->right); delete n; } // postorder!

int main() {
    Node* root = new Node('A', new Node('B', new Node('D'),
                                           new Node('E', new Node('G'), new Node('H'))),
                           new Node('C', nullptr, new Node('F')));
    inorder(root); cout << endl; // D B G E H A C F
    levelOrder(root); cout << endl; // A B C D E F G H
    cout << height(root) << endl; // 3
    destroy(root);
}
```

9. Common Mistakes

Mistake	Effect	Fix
Forgetting the None base case	AttributeError on None.left	Start every recursive function with if node is None
Mixing up height in nodes vs edges	Off-by-one answers	State your definition; empty tree = -1 here
Pushing left before right in iterative preorder	Right subtree printed first	Push right first, then left
Using a list with pop(0) for BFS	$O(n^2)$ for big trees	Use collections.deque

Mistake	Effect	Fix
Returning the result of only one subtree	Wrong counts / sums	Combine both left and right results

10. Lab Tasks

Solve all tasks in `Lab13_Student.ipynb`. Each task has a **test cell**: when it prints  your solution is correct. For each task, **write pseudocode first**, then code, then test with your own inputs including edge cases. Difficulty:  Easy  Medium  Hard.

Task 1 Sum, Maximum and Leaf Values

The notebook provides `TreeNode` and `build_tree(level_order_list)`. Write recursive functions `tree_sum(root)`, `tree_max(root)` (return `None` for an empty tree) and `leaf_values(root)` (left-to-right list of leaf data).

```
build_tree([5, 3, 8, 1, 4, None, 9]) → sum 30, max 9, leaves [1, 4, 9]
```

Task 2 Mirror (Invert) a Binary Tree

Write `mirror(root)` that swaps the left and right child of **every** node and returns the root. Check your answer with level-order traversal.

```
[4, 2, 7, 1, 3, 6, 9] → level order [4, 7, 2, 9, 6, 3, 1]
```

Task 3 Same Tree and Symmetric Tree

Write `same_tree(a, b)` (same shape and same values) and `is_symmetric(root)` (the tree is a mirror image of itself around its centre). Hint: a tree is symmetric if its left subtree is the mirror of its right subtree; write a helper `is_mirror(x, y)`.

```
[1, 2, 2, 3, 4, 4, 3] → symmetric
[1, 2, 2, None, 3, None, 3] → not symmetric
```

Task 4 Level-by-Level Report

Write `levels(root)` that returns a list of lists: the values at each level, from top to bottom. Then write `right_view(root)`: the last value of each level (what you see when looking at the tree from the right side).

```
[3, 9, 20, None, None, 15, 7] → levels [[3], [9, 20], [15, 7]], right view [3, 20, 7]
```

Task 5 Iterative Inorder Traversal

Write `inorder_iter(root)` **without recursion**, using an explicit stack. Hint: go left as far as possible pushing nodes; pop a node, visit it, then move to its right child.

```
[1, None, 2, 3] → [1, 3, 2]
```

Task 6 Diameter of a Binary Tree

The **diameter** is the number of edges on the longest path between **any two nodes** (the path may or may not pass through the root). A naive solution computes height at every node ($O(n^2)$).

Write an **$O(n)$** solution with a helper that returns the height of a subtree while updating the best diameter seen so far.

```
[1, 2, 3, 4, 5] → 3    (path 4 → 2 → 1 → 3)
```

Task 7 Rebuild a Tree from Traversals

Given the **preorder** and **inorder** traversals of a binary tree with unique values, rebuild the tree and return its root.

Key idea: the first preorder value is the root; its position in inorder splits the left and right subtrees. Use a dictionary of inorder positions for $O(n)$.

```
preorder [3, 9, 20, 15, 7], inorder [9, 3, 15, 20, 7] → tree [3, 9, 20, None,
None, 15, 7]
```



11. LeetCode Practice

Solve these on **leetcode.com** and upload screenshots of the “Accepted” result to your GitHub **DSA-Labs/Lab13** folder.

#	Problem	Level	Concept practised
144	Binary Tree Preorder Traversal	Easy	DFS
94	Binary Tree Inorder Traversal	Easy	DFS / stack (Task 5)
145	Binary Tree Postorder Traversal	Easy	DFS
104	Maximum Depth of Binary Tree	Easy	Height
226	Invert Binary Tree	Easy	Mirror (Task 2)
101	Symmetric Tree	Easy	Mirror check (Task 3)
102	Binary Tree Level Order Traversal	Medium	BFS (Task 4)
543	Diameter of Binary Tree	Easy	Height + global best (Task 6)

#	Problem	Level	Concept practised
105	Construct Binary Tree from Preorder and Inorder Traversal	Medium	Divide & conquer (Task 7)



12. Home Task

1. Finish all tasks and push `Lab13_Student.ipynb` to GitHub.
2. Write `is_complete(root)` (use BFS: once you see a missing child, no later node may have children) and `is_balanced(root)` in $O(n)$.
3. Write iterative **postorder** traversal using two stacks.
4. **Mini-project: Folder Size Analyser.** Represent a folder hierarchy as a general tree (each node has a list of children and a file size). Use a postorder traversal to compute the total size of every folder, print the tree with indentation, and list the 3 largest folders. Then try it on a real folder with `os.scandir`.
5. Solve at least 3 LeetCode problems from the list.

? 13. Viva Questions

1. Why is a tree called a non-linear data structure?
2. Define root, leaf, sibling, depth and height.
3. How many edges does a tree with n nodes have?
4. Differentiate between full, complete and perfect binary trees.
5. What is the maximum number of nodes in a binary tree of height h ?
6. In the array representation, where are the children of index i ?
7. Write the preorder, inorder and postorder of a small tree you draw.
8. Which data structure does level-order traversal use? Which does iterative DFS use?
9. What is the time and space complexity of tree traversals?
10. What is an expression tree? Which traversal evaluates it?



14. Assessment Rubric

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Correctness	All tasks pass all tests	Most tasks correct	Some tasks correct	Mostly incorrect
Logic & pseudocode	Clear pseudocode and dry run for every task	Pseudocode for most tasks	Partial pseudocode	None
Code quality	Meaningful names, functions, comments	Mostly clean	Hard to read	Copied or unreadable
Complexity & edge cases	States Big-O and handles all edge cases	Some edge cases / Big-O	Only basic cases	Not considered
Viva	Explains every line confidently	Explains most parts	Basic understanding	Cannot explain



Lab Rules

- Bring your own laptop with Colab or Jupyter working.
- Arrive on time; late arrival may result in no attendance for the lab.
- Copying another student's code or pasting AI output without understanding it will be awarded zero. You must be able to explain every line in the viva.



15. References and Further Learning

- Binary tree animations: visualgo.net/en/bst
- scikit-learn Decision Trees: scikit-learn.org/stable/modules/tree.html
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, **Introduction to Algorithms** (CLRS), 4th ed., MIT Press, 2022.
- M. T. Goodrich, R. Tamassia, M. H. Goldwasser, **Data Structures and Algorithms in Python**, Wiley.
- MIT OpenCourseWare 6.006 Introduction to Algorithms; Harvard CS50x (free online courses).
- Visualise code step by step: pythontutor.com; animated data structures: visualgo.net.



About the Author

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

GitHub: github.com/RamzanPUCIT

LinkedIn: linkedin.com/in/ramzan-ai

ResearchGate: researchgate.net/profile/Muhammad-Ramzan-105

© 2026 Muhammad Ramzan. Prepared for the Department of Computer Science, University of Okara.

UNIVERSITY OF OKARA

Department of Computer Science

DATA STRUCTURES & ALGORITHMS

Lab Manual

LAB 14 | Binary Search Trees: Search, Insert, Delete and Balance

Week 14 of 16 • Duration: 3 hours (in-lab) + home practice

Prepared by

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

Applicable to: BSCS • BSIT • BSSE • BSAI • BSDS • BS Cyber Security and all programs offering DSA

Fall 2026 Edition

About This Lab Manual

This manual is part of a **16-week Data Structures & Algorithms (DSA) lab series**. It is designed so that a student starting with **zero programming knowledge** in Week 1 can confidently design, implement and analyse data structures by Week 16. Every lab builds on the previous one, so please do not skip labs.

Language policy: concepts are explained in a language-independent way (diagrams, pseudocode, dry runs). Full implementations are in **Python**, and every lab includes a **C++ Corner** so instructors teaching in C++ can use the same manual.

Every lab follows the same structure

Section	What you get
 Objectives & CLOs	What you will be able to do after this lab
 Motivation	Where this topic is used in real life, industry, research and the future (2026–2030)
 Concepts	Simple explanations with real examples, pseudocode and dry runs
 Code	Python implementation + C++ Corner
 Lab Tasks	Graded tasks:  Easy,  Medium,  Hard
 LeetCode	Real interview problems to practise
 Home Task &  Viva	Practice at home and prepare for oral questions
 Notebooks	Instructor notebook (complete code) and Student notebook (with TODO blanks and auto-check tests)

Lab 14 at a glance

Item	Details
Lab title	Binary Search Trees: Search, Insert, Delete and Balance
Week	14 of 16
Prerequisites	Lab 04 (Binary Search), Lab 13 (Binary Trees)
Tools	Google Colab / Jupyter / VS Code (Python 3); any C++ compiler for the C++ Corner
Files	Lab14_Instructor.ipynb , Lab14_Student.ipynb
Next lab	Lab 15: Heaps and Priority Queues

1. Learning Objectives

By the end of this lab, you will be able to:

1. State the **BST property** and check whether a tree satisfies it.
2. Implement **search, insert, minimum and maximum** recursively and iteratively.
3. Implement **deletion** for all three cases: leaf, one child, two children (inorder successor).
4. Explain why **inorder traversal of a BST gives sorted output**.
5. Analyse BST operations as **O(h)**: $O(\log n)$ when balanced, $O(n)$ when skewed.
6. Explain **balancing** with the AVL balance factor and rotations.
7. Solve range, k-th smallest, floor/ceiling and lowest-common-ancestor problems.

Course Learning Outcomes (CLO) mapping

CLO	Description	Taxonomy level
CLO-2	Implement binary search tree operations	Apply (C3)
CLO-3	Analyse BST performance and the need for balancing	Analyse (C4)

2. Motivation: Binary Search That Can Grow

Binary search on a sorted array is very fast ($O(\log n)$), but inserting a new element into a sorted array is slow ($O(n)$ shifting). A linked list inserts fast but searches slowly. A **Binary Search Tree (BST)** gives us the best of both: **search, insert and delete in $O(\log n)$** when the tree is balanced, while keeping data in sorted order.

Structure	Search	Insert	Delete	Sorted order?
Unsorted array	$O(n)$	$O(1)$	$O(n)$	No
Sorted array	$O(\log n)$	$O(n)$	$O(n)$	Yes
Linked list	$O(n)$	$O(1)$ at front	$O(n)$	No
Hash table	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	No
Balanced BST	$O(\log n)$	$O(\log n)$	$O(\log n)$	Yes

2.1 BSTs in Daily Life

- **Guess-the-number game:** “Is it bigger or smaller than 50?” Each question moves you left or right, just like searching a BST.
- **A dictionary’s guide words:** decide whether your word comes before or after, and narrow down.
- **Leaderboards and rankings** that change constantly (game scores, PSL standings) need fast insert + sorted order.

2.2 BSTs in Industry

- **Ordered maps and sets:** C++ `std::map/std::set` and Java `TreeMap/TreeSet` are balanced BSTs (Red-Black trees).
- **Databases:** indexes are B-trees and B+ trees, multi-way generalisations of BSTs optimised for disks.
- **Operating systems:** the Linux Completely Fair Scheduler uses a Red-Black tree to pick the next process.
- **Range queries:** “all orders between Rs. 1,000 and Rs. 5,000”, “all students with CGPA 3.0–3.5” are fast on ordered trees.

2.3 BSTs in Research

- **Spatial data / GIS:** k-d trees (a BST that alternates between x and y coordinates) find nearest points on a map.
- **Machine learning:** k-d trees and ball trees speed up k-nearest-neighbour search.
- **Self-adjusting and learned index structures** are active database research topics.

2.4 Why It Matters in 2026–2030



Ordered data is still king

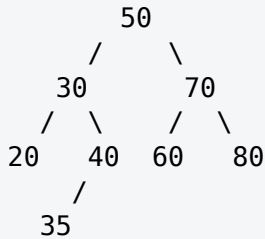
Hash tables are fast but forget order. Whenever you need **ordered** data with fast updates (time-series, leaderboards, ranges, nearest values), balanced trees are the answer. Understanding the BST is also the gateway to B-trees (databases), k-d trees (AI and GIS) and interval trees (scheduling).

3. Part A: The BST Property

A **Binary Search Tree** is a binary tree where, for **every** node:

- All keys in the **left subtree** are **smaller** than the node's key.
- All keys in the **right subtree** are **greater** than the node's key.
- Both subtrees are also BSTs. (In this lab, **no duplicate keys**.)

A valid BST



Inorder: 20 30 35 40 50 60 70 80 (sorted!)

! A common trap

It is **not enough** that each node is bigger than its left child and smaller than its right child. Suppose 55 is placed as the right child of 40 in the tree above. Locally $40 < 55$ looks fine, but 55 is in the **left subtree of 50** while being greater than 50, so the tree is **not** a BST. Every key must respect **all** of its ancestors.

4. Part B: Search, Insert, Min and Max

4.1 Search

Compare the key with the current node: equal → found; smaller → go left; greater → go right; reach **None** → not present. At each step we discard one subtree, like binary search.

```

class Node:
    def __init__(self, key, left=None, right=None):
        self.key = key
        self.left = left
        self.right = right

def insert(root, key):
    """Insert key into the BST and return the (possibly new) root."""
    if root is None:
        return Node(key)
    if key < root.key:
        root.left = insert(root.left, key)
    elif key > root.key:
        root.right = insert(root.right, key)
    return root
    # duplicates are ignored

def build_bst(keys):
    root = None
    for k in keys:
        root = insert(root, k)
    return root

def inorder(root):
    return inorder(root.left) + [root.key] + inorder(root.right) if root else []

def search(root, key):
    # recursive
    if root is None or root.key == key:
        return root
    return search(root.left, key) if key < root.key else search(root.right, key)

def search_iter(root, key):
    # iterative: O(1) extra space
    while root and root.key != key:
        root = root.left if key < root.key else root.right
    return root

root = build_bst([50, 30, 70, 20, 40, 60, 80, 35])
print("inorder:", inorder(root))
print("search 35:", search(root, 35) is not None, "| search 65:", search_iter(root, 65) is
not None)

```

4.2 Insert

Search for the key; the place where the search falls off the tree (**None**) is exactly where the new node belongs. New keys are always inserted as **leaves**.

Insert	Path	Placed as
50	-	root
30	$30 < 50 \rightarrow \text{left}$	left of 50
70	$70 > 50 \rightarrow \text{right}$	right of 50
40	$40 < 50 \rightarrow \text{left}, 40 > 30 \rightarrow \text{right}$	right of 30
35	$35 < 50, > 30, < 40$	left of 40

4.3 Minimum and Maximum

```
def find_min(root):                                # keep going left
    while root.left:
        root = root.left
    return root.key

def find_max(root):                                # keep going right
    while root.right:
        root = root.right
    return root.key

print("min:", find_min(root), "| max:", find_max(root))
```

5. Part C: Deletion (Three Cases)

Case	What to do	Example (tree above)
1. Leaf node	Simply remove it	Delete 20
2. One child	Replace the node with its only child	Delete 40 → 35 takes its place
3. Two children	Replace the key with its inorder successor (smallest key in the right subtree), then delete the successor from the right subtree	Delete 50 → 60 becomes the root

Case 3 step by step

Delete 50 (two children):



successor = smallest in right subtree = 60

```

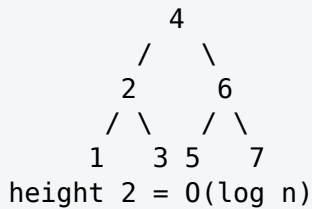
def delete(root, key):
    if root is None:
        return None
    if key < root.key:
        root.left = delete(root.left, key)
    elif key > root.key:
        root.right = delete(root.right, key)
    else:
        # found the node to delete
        # case 1 or 2
        if root.left is None:
            return root.right
        if root.right is None:
            return root.left
        # case 2
        succ = root.right
        # case 3: inorder successor
        while succ.left:
            succ = succ.left
        root.key = succ.key
        root.right = delete(root.right, succ.key)
    return root

r = build_bst([50, 30, 70, 20, 40, 60, 80, 35])
for k in [20, 40, 50]:
    r = delete(r, k)
    print(f"after deleting {k}: inorder = {inorder(r)}, root = {r.key}")
    
```

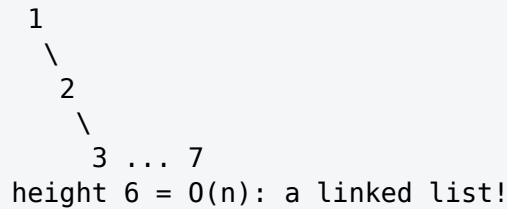
6. Part D: Balanced vs Skewed Trees

All BST operations walk one path from the root, so they cost **$O(h)$** where h is the height. The height depends on the **order of insertion**:

Insert 4, 2, 6, 1, 3, 5, 7



Insert 1, 2, 3, 4, 5, 6, 7 (sorted!)



```

import random, sys
sys.setrecursionlimit(10_000)

def height(node):
    return -1 if node is None else 1 + max(height(node.left),
    height(node.right))

n = 1000
keys = list(range(n))
print("sorted insert height  :", height(build_bst(keys)))
random.shuffle(keys)
print("random insert height  :", height(build_bst(keys)))
import math
print("best possible (log2 n):", math.floor(math.log2(n)))
  
```

6.1 Self-Balancing Trees: AVL Rotations

An **AVL tree** keeps the **balance factor** = $\text{height}(\text{left}) - \text{height}(\text{right})$ in $\{-1, 0, +1\}$ at every node. After an insert or delete, if a node becomes unbalanced, a **rotation** fixes it in $O(1)$. This guarantees height $O(\log n)$.

Right rotation

Right rotation at z (left-left case):



```

def rotate_right(z):
    y = z.left
    z.left = y.right      # T3 moves under z
    y.right = z
    return y              # y is the new root of this subtree

def rotate_left(z):
    y = z.right
    z.right = y.left
    y.left = z
    return y

skewed = build_bst([30, 20, 10])      # a left-left chain
balanced = rotate_right(skewed)
print("root:", balanced.key, "| left:", balanced.left.key, "| right:",
      balanced.right.key)

```

Imbalance case	Fix
Left-Left	Right rotation
Right-Right	Left rotation
Left-Right	Left rotation on child, then right rotation
Right-Left	Right rotation on child, then left rotation

Other balanced trees: **Red-Black trees** (used in C++ `map`, Java `TreeMap`, Linux), **B-trees** (databases), **Splay trees** and **Treaps**.

7. C++ Corner

C++

```
#include <iostream>
#include <map>
#include <set>
using namespace std;

struct Node { int key; Node *left = nullptr, *right = nullptr; Node(int k) :
key(k) {} };

Node* insert(Node* root, int key) {
    if (!root) return new Node(key);
    if (key < root->key) root->left = insert(root->left, key);
    else if (key > root->key) root->right = insert(root->right, key);
    return root;
}

bool search(Node* root, int key) {
    while (root && root->key != key) root = key < root->key ? root->left : root->right;
    return root != nullptr;
}

int main() {
    Node* root = nullptr;
    for (int k : {50, 30, 70, 20, 40, 60, 80}) root = insert(root, k);
    cout << search(root, 60) << search(root, 65) << endl;    // 10

    set<int> s = {50, 30, 70, 20};    // balanced BST (Red-Black tree)
    auto it = s.lower_bound(35);    // smallest key >= 35 -> 50
    cout << *it << endl;
    map<string, double> cgpa = {"Sara", 3.9}, {"Ali", 3.4}};
    for (auto& [name, g] : cgpa) cout << name << " " << g << endl;    // sorted by
name
}
```

8. Common Mistakes

Mistake	Effect	Fix
Checking only parent–child order when validating	Accepts invalid BSTs	Pass (low, high) bounds down the recursion
Forgetting <code>root.left = insert(...)</code>	New node is created but never attached	Always assign the returned subtree
Deleting a two-child node by removing it	Tree breaks apart	Copy the successor's key, delete the successor
Inserting sorted data into a plain BST	Height n , $O(n)$ operations	Shuffle, or use a balanced tree

Mistake	Effect	Fix
Allowing duplicates without a rule	Search/delete become inconsistent	Ignore duplicates or store a count

9. Lab Tasks

Solve all tasks in `Lab14_Student.ipynb`. Each task has a **test cell**: when it prints  your solution is correct. For each task, **write pseudocode first**, then code, then test with your own inputs including edge cases. Difficulty:  Easy  Medium  Hard.

Task 1 Validate a BST

Write `is_bst(root)` that checks the **full** BST property. Pass down the allowed range (`low`, `high`) for each subtree.

The notebook provides `Node`, `insert`, `build_bst` and `inorder`.

```
Tree 50 → (30 → right 55), 70 → False (55 is in the left subtree of 50)
```

Task 2 k-th Smallest Mark

Marks are stored in a BST. Write `kth_smallest(root, k)` using an inorder traversal that **stops early** once the k-th element is reached (do not build the whole list).

```
BST of [50, 30, 70, 20, 40, 60, 80], k = 3 → 40
```

Task 3 Range Query: Students Between Two CGPAs

Keys are CGPA × 100 (integers). Write `range_keys(root, low, high)` that returns all keys with $low \leq key \leq high$ in sorted order. **Prune**: do not visit the left subtree if `root.key < low`, nor the right subtree if `root.key > high`.

```
keys [310, 250, 365, 290, 330, 390, 280], range [280, 340] → [280, 290, 310, 330]
```

Task 4 Floor and Ceiling

`floor(root, x)` = largest key ≤ x; `ceiling(root, x)` = smallest key ≥ x (return `None` if none). Solve both in $O(h)$ **without** traversing the whole tree.

Use: a bus timetable stored as minutes after midnight; find the last bus before you arrive and the next bus after you arrive.

```
times [480, 540, 600, 720, 840], x = 610 → floor 600, ceiling 720
```

Task 5 🟡 Lowest Common Ancestor in a BST

The **lowest common ancestor (LCA)** of two keys is the deepest node that has both keys in its subtree (a node can be its own ancestor). In a BST, use the ordering: if both keys are smaller go left, if both are bigger go right, otherwise the current node is the answer. $O(h)$.

```
BST [6, 2, 8, 0, 4, 7, 9, 3, 5]: lca(2, 8) → 6, lca(2, 4) → 2, lca(3, 5) → 4
```

Task 6 🔴 Sorted Array to Balanced BST

Given sorted roll numbers, build a **height-balanced** BST: make the middle element the root, then recursively build the left half and right half. Return the root.

Verify that the height is $\lfloor \log_2 n \rfloor$ for $n = 1,023$ and that inorder gives back the array.

```
[1, 2, 3, 4, 5, 6, 7] → root 4, left subtree root 2, right subtree root 6
```

Task 7 🔴 A Complete BST Map Class

Build a class **BSTMap** storing **key** → **value** pairs with methods **put(key, value)** (insert or update), **get(key)** (or **None**), **remove(key)**, **__len__**, **items()** (sorted list of pairs) and **height()**.

Use it as a sorted phone book. Internally, store **key** and **value** in each node and reuse the deletion logic from Part C.

```
put("Sara", "0300"); put("Ali", "0321"); items() → [("Ali", "0321"), ("Sara", "0300")]
```



10. LeetCode Practice

Solve these on **leetcode.com** and upload screenshots of the “Accepted” result to your GitHub **DSA-Labs/Lab14** folder.

#	Problem	Level	Concept practised
700	Search in a Binary Search Tree	Easy	BST search
701	Insert into a Binary Search Tree	Medium	BST insert
450	Delete Node in a BST	Medium	Three deletion cases
98	Validate Binary Search Tree	Medium	Bounds (Task 1)
230	Kth Smallest Element in a BST	Medium	Inorder (Task 2)

#	Problem	Level	Concept practised
938	Range Sum of BST	Easy	Pruning (Task 3)
235	Lowest Common Ancestor of a BST	Medium	Ordering (Task 5)
108	Convert Sorted Array to Binary Search Tree	Easy	Divide & conquer (Task 6)



11. Home Task

1. Finish all tasks and push `Lab14_Student.ipynb` to GitHub.
2. Implement `inorder_successor(root, key)` and `inorder_predecessor(root, key)` in $O(h)$.
3. Implement a full **AVL tree insert** that stores heights in nodes and applies the four rotation cases. Insert 1 to 1,000 in sorted order and confirm that the height stays near $\log_2 n$.
4. **Mini-project: Live Leaderboard.** Players get scores over time. Store (score, name) in your `BSTMap` and support: add/update score, top-10 players (reverse inorder), rank of a player, and players with scores in a range.
5. Solve at least 3 LeetCode problems from the list.

? 12. Viva Questions

1. State the BST property.
2. Why does inorder traversal of a BST give sorted output?
3. Explain how to search for a key in a BST. What is its complexity?
4. Where is a new key always inserted in a BST?
5. Explain the three cases of deletion.
6. What is an inorder successor? Why is it used in deletion?
7. When does a BST become a linked list? What is the complexity then?
8. What is the balance factor in an AVL tree?
9. Name two real systems that use balanced BSTs.
10. Compare a BST with a hash table: when would you choose each?



13. Assessment Rubric

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Correctness	All tasks pass all tests	Most tasks correct	Some tasks correct	Mostly incorrect
Logic & pseudocode	Clear pseudocode and dry run for every task	Pseudocode for most tasks	Partial pseudocode	None
Code quality	Meaningful names, functions, comments	Mostly clean	Hard to read	Copied or unreadable
Complexity & edge cases	States Big-O and handles all edge cases	Some edge cases / Big-O	Only basic cases	Not considered
Viva	Explains every line confidently	Explains most parts	Basic understanding	Cannot explain



Lab Rules

- Bring your own laptop with Colab or Jupyter working.
- Arrive on time; late arrival may result in no attendance for the lab.
- Copying another student's code or pasting AI output without understanding it will be awarded zero. You must be able to explain every line in the viva.



14. References and Further Learning

- BST and AVL animations: visualgo.net/en/bst
- C++ `std::map` / Java `TreeMap` documentation (Red-Black trees).
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, **Introduction to Algorithms** (CLRS), 4th ed., MIT Press, 2022.
- M. T. Goodrich, R. Tamassia, M. H. Goldwasser, **Data Structures and Algorithms in Python**, Wiley.
- MIT OpenCourseWare 6.006 Introduction to Algorithms; Harvard CS50x (free online courses).
- Visualise code step by step: pythontutor.com; animated data structures: visualgo.net.



About the Author

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

GitHub: github.com/RamzanPUCIT

LinkedIn: linkedin.com/in/ramzan-ai

ResearchGate: researchgate.net/profile/Muhammad-Ramzan-105

© 2026 Muhammad Ramzan. Prepared for the Department of Computer Science, University of Okara.

UNIVERSITY OF OKARA

Department of Computer Science

DATA STRUCTURES & ALGORITHMS

Lab Manual

LAB 15 | Heaps, Priority Queues and Heap Sort

Week 15 of 16 • Duration: 3 hours (in-lab) + home practice

Prepared by

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

Applicable to: BSCS • BSIT • BSSE • BSAI • BSDS • BS Cyber Security and all programs offering DSA

Fall 2026 Edition

About This Lab Manual

This manual is part of a **16-week Data Structures & Algorithms (DSA) lab series**. It is designed so that a student starting with **zero programming knowledge** in Week 1 can confidently design, implement and analyse data structures by Week 16. Every lab builds on the previous one, so please do not skip labs.

Language policy: concepts are explained in a language-independent way (diagrams, pseudocode, dry runs). Full implementations are in **Python**, and every lab includes a **C++ Corner** so instructors teaching in C++ can use the same manual.

Every lab follows the same structure

Section	What you get
 Objectives & CLOs	What you will be able to do after this lab
 Motivation	Where this topic is used in real life, industry, research and the future (2026–2030)
 Concepts	Simple explanations with real examples, pseudocode and dry runs
 Code	Python implementation + C++ Corner
 Lab Tasks	Graded tasks:  Easy,  Medium,  Hard
 LeetCode	Real interview problems to practise
 Home Task &  Viva	Practice at home and prepare for oral questions
 Notebooks	Instructor notebook (complete code) and Student notebook (with TODO blanks and auto-check tests)

Lab 15 at a glance

Item	Details
Lab title	Heaps, Priority Queues and Heap Sort
Week	15 of 16
Prerequisites	Lab 11 (Queues), Lab 13 (Binary Trees, array representation)
Tools	Google Colab / Jupyter / VS Code (Python 3); any C++ compiler for the C++ Corner
Files	Lab15_Instructor.ipynb , Lab15_Student.ipynb
Next lab	Lab 16: Graphs, BFS, DFS and Shortest Paths

1. Learning Objectives

By the end of this lab, you will be able to:

1. Explain the **heap property** for **max-heaps** and **min-heaps** and why a heap is a **complete binary tree**.
2. Store a heap in an **array** and navigate it with index formulas.
3. Implement **insert (sift-up)**, **extract (sift-down)** and **peek** in $O(\log n)$ and $O(1)$.
4. Build a heap from an array in **$O(n)$** with bottom-up heapify.
5. Implement **Heap Sort** in place and analyse it.
6. Use Python's **heapq** module, including max-heap tricks and **nlargest**.
7. Solve top-k, k-way merge and running-median problems with heaps.

Course Learning Outcomes (CLO) mapping

CLO	Description	Taxonomy level
CLO-2	Implement heaps and heap-based algorithms	Apply (C3)
CLO-3	Analyse heap operations and choose heaps for priority problems	Analyse (C4)

2. Motivation: Always Serve the Most Important First

In Lab 11 you met the **priority queue**: the element with the highest priority leaves first. How do we build one efficiently? A sorted list makes insertion $O(n)$; an unsorted list makes removal $O(n)$. A **heap** does **both in $O(\log n)$** and shows the top priority in **$O(1)$** .

Implementation	insert	get max/min	remove max/min
Unsorted list	$O(1)$	$O(n)$	$O(n)$
Sorted list	$O(n)$	$O(1)$	$O(1)$
Balanced BST	$O(\log n)$	$O(\log n)$	$O(\log n)$
Binary heap	$O(\log n)$	$O(1)$	$O(\log n)$

2.1 Heaps in Daily Life

Situation	Priority
Hospital emergency (triage)	Most serious patient first

Situation	Priority
Airport boarding	Business class and special assistance first
Your to-do list before exams	Nearest deadline first
Ride-hailing (Careem / inDrive)	Nearest available driver first
University merit list (top 50 admissions)	Highest aggregate first

2.2 Heaps in Industry

- **Operating systems** schedule processes and timers with priority queues.
- **Shortest-path algorithms** (Dijkstra, A*) in Google Maps and games pick the closest unexplored location from a heap (Lab 16).
- **Top-K queries:** trending hashtags, top-selling products, most active users computed over huge streams with small heaps.
- **Event-driven simulation** and **job schedulers** (cron systems, Kubernetes) process events in time order with heaps.
- **Data compression:** Huffman coding (ZIP, JPEG, MP3) repeatedly combines the two least frequent symbols using a min-heap.

2.3 Heaps in Research

- **Search algorithms in AI:** best-first search, A* and beam search (used when language models generate text) keep candidates in priority queues.
- **Nearest-neighbour search** for vector databases keeps the current best k results in a heap.
- **Streaming algorithms** compute medians and percentiles on unbounded data with heaps.

2.4 Why It Matters in 2026–2030

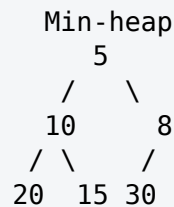
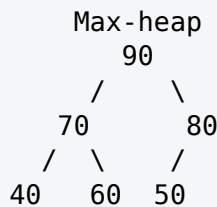
Prioritisation at scale

As systems handle millions of requests and AI models generate many candidate answers, deciding **what to process first** becomes critical. Heaps are the simple, fast answer to “give me the best k items” and “what is next?”. They also give you an $O(n \log n)$ sorting algorithm that needs no extra memory.

3. Part A: What Is a Heap?

A **binary heap** is a binary tree with two properties:

1. **Shape property:** it is a **complete binary tree** (all levels full except possibly the last, which is filled from the left).
2. **Heap property:** in a **max-heap**, every parent $\geq$ its children, so the **largest** key is at the root. In a **min-heap**, every parent $\leq$ its children, so the **smallest** key is at the root.



array: [90, 70, 80, 40, 60, 50]

array: [5, 10, 8, 20, 15, 30]

! A heap is not sorted

Only parent–child order is guaranteed. Siblings and cousins can be in any order. 80 is in level 1 and 60 in level 2, but nothing says $70 > 80$. This partial order is what makes heaps fast to maintain.

3.1 Array Representation

Because a heap is complete, we store it in an array **without gaps** (as in Lab 13). For index i : **left** = $2i + 1$, **right** = $2i + 2$, **parent** = $(i - 1) // 2$. The height of a heap with n nodes is $\lfloor \log_2 n \rfloor$, which is why its operations are $O(\log n)$.

⚙️ 4. Part B: Heap Operations (Min-Heap)

4.1 Insert: Add at the End, then Sift Up

Put the new key at the end of the array (keeps the shape). Then, while it is smaller than its parent, **swap it with the parent** (“bubble up”).

Sift-up

Insert 3 into [5, 10, 8, 20, 15, 30]

[5, 10, 8, 20, 15, 30, 3] 3 at index 6, parent index 2 (8) -> swap

[5, 10, 3, 20, 15, 30, 8] 3 at index 2, parent index 0 (5) -> swap

[3, 10, 5, 20, 15, 30, 8] 3 is the new root

4.2 Extract Min: Move the Last to the Root, then Sift Down

The minimum is at index 0. Replace the root with the **last element**, remove the last slot, and then, while the new root is larger than its **smaller child**, swap them (“bubble down”).

```
class MinHeap:
    def __init__(self):
        self.a = []

    def __len__(self):
        return len(self.a)

    def peek(self):
        return self.a[0] if self.a else None           # 0(1)

    def push(self, x):
        self.a.append(x)
        self._sift_up(len(self.a) - 1)                 # 0(log n)

    def pop(self):
        if not self.a:
            raise IndexError("pop from empty heap")
        top = self.a[0]
        last = self.a.pop()
        if self.a:
            self.a[0] = last
            self._sift_down(0)
        return top

    def _sift_up(self, i):
        while i > 0:
            parent = (i - 1) // 2
            if self.a[i] < self.a[parent]:
                self.a[i], self.a[parent] = self.a[parent], self.a[i]
                i = parent
            else:
                break

    def _sift_down(self, i):
        n = len(self.a)
        while True:
            smallest, l, r = i, 2 * i + 1, 2 * i + 2
            if l < n and self.a[l] < self.a[smallest]:
                smallest = l
            if r < n and self.a[r] < self.a[smallest]:
                smallest = r
            if smallest == i:
                break
            self.a[i], self.a[smallest] = self.a[smallest], self.a[i]
            i = smallest

h = MinHeap()
```

```
for x in [20, 5, 15, 30, 8, 10]:  
    h.push(x)  
    print(f"push {x:>2}: {h.a}")  
print("pop order:", [h.pop() for _ in range(len(h))])
```

Operation	Time	Why
peek	$O(1)$	Root of the heap
push	$O(\log n)$	Sift-up along one path
pop	$O(\log n)$	Sift-down along one path
build (heapify)	$O(n)$	Most nodes are near the bottom
search for arbitrary key	$O(n)$	Heaps are not ordered for searching

5. Part C: Building a Heap in $O(n)$

Inserting n items one by one costs $O(n \log n)$. **Bottom-up heapify** is faster: leaves are already heaps, so call sift-down on every non-leaf node from index $n/2 - 1$ down to 0. Most nodes are near the bottom and sift down only a little, giving a total of $O(n)$.

```
def heapify(arr):                                # turn arr into a MIN-heap in place
    h = MinHeap()
    h.a = arr
    for i in range(len(arr) // 2 - 1, -1, -1):
        h._sift_down(i)
    return arr

data = [9, 4, 7, 1, 8, 2, 6, 3]
print(heapify(data))
```

6. Part D: Heap Sort

Heap sort uses a **max-heap**: build it in $O(n)$, then repeatedly **swap the root (maximum) with the last element of the heap**, shrink the heap by one, and sift down. The array becomes sorted in ascending order **in place**.

```
def sift_down_max(a, i, n):
    while True:
        largest, l, r = i, 2 * i + 1, 2 * i + 2
        if l < n and a[l] > a[largest]:
            largest = l
        if r < n and a[r] > a[largest]:
            largest = r
        if largest == i:
            return
        a[i], a[largest] = a[largest], a[i]
        i = largest

def heap_sort(a):
    n = len(a)
    for i in range(n // 2 - 1, -1, -1):          # 1. build a max-heap:  $O(n)$ 
        sift_down_max(a, i, n)
    for end in range(n - 1, 0, -1):              # 2. move max to the end:  $n \times O(\log n)$ 
        a[0], a[end] = a[end], a[0]
        sift_down_max(a, 0, end)
    return a

print(heap_sort([12, 3, 17, 8, 34, 25, 1]))
```

Property	Heap sort
Time (all cases)	$O(n \log n)$
Extra space	$O(1)$
Stable?	No
Used in	Introsort's fallback (C++ <code>std::sort</code>), embedded systems



7. Part E: Python's `heapq` Module

`heapq` provides a **min-heap** on a plain list. For a **max-heap**, push negated values or tuples with negated priority.

```
import heapq

marks = [67, 89, 45, 92, 78, 55, 81]
heap = marks[:]
heapq.heapify(heap)                # O(n)
heapq.heappush(heap, 50)
print("smallest:", heap[0], "| popped:", heapq.heappop(heap))
print("top 3:", heapq.nlargest(3, marks), "| bottom 2:",
      heapq.nsmallest(2, marks))

max_heap = []
for m in marks:
    heapq.heappush(max_heap, -m)    # negate for max-heap
print("max:", -heapq.heappop(max_heap))

tasks = []
heapq.heappush(tasks, (2, "Prepare viva"))
heapq.heappush(tasks, (1, "Submit DSA lab"))
heapq.heappush(tasks, (3, "Watch PSL match"))
while tasks:
    print(heapq.heappop(tasks))
```



8. Part F: Classic Heap Patterns

8.1 Top-K from a Stream: Keep a Min-Heap of Size K

To keep the k largest values seen so far, keep a **min-heap of size k** . Its root is the smallest of the top k ; a new value enters only if it beats that root. Time $O(n \log k)$, memory $O(k)$: perfect for huge streams.

```
def top_k(stream, k):
    heap = []
    for x in stream:
        if len(heap) < k:
            heapq.heappush(heap, x)
        elif x > heap[0]:
            heapq.heapreplace(heap, x)      # pop smallest, push x
    return sorted(heap, reverse=True)

import random
scores = (random.randint(0, 10_000) for _ in range(100_000))  # a generator:
a "stream"
print(top_k(scores, 5))
```

8.2 Running Median with Two Heaps

Keep the smaller half of the numbers in a **max-heap** and the larger half in a **min-heap**, balanced in size. The median is at the top of one or both heaps. Each new number costs $O(\log n)$.

```
class RunningMedian:
    def __init__(self):
        self.low = []          # max-heap (store negatives)
        self.high = []         # min-heap

    def add(self, x):
        heapq.heappush(self.low, -x)
        heapq.heappush(self.high, -heapq.heappop(self.low))  # move the
largest of low
        if len(self.high) > len(self.low):
            heapq.heappush(self.low, -heapq.heappop(self.high))

    def median(self):
        if len(self.low) > len(self.high):
            return -self.low[0]
        return (-self.low[0] + self.high[0]) / 2

rm = RunningMedian()
for bpm in [72, 90, 65, 80, 100, 70]:
    rm.add(bpm)
    print(f"added {bpm:>3} -> median {rm.median()}")
```

9. C++ Corner

C++

```

#include <iostream>
#include <queue>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    priority_queue<int> maxHeap; // MAX-heap
    // by default
    priority_queue<int, vector<int>, greater<int>> minHeap; // MIN-heap
    for (int x : {20, 5, 15, 30, 8}) { maxHeap.push(x); minHeap.push(x); }
    cout << maxHeap.top() << " " << minHeap.top() << endl; // 30 5

    vector<int> v = {12, 3, 17, 8, 34, 25, 1};
    make_heap(v.begin(), v.end()); // O(n) build, max-heap
    sort_heap(v.begin(), v.end()); // heap sort
    for (int x : v) cout << x << " ";
    cout << endl;

    // Patients: (severity, name); highest severity first
    priority_queue<pair<int, string>> er;
    er.push({2, "Ali"}); er.push({5, "Sara"}); er.push({3, "Usman"});
    while (!er.empty()) { cout << er.top().second << " "; er.pop(); }
}

```

Python vs C++ defaults

Python **heapq** is a **min-heap**; C++ **priority_queue** is a **max-heap**. Mixing them up is a very common bug when translating code.

10. Common Mistakes

Mistake	Effect	Fix
Thinking a heap is sorted	Wrong answers from heap[1], heap[2]...	Only heap[0] is guaranteed
Forgetting to sift after changing an element	Heap property broken	Sift up or down after every change
Sift-down comparing with only the left child	Wrong child swapped	Pick the smaller (or larger) of both children
Using a max-heap for top-K largest	$O(n \log n)$ and $O(n)$ memory	Use a min-heap of size k

Mistake	Effect	Fix
Pushing tuples with uncomparable items	TypeError when priorities tie	Add a counter: (priority, count, item)

11. Lab Tasks

Solve all tasks in `Lab15_Student.ipynb`. Each task has a **test cell**: when it prints  your solution is correct. For each task, **write pseudocode first**, then code, then test with your own inputs including edge cases. Difficulty:  Easy  Medium  Hard.

Task 1 Is It a Heap?

Write `is_min_heap(arr)` that checks whether an array satisfies the min-heap property (every parent $\leq$ its children). It must be $O(n)$.

```
[1, 3, 2, 7, 4] → True, [1, 3, 2, 0] → False
```

Task 2 Last Stone Weight

You have stones with given weights. Each turn, take the **two heaviest** stones and smash them: if equal, both disappear; otherwise the lighter is destroyed and the heavier loses that weight. Return the weight of the last stone (0 if none). Use `heapq` as a max-heap.

```
[2, 7, 4, 1, 8, 1] → 1
```

Task 3 Max-Heap Class with Update

Write `MaxHeap` (your own, not `heapq`) with `push`, `pop`, `peek` and `__len__`. Then add `increase_key(i, new_value)`, which raises the value at index `i` and restores the heap (sift up).

Priority updates are needed in schedulers and in Dijkstra's algorithm.

Task 4 Top-K Frequent Hashtags

Given a list of hashtags, return the `k` most frequent ones (ties broken alphabetically). Count with a dictionary, then use a heap (`heapq.nsmallest` with key `(-count, tag)` or your own size-`k` heap). Do not sort the whole dictionary.

```
["#PSL", "#DSA", "#PSL", "#0kara", "#DSA", "#PSL"], k = 2 → ["#PSL", "#DSA"]
```

Task 5 Merge K Sorted Result Lists

Each of `k` sections has a merit list sorted in ascending order. Merge them into one sorted list in $O(N \log k)$ (N = total items) using a min-heap holding the current front of each list as `(value, list_index, element_index)`.

This is exactly how databases merge sorted runs during external sorting.

```
[[1, 4, 7], [2, 5, 8], [0, 9]] → [0, 1, 2, 4, 5, 7, 8, 9]
```

Task 6 ● K Closest Petrol Pumps

Petrol pumps are at coordinates (x, y) relative to your car at $(0, 0)$. Return the k closest pumps (by Euclidean distance; ties by coordinates), using a **max-heap of size k** so that the solution is $O(n \log k)$.

Hint: compare squared distances (no square roots needed); push $(-dist^2, -x, -y)$ into `heapq`.

```
[(1, 3), (-2, 2), (5, 8), (0, 1)], k = 2 → [(0, 1), (-2, 2)]
```

Task 7 ● CPU Task Scheduler Simulation

Tasks arrive as $(arrival_time, duration, name)$. A single CPU always runs, among the tasks that have **already arrived**, the one with the **shortest duration** (ties: earlier arrival, then name). Tasks run to completion. Return the order of task names.

Sort tasks by arrival; keep a min-heap of available tasks keyed by $(duration, arrival, name)$; when the CPU is idle and nothing is available, jump time to the next arrival.

```
[(0, 8, "A"), (1, 4, "B"), (2, 9, "C"), (3, 5, "D")] → ["A", "B", "D", "C"]
```



12. LeetCode Practice

Solve these on **leetcode.com** and upload screenshots of the “Accepted” result to your GitHub `DSA-Labs/Lab15` folder.

#	Problem	Level	Concept practised
1046	Last Stone Weight	Easy	Max-heap (Task 2)
703	Kth Largest Element in a Stream	Easy	Min-heap of size k
215	Kth Largest Element in an Array	Medium	Heap vs quickselect
347	Top K Frequent Elements	Medium	Counting + heap (Task 4)
973	K Closest Points to Origin	Medium	Max-heap of size k (Task 6)
23	Merge k Sorted Lists	Hard	K-way merge (Task 5)

#	Problem	Level	Concept practised
295	Find Median from Data Stream	Hard	Two heaps
1834	Single-Threaded CPU	Medium	Scheduling (Task 7)



13. Home Task

1. Finish all tasks and push `Lab15_Student.ipynb` to GitHub.
2. Prove (or explain with a table) why bottom-up heapify is $O(n)$: count how many nodes are at each height and how far each can sift down.
3. Implement a **d-ary heap** (each node has d children) and compare the number of comparisons for $d = 2, 3, 4$ on 10,000 pushes and pops.
4. **Mini-project: Hospital Triage System.** Patients arrive with severity 1–10 and arrival time. Use a heap so the most severe is treated first and equal severities are treated in arrival order. Support “condition worsened” (increase severity) by marking old entries invalid (lazy deletion). Report average waiting time per severity.
5. Solve at least 3 LeetCode problems from the list.



14. Viva Questions

1. What are the two properties of a binary heap?
2. What is the difference between a max-heap and a min-heap?
3. Why can a heap be stored in an array without gaps?
4. Give the index formulas for parent and children.
5. Explain sift-up during insertion.
6. Explain extract-max/min and sift-down.
7. Why is building a heap bottom-up $O(n)$ instead of $O(n \log n)$?
8. Explain heap sort. Is it stable? Is it in-place?
9. How do you make a max-heap with Python's `heapq`?
10. How would you find the top 10 scores in a stream of one billion scores?



15. Assessment Rubric

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Correctness	All tasks pass all tests	Most tasks correct	Some tasks correct	Mostly incorrect
Logic & pseudocode	Clear pseudocode and dry run for every task	Pseudocode for most tasks	Partial pseudocode	None
Code quality	Meaningful names, functions, comments	Mostly clean	Hard to read	Copied or unreadable
Complexity & edge cases	States Big-O and handles all edge cases	Some edge cases / Big-O	Only basic cases	Not considered
Viva	Explains every line confidently	Explains most parts	Basic understanding	Cannot explain



Lab Rules

- Bring your own laptop with Colab or Jupyter working.
- Arrive on time; late arrival may result in no attendance for the lab.
- Copying another student's code or pasting AI output without understanding it will be awarded zero. You must be able to explain every line in the viva.



16. References and Further Learning

- Python `heapq` documentation: docs.python.org/3/library/heapq.html
- Heap animations: visualgo.net/en/heap
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, **Introduction to Algorithms** (CLRS), 4th ed., MIT Press, 2022.
- M. T. Goodrich, R. Tamassia, M. H. Goldwasser, **Data Structures and Algorithms in Python**, Wiley.
- MIT OpenCourseWare 6.006 Introduction to Algorithms; Harvard CS50x (free online courses).
- Visualise code step by step: pythontutor.com; animated data structures: visualgo.net.



About the Author

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

GitHub: github.com/RamzanPUCIT

LinkedIn: linkedin.com/in/ramzan-ai

ResearchGate: researchgate.net/profile/Muhammad-Ramzan-105

© 2026 Muhammad Ramzan. Prepared for the Department of Computer Science, University of Okara.

UNIVERSITY OF OKARA

Department of Computer Science

DATA STRUCTURES & ALGORITHMS

Lab Manual

LAB 16 | Graphs: Representation, BFS, DFS, Topological Sort and Shortest Paths

Week 16 of 16 • Duration: 3 hours (in-lab) + home practice

Prepared by

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

Applicable to: BSCS • BSIT • BSSE • BSAI • BSDS • BS Cyber Security and all programs offering DSA

Fall 2026 Edition

About This Lab Manual

This manual is part of a **16-week Data Structures & Algorithms (DSA) lab series**. It is designed so that a student starting with **zero programming knowledge** in Week 1 can confidently design, implement and analyse data structures by Week 16. Every lab builds on the previous one, so please do not skip labs.

Language policy: concepts are explained in a language-independent way (diagrams, pseudocode, dry runs). Full implementations are in **Python**, and every lab includes a **C++ Corner** so instructors teaching in C++ can use the same manual.

Every lab follows the same structure

Section	What you get
 Objectives & CLOs	What you will be able to do after this lab
 Motivation	Where this topic is used in real life, industry, research and the future (2026–2030)
 Concepts	Simple explanations with real examples, pseudocode and dry runs
 Code	Python implementation + C++ Corner
 Lab Tasks	Graded tasks:  Easy,  Medium,  Hard
 LeetCode	Real interview problems to practise
 Home Task &  Viva	Practice at home and prepare for oral questions
 Notebooks	Instructor notebook (complete code) and Student notebook (with TODO blanks and auto-check tests)

Lab 16 at a glance

Item	Details
Lab title	Graphs: Representation, BFS, DFS, Topological Sort and Shortest Paths
Week	16 of 16
Prerequisites	Lab 10 (Stacks), Lab 11 (Queues), Lab 12 (Hashing), Lab 15 (Heaps)
Tools	Google Colab / Jupyter / VS Code (Python 3); any C++ compiler for the C++ Corner
Files	Lab16_Instructor.ipynb , Lab16_Student.ipynb

Item	Details
Next lab	Course complete! See “Where to Go Next” in this manual.

1. Learning Objectives

By the end of this lab, you will be able to:

1. Explain graph terminology: **vertex**, **edge**, **directed**, **undirected**, **weighted**, **degree**, **path**, **cycle**, **connected**, **DAG**.
2. Represent graphs with an **adjacency matrix**, an **adjacency list** and an **edge list**, and compare them.
3. Implement **Breadth-First Search (BFS)** and find shortest paths in unweighted graphs.
4. Implement **Depth-First Search (DFS)** recursively and iteratively.
5. Find **connected components** and detect **cycles**.
6. Order tasks with **topological sort** (Kahn's algorithm).
7. Find shortest paths in weighted graphs with **Dijkstra's algorithm** using a heap.

Course Learning Outcomes (CLO) mapping

CLO	Description	Taxonomy level
CLO-1	Explain graph representations and terminology	Understand (C2)
CLO-2	Implement graph traversal and shortest-path algorithms	Apply (C3)
CLO-3	Model real problems as graphs and choose suitable algorithms	Analyse / Create (C4–C6)

2. Motivation: Everything Is Connected

Trees model hierarchies, but many relationships are not hierarchical. Cities are connected by roads in all directions; people follow each other on social media; web pages link to each other. A **graph** is the most general data structure for **connections**, and it brings together everything you learned this semester: lists, stacks, queues, hash tables and heaps.

A road network is a weighted graph (distances are illustrative)

```

Okara ---- 110 km ---- Lahore
 |      \                /      |
60 km   \ 125 km       45 km  50 km
 |      \                /      |
Sahiwal  Faisalabad ---- Sheikhpura
  
```

2.1 Graphs in Daily Life

Graph	Vertices	Edges
Google Maps / Careem	Places, intersections	Roads (weighted by distance or time)
Facebook / LinkedIn	People	Friendships / connections (undirected)
Instagram / X (Twitter)	Accounts	“Follows” (directed)
Your degree plan	Courses	Prerequisites (directed, no cycles)
Metro / bus routes (Lahore Orange Line)	Stations	Track segments
The Internet	Routers and computers	Cables and wireless links

2.2 Graphs in Industry

- **Navigation:** Google Maps, Waze and ride-hailing apps compute shortest and fastest routes with Dijkstra and A*.
- **Social networks:** friend suggestions (“people you may know”) explore friends-of-friends with BFS.
- **Search engines:** Google’s original **PageRank** ranks web pages using the link graph.
- **Build systems and package managers** (pip, npm, Make) order tasks with topological sort.
- **Telecom and electricity networks** plan cables with minimum spanning trees; **fraud detection** finds suspicious rings of accounts as cycles.

2.3 Graphs in Research

- **Graph Neural Networks (GNNs)** learn from molecules, social networks and road traffic.
- **Knowledge graphs** give AI assistants structured facts; vector search uses graph indexes (HNSW).
- **Deep learning frameworks** (PyTorch, TensorFlow) represent every model as a **computation graph** and run backpropagation in topological order.
- **Remote Sensing & GIS:** road extraction from satellite images, river networks and flood routing are graph problems.

- **Bioinformatics:** protein-interaction networks and genome assembly (de Bruijn graphs).

2.4 Why It Matters in 2026–2030

Graphs are the language of connected data

Recommendation systems, logistics, supply chains, AI agents planning steps, and neural networks themselves are all graphs. Graph algorithms are among the most valuable skills for AI, data science and systems engineering in the coming years, and graph problems are a favourite of technical interviews.

3. Part A: Graph Terminology

A **graph** $G = (V, E)$ is a set of **vertices** (nodes) V and a set of **edges** E connecting pairs of vertices.

Term	Meaning
Undirected graph	Edges have no direction ($A - B$ means both ways), e.g., friendship
Directed graph (digraph)	Edges have a direction ($A \rightarrow B$), e.g., following, prerequisites
Weighted graph	Each edge has a cost/weight, e.g., distance, time, price
Adjacent / neighbours	Two vertices joined by an edge
Degree	Number of edges at a vertex (in-degree / out-degree for directed graphs)
Path	A sequence of vertices connected by edges
Cycle	A path that starts and ends at the same vertex
Connected graph	There is a path between every pair of vertices (undirected)
Connected component	A maximal group of vertices that are connected to each other
DAG	Directed Acyclic Graph: directed, with no cycles (e.g., course prerequisites)
Tree	A connected undirected graph with no cycles (n vertices, $n - 1$ edges)



4. Part B: Representing a Graph

Example (undirected):

```

    0 --- 1
    |   / |
    |   / |
    2 --- 3      4 (isolated)
  
```

Edges: (0,1) (0,2) (1,2) (1,3) (2,3)

4.1 Adjacency Matrix

An $n \times n$ table where $M[u][v] = 1$ (or the weight) if there is an edge $u-v$. Checking an edge is $O(1)$, but it uses $O(V^2)$ memory even if there are few edges.

4.2 Adjacency List

For each vertex, store a **list of its neighbours** (in Python, a dictionary of lists). Memory is $O(V + E)$; this is the standard choice for real (sparse) graphs.

```

n = 5
edges = [(0, 1), (0, 2), (1, 2), (1, 3), (2, 3)]

matrix = [[0] * n for _ in range(n)]
adj = {v: [] for v in range(n)}
for u, v in edges:
    matrix[u][v] = matrix[v][u] = 1          # undirected: both directions
    adj[u].append(v)
    adj[v].append(u)

print("Adjacency matrix:")
for row in matrix:
    print(" ", row)
print("Adjacency list:", adj)
print("degree of 1:", len(adj[1]), "| edge 0-3?", matrix[0][3] == 1)
  
```

	Adjacency matrix	Adjacency list
Memory	$O(V^2)$	$O(V + E)$
Check if edge $u-v$ exists	$O(1)$	$O(\text{degree of } u)$
List all neighbours of u	$O(V)$	$O(\text{degree of } u)$
Best for	Dense graphs, small V	Sparse graphs (roads, social networks)

5. Part C: Breadth-First Search (BFS)

BFS explores the graph **level by level**: first the start vertex, then all its neighbours, then their neighbours, and so on. It uses a **queue** and a **visited** set. In an unweighted graph, BFS finds the **shortest path (fewest edges)** from the start to every vertex.

Pseudocode

```
BFS(G, s):  
    visited ← {s} ; queue ← [s] ; dist[s] ← 0  
    WHILE queue not empty:  
        u ← dequeue()  
        FOR each neighbour v of u:  
            IF v not in visited:  
                visited.add(v) ; dist[v] ← dist[u] + 1 ; parent[v] ← u  
                enqueue(v)
```

```

from collections import deque

city = {
    "Okara": ["Sahiwal", "Lahore", "Faisalabad"],
    "Sahiwal": ["Okara", "Multan"],
    "Lahore": ["Okara", "Sheikhupura", "Gujranwala"],
    "Faisalabad": ["Okara", "Sheikhupura"],
    "Sheikhupura": ["Lahore", "Faisalabad"],
    "Gujranwala": ["Lahore"],
    "Multan": ["Sahiwal"],
}

def bfs(graph, start):
    dist, parent = {start: 0}, {start: None}
    q = deque([start])
    while q:
        u = q.popleft()
        for v in graph[u]:
            if v not in dist:                # dist doubles as the visited
                dist[v] = dist[u] + 1
                parent[v] = u
                q.append(v)
    return dist, parent

def path_to(parent, target):
    path = []
    while target is not None:
        path.append(target)
        target = parent[target]
    return path[::-1]

dist, parent = bfs(city, "Multan")
print(dist)
print("Route Multan → Gujranwala:", " → ".join(path_to(parent,
"Gujranwala")))

```

6. Part D: Depth-First Search (DFS)

DFS goes **as deep as possible** along one branch before backtracking, like exploring a maze by always taking the first unexplored corridor. It uses **recursion** (the call stack) or an explicit **stack**.

```

def dfs_recursive(graph, u, visited=None, order=None):
    if visited is None:
        visited, order = set(), []
    visited.add(u)
    order.append(u)
    for v in graph[u]:
        if v not in visited:
            dfs_recursive(graph, v, visited, order)
    return order

def dfs_iterative(graph, start):
    visited, order, stack = set(), [], [start]
    while stack:
        u = stack.pop()
        if u in visited:
            continue
        visited.add(u)
        order.append(u)
        for v in reversed(graph[u]):           # reversed: same order as
recursive
            if v not in visited:
                stack.append(v)
    return order

print("DFS (recursive):", dfs_recursive(city, "Okara"))
print("DFS (stack)      :", dfs_iterative(city, "Okara"))

```

	BFS	DFS
Data structure	Queue	Stack / recursion
Order	Level by level	Deep first, then backtrack
Shortest path (unweighted)	Yes	No
Typical uses	Shortest routes, friend suggestions, broadcasting	Cycle detection, topological sort, mazes, components
Time	$O(V + E)$	$O(V + E)$

6.1 Connected Components and Cycle Detection

```
def components(graph):
    seen, groups = set(), []
    for v in graph:
        if v not in seen:
            group = dfs_recursive(graph, v, seen, [])
            groups.append(sorted(group))
    return groups

def has_cycle_undirected(graph):
    seen = set()
    def dfs(u, parent):
        seen.add(u)
        for v in graph[u]:
            if v not in seen:
                if dfs(v, u):
                    return True
            elif v != parent:
                # visited and not the edge we came from
                return True
        return False
    return any(dfs(v, None) for v in graph if v not in seen)

friends = {"Ali": ["Sara"], "Sara": ["Ali", "Hamza"], "Hamza": ["Sara"],
          "Zoya": ["Iqra"], "Iqra": ["Zoya"], "Omar": []}
print("friend groups:", components(friends))
print("cycle in city graph?", has_cycle_undirected(city), "| in friends?",
      has_cycle_undirected(friends))
```



7. Part E: Topological Sort (Ordering Tasks)

In a **DAG**, a topological order lists the vertices so that every edge $u \rightarrow v$ has u **before** v . Example: you must pass Programming Fundamentals before OOP, and OOP before DSA. **Kahn's algorithm**: repeatedly take a vertex with **in-degree 0** (no remaining prerequisites), output it, and remove its outgoing edges. If some vertices are never output, the graph has a **cycle** (impossible plan).

```
def topological_sort(graph):
    indegree = {v: 0 for v in graph}
    for u in graph:
        for v in graph[u]:
            indegree[v] += 1
    q = deque(sorted(v for v in graph if indegree[v] == 0))
    order = []
    while q:
        u = q.popleft()
        order.append(u)
        for v in graph[u]:
            indegree[v] -= 1
            if indegree[v] == 0:
                q.append(v)
    return order if len(order) == len(graph) else None    # None = cycle

courses = {
    "PF": ["OOP", "DLD"], "OOP": ["DSA"], "DLD": ["COAL"], "Discrete":
["DSA"],
    "DSA": ["Algorithms", "DB"], "COAL": ["OS"], "Algorithms": [], "DB": [],
    "OS": [],
}
print("Study order:", " → ".join(topological_sort(courses)))
```



8. Part F: Dijkstra's Shortest Path (Weighted Graphs)

When edges have weights (distances, times, costs), BFS no longer gives the shortest path. **Dijkstra's algorithm** always expands the **closest unexplored vertex**, which it gets from a **min-heap**. It works when all weights are **non-negative**. Time: $O((V + E) \log V)$.

Pseudocode

```
Dijkstra(G, s):
    dist[v] ← ∞ for all v ; dist[s] ← 0 ; heap ← [(0, s)]
    WHILE heap not empty:
        (d, u) ← pop smallest
        IF d > dist[u]: CONTINUE                // outdated entry
        FOR each (v, w) in neighbours of u:
            IF dist[u] + w < dist[v]:           // relaxation
                dist[v] ← dist[u] + w ; parent[v] ← u ; push (dist[v], v)
```

```

import heapq

roads = {
    # km (illustrative)
    "Okara": [("Lahore", 110), ("Sahiwal", 60), ("Faisalabad", 125)],
    "Lahore": [("Okara", 110), ("Sheikhupura", 45)],
    "Sahiwal": [("Okara", 60), ("Faisalabad", 115)],
    "Faisalabad": [("Okara", 125), ("Sahiwal", 115), ("Sheikhupura", 90)],
    "Sheikhupura": [("Lahore", 45), ("Faisalabad", 90)],
}

def dijkstra(graph, source):
    dist = {v: float("inf") for v in graph}
    parent = {source: None}
    dist[source] = 0
    heap = [(0, source)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in graph[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                parent[v] = u
                heapq.heappush(heap, (dist[v], v))
    return dist, parent

dist, parent = dijkstra(roads, "Sahiwal")
for city_name, d in sorted(dist.items(), key=lambda kv: kv[1]):
    print(f"{city_name:<12} {d:>4} km    via {' → '.join(path_to(parent, city_name))}")

```

Problem	Algorithm	Time
Visit all vertices / reachability	BFS or DFS	$O(V + E)$
Shortest path, unweighted	BFS	$O(V + E)$
Shortest path, non-negative weights	Dijkstra (heap)	$O((V + E) \log V)$
Shortest path with negative weights	Bellman-Ford	$O(V \cdot E)$
Ordering with dependencies	Topological sort	$O(V + E)$
Connect all vertices at minimum cost	Prim / Kruskal (MST)	$O(E \log V)$

9. C++ Corner

C++

```
#include <iostream>
#include <vector>
#include <queue>
#include <limits>
using namespace std;

int main() {
    int n = 5;
    vector<vector<pair<int,int>>> adj(n);          // adjacency list: (neighbour, weight)
    auto addEdge = [&](int u, int v, int w) { adj[u].push_back({v, w}); adj[v].push_back({u,
w}); };
    addEdge(0, 1, 4); addEdge(0, 2, 1); addEdge(2, 1, 2); addEdge(1, 3, 5); addEdge(3, 4,
3);

    // BFS (ignoring weights)
    vector<int> level(n, -1); queue<int> q; q.push(0); level[0] = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (auto [v, w] : adj[u]) if (level[v] == -1) { level[v] = level[u] + 1; q.push(v);
    }
    }

    // Dijkstra
    vector<int> dist(n, INT_MAX); dist[0] = 0;
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq; pq.push({0, 0});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;
        for (auto [v, w] : adj[u])
            if (d + w < dist[v]) { dist[v] = d + w; pq.push({dist[v], v}); }
    }
    for (int i = 0; i < n; i++) cout << i << ": level " << level[i] << ", dist " << dist[i]
<< endl;
}
```

10. Common Mistakes

Mistake	Effect	Fix
Forgetting the visited set	Infinite loop on cycles	Mark vertices visited when they are discovered
Marking visited when dequeued (BFS)	Vertices enqueued many times	Mark when enqueued
Adding only one direction for undirected edges	Missing paths	Add $u \rightarrow v$ and $v \rightarrow u$
Using BFS for weighted shortest paths	Wrong distances	Use Dijkstra
Dijkstra with negative weights	Wrong answers	Use Bellman-Ford

Mistake	Effect	Fix
Deep recursion in DFS on big graphs	RecursionError	Use the iterative stack version

11. Course Wrap-Up: What You Have Built

Congratulations! In 16 weeks you went from writing your first Python function to implementing the data structures that power operating systems, databases, search engines and AI systems.

Weeks	You learned	Real-world power
1–2	Python, Big-O	Judge the efficiency of any code, human or AI-written
3–7	Arrays, searching, sorting, recursion	Fast search and ordering of data
8–11	Linked lists, stacks, queues	Undo, browsers, schedulers, caches
12	Hashing	O(1) lookup behind dicts, caches and databases
13–15	Trees, BSTs, heaps	Indexes, ordered maps, priority systems
16	Graphs	Maps, social networks, dependencies, AI planning

11.1 Capstone Project (Choose One, Teams of 2–3)

1. **Smart Campus Navigator:** model University of Okara buildings as a weighted graph; shortest walking routes (Dijkstra), accessible routes (avoid stairs), and a list of nearest facilities (heap).
2. **Mini Search Engine:** index a folder of text documents with a hash map (word → documents), rank results by frequency with a heap, and support prefix auto-complete (sorted list + binary search, or a trie).
3. **Course Planner:** read the department's courses and prerequisites, detect impossible plans (cycles), produce a valid semester-by-semester plan (topological sort by levels).
4. **Hospital / Bank Simulator:** queues and priority queues with arrival times, multiple servers and statistics, plus an LRU cache for patient records.
5. **Your own idea** that uses at least **four** different data structures from this course (approved by your instructor).

Deliverables: GitHub repository with code and README, a 5-minute demo, and a short report explaining which data structures you chose and the **Big-O** of the main operations.

11.2 Where to Go Next

- **Algorithms course:** dynamic programming, greedy algorithms, minimum spanning trees, network flow, NP-completeness.
- **Advanced data structures:** tries, segment trees, Fenwick trees, union-find (disjoint sets), B-trees, skip lists, Bloom filters.
- **Practice:** aim for 150+ LeetCode problems (e.g., the “NeetCode 150” or “Blind 75” lists) before internship interviews.
- **Apply it:** NumPy/Pandas (arrays), databases (B-trees, hashing), PyTorch (computation graphs), and graph libraries like NetworkX.

12. Lab Tasks

Solve all tasks in `Lab16_Student.ipynb`. Each task has a **test cell**: when it prints  your solution is correct. For each task, **write pseudocode first**, then code, then test with your own inputs including edge cases. Difficulty:  Easy  Medium  Hard.

Task 1 Degrees and the Class Monitor

In a class of n students ($0 \dots n-1$), `trust = [(a, b), ...]` means student a trusts student b (directed). The **monitor** is trusted by everyone else and trusts nobody. Return the monitor's number or -1 .

Hint: compute in-degree and out-degree for every student. The notebook provides `build_graph(n, edges, directed)`.

```
n = 3, trust = [(0, 2), (1, 2)] → 2
```

Task 2 Is There a Route?

Given an undirected graph with n vertices and an edge list, return `True` if there is a path from `source` to `target`. Solve it with DFS using an explicit stack.

```
n = 6, edges = [(0,1),(0,2),(3,5),(5,4),(4,3)], 0 → 5 → False
```

Task 3 Shortest Route with BFS

Return the **actual shortest path** (list of vertices) from `start` to `goal` in an unweighted undirected graph, or `[]` if unreachable. Use BFS with a `parent` dictionary.

```
edges [(0,1),(1,2),(0,3),(3,4),(4,2)], 0 → 2 → [0, 1, 2]
```

Task 4 Count Flooded Regions (Connected Components on a Grid)

A flood map from satellite imagery is a grid of "W" (water) and "." (dry). Water cells connected up/down/left/right form one flooded region. Count the regions and return the **size of the largest** one as `(count, largest)`.

Use BFS from every unvisited water cell (each cell is a vertex; neighbours are adjacent cells).

```
["WW..", "W..W", "..WW", "...."] → (2, 3)
```

Task 5 Can I Finish My Degree? (Course Schedule)

There are n courses ($0 \dots n-1$) and prerequisite pairs (a, b) meaning **b must be taken before a**. Return a valid order of all courses, or $[]$ if it is impossible because of a cycle.

Use Kahn's topological sort. To make the answer deterministic, always take the **smallest** available course first (use a heap instead of a queue).

```
n = 4, prereqs = [(1, 0), (2, 0), (3, 1), (3, 2)] → [0, 1, 2, 3]
```

Task 6 Network Delay Time (Dijkstra)

A message is sent from server k in a network of n servers ($1 \dots n$). $times = [(u, v, w), \dots]$ means a directed link from u to v taking w milliseconds. Return the time for **all** servers to receive the message, or -1 if some cannot be reached.

Run Dijkstra from k ; the answer is the maximum shortest distance.

```
times = [(2,1,1), (2,3,1), (3,4,1)], n = 4, k = 2 → 2
```

Task 7 Spread of Crop Disease (Multi-Source BFS)

A field is a grid: 0 empty, 1 healthy plant, 2 infected plant. Every day, each infected plant infects its healthy neighbours (up/down/left/right). Return the number of days until no healthy plant remains, or -1 if some plants can never be infected.

Start BFS from **all** infected plants at once (put them all in the queue with day 0).

```
[[2,1,1],[1,1,0],[0,1,1]] → 4
```



13. LeetCode Practice

Solve these on **leetcode.com** and upload screenshots of the “Accepted” result to your GitHub **DSA-Labs/Lab16** folder.

#	Problem	Level	Concept practised
997	Find the Town Judge	Easy	Degrees (Task 1)
1971	Find if Path Exists in Graph	Easy	DFS / BFS (Task 2)
733	Flood Fill	Easy	Grid traversal
200	Number of Islands	Medium	Components (Task 4)
207	Course Schedule	Medium	Topological sort (Task 5)

#	Problem	Level	Concept practised
133	Clone Graph	Medium	BFS/DFS with a hash map
994	Rotting Oranges	Medium	Multi-source BFS (Task 7)
743	Network Delay Time	Medium	Dijkstra (Task 6)



14. Home Task

1. Finish all tasks and push `Lab16_Student.ipynb` to GitHub.
2. Implement cycle detection in a **directed** graph using DFS with three colours (white, grey, black).
3. Implement **Prim's** or **Kruskal's** algorithm to connect a set of villages with roads at minimum total cost.
4. Start your **Capstone Project** (see the Course Wrap-Up section): form a team, choose a topic, and create the GitHub repository with a README describing the data structures you plan to use.
5. Solve at least **3** LeetCode problems from the list.

? 15. Viva Questions

1. Define a graph. What is the difference between directed and undirected graphs?
2. What is a weighted graph? Give a real example.
3. Compare adjacency matrix and adjacency list representations.
4. Explain BFS. Which data structure does it use and why does it find shortest paths in unweighted graphs?
5. Explain DFS. How is it different from BFS?
6. What is the time complexity of BFS and DFS with an adjacency list?
7. What is a connected component? How do you count them?
8. What is a DAG? What is a topological order?
9. Explain Dijkstra's algorithm. Why does it need a priority queue?
10. Why does Dijkstra fail with negative edge weights?



16. Assessment Rubric

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Correctness	All tasks pass all tests	Most tasks correct	Some tasks correct	Mostly incorrect
Logic & pseudocode	Clear pseudocode and dry run for every task	Pseudocode for most tasks	Partial pseudocode	None
Code quality	Meaningful names, functions, comments	Mostly clean	Hard to read	Copied or unreadable
Complexity & edge cases	States Big-O and handles all edge cases	Some edge cases / Big-O	Only basic cases	Not considered
Viva	Explains every line confidently	Explains most parts	Basic understanding	Cannot explain



Lab Rules

- Bring your own laptop with Colab or Jupyter working.
- Arrive on time; late arrival may result in no attendance for the lab.
- Copying another student's code or pasting AI output without understanding it will be awarded zero. You must be able to explain every line in the viva.



17. References and Further Learning

- Graph algorithm animations: visualgo.net/en/graphds, visualgo.net/en/dfsdfs, visualgo.net/en/sssp
- NetworkX documentation: networkx.org
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, **Introduction to Algorithms** (CLRS), 4th ed., MIT Press, 2022.
- M. T. Goodrich, R. Tamassia, M. H. Goldwasser, **Data Structures and Algorithms in Python**, Wiley.
- MIT OpenCourseWare 6.006 Introduction to Algorithms; Harvard CS50x (free online courses).

- Visualise code step by step: **pythontutor.com**; animated data structures: **visualgo.net**.



About the Author

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

GitHub: github.com/RamzanPUCIT

LinkedIn: linkedin.com/in/ramzan-ai

ResearchGate: researchgate.net/profile/Muhammad-Ramzan-105

© 2026 Muhammad Ramzan. Prepared for the Department of Computer Science, University of Okara.